

Automatic Image Analysis Script

September 29, 2026

Contents

1	Introduction to Automatic Image Analysis	4
1.1	Lecture Aim and the Inverse Problem	4
1.2	From Pixels to Interpretation	6
1.3	Data Types and Interpretation Tasks	9
1.4	The Classical Processing Chain	10
1.5	Models, Data, and Evaluation	11
1.6	Topic Roadmap	14
2	Neural Networks and Backpropagation	14
2.1	From Perceptrons to Multilayer Networks	14
2.2	Losses, Optimization, and the Need for Gradients	17
2.3	Backpropagation	19
2.4	Automatic Differentiation and Practical Training	20
2.5	Generalization: Overfitting, Bias, Variance, and Regularization	21
3	Visual Object Models	22
3.1	Detecting Lines: The Hough Transform	23
3.2	Detecting Arbitrary Shapes: The Generalized Hough Transform	29
3.3	Where to Look: Salient Regions	32
3.4	What to Record: Local Features and Shape Descriptors	35
3.5	Putting It Together: The Implicit Shape Model	37
4	Bayesian Decision Theory and Probability Estimation	39
4.1	Probability Basics	39
4.2	Bayesian Decision Theory	41
4.3	Discriminant Functions	43
4.4	Non-Parametric Density Estimation	46
4.5	Parametric Estimation	48
4.6	Markov Random Fields	51
5	Convolutional Neural Networks	53
5.1	Why Images Need Convolution	53
5.2	The Convolutional Layer	54
5.3	Channels, Receptive Fields, and Why Convolution Helps	56
5.4	Convolutional Networks for Image Classification	59
5.5	Object Detection and Semantic Segmentation	62

6	Residual Networks	64
6.1	Initialization and the Variance of Activations	64
6.2	Why Deeper Plain Networks Fail	65
6.3	Residual Connections	66
6.4	Batch Normalization	68
6.5	Common Residual Architectures	69
7	Transformers	72
7.1	Tokens and Embeddings	72
7.2	Self-Attention	72
7.3	Transformer Layers	73
7.4	Encoders and Transfer Learning	74
7.5	Decoders and Cross-Attention	74
7.6	Vision and Multiscale Transformers	75
8	Generative Adversarial Networks	76
8.1	Learning Without Human Labels	76
8.2	Latent Variables and Generative Models	77
8.3	Adversarial Training	78
8.4	DCGAN and Training Difficulties	80
8.5	Wasserstein GANs	81
8.6	Improving Quality and Diversity	82
8.7	Conditional Generation	83
8.8	Image Translation	84
9	Diffusion Models	86
9.1	Destroying and Recreating Images	86
9.2	The Forward Noising Process	87
9.3	Training a Network to Predict the Noise	88
9.4	Generating Images by Reverse Denoising	90
9.5	Latent Diffusion and Stable Diffusion	91
9.6	Fine-Tuning with Low-Rank Adaptation	92
10	Weakly Supervised Learning	94
10.1	Three Types of Weak Supervision	94
10.2	Incomplete Supervision: Learning with Unlabeled Data	95
10.3	Inexact Supervision: Multi-Instance Learning	98
10.4	Inaccurate Supervision and Memorization	98
10.5	Early-Learning Regularization	100
10.6	Negative Learning for Noisy Labels	101
10.7	How Much Does Weak Supervision Really Gain?	104
11	Multimodal Language Models and LLM Applications	104
11.1	Decoder-Only Language Models and Prompts	105
11.2	PaLM-E: Injecting Observations into the Language Embedding Space	105
11.3	Encoders for Different Sensor Modalities	107
11.4	Training PaLM-E and Its Design Choices	108
11.5	Using an LLM Through an API: A Visual Task Agent	109
11.6	Trained Integration Versus API Composition	111
A	Edge Detection Essentials	112

B	Fourier Transform and Fourier Descriptors	112
B.1	The Discrete Fourier Transform	113
B.2	Fourier Descriptors	114
C	Classical Classifiers and Part-Based Detectors	115
D	Density Models and Calibration	116
E	Estimators, Loss Gradients, and Approximation	117
F	Graphical Models	118
G	Foundation Models and DINOv2	120
H	Contrastive and Energy-Based Self-Supervision	120

1 Introduction to Automatic Image Analysis

Automatic image analysis starts from the central difficulty of vision, then moves through the main processing levels, the recurring goals of the field, and the practical question of how to evaluate whether a vision system is actually reliable.

1.1 Lecture Aim and the Inverse Problem

Automatic image analysis is the problem of turning measurements into explanations. A digital image is easy to store: it is a grid of values. It is much harder to understand: we want to infer objects, surfaces, motion, geometry, lighting, and meaning from those values.

Computer vision is the computational study of how useful descriptions of the visual world can be recovered from images and video. The description may be semantic, such as “there is a pedestrian in front of the car”; geometric, such as “this surface is two meters away”; or operational, such as “reject this part because it contains a defect”. This definition is deliberately broader than object recognition. A vision system may label an image, reconstruct a three-dimensional scene, align several images, estimate motion, improve an image for a human observer, or supply measurements for a larger decision-making system.

Computer vision is therefore not just image enhancement. It is inference under uncertainty. The image is evidence; the scene that produced it is hidden.

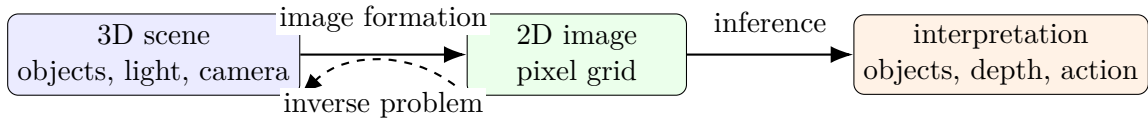


Figure 1: Computer vision tries to invert the image formation process: pixels are observed, but the scene explanation is hidden.

Human vision feels immediate, but this ease is misleading. A camera records a two-dimensional projection of a three-dimensional world. During that projection, many causes are mixed together: object shape, material, illumination, viewpoint, occlusion, sensor noise, and motion. Different scenes can produce very similar images, so there is usually no unique answer that can be read directly from the pixels.

This makes vision an inverse problem. The forward direction is comparatively clear: if we know the scene, the light, the camera, and the materials, we can model how an image is formed. The inverse direction asks us to go backward from image measurements to a plausible explanation of the world. That backward step requires assumptions, models, data, and algorithms.

A compact way to write the forward model is

$$I(x, y) = f(\Pi(X, Y, Z), L, M, C) + \varepsilon, \quad (1)$$

where $I(x, y)$ is the measured pixel value, Π is the camera projection from 3D to 2D, L describes illumination, M material and surface properties, C camera response, and ε sensor noise. Vision asks for some hidden scene state S from the image I :

$$\hat{S} = \arg \max_S p(S | I) = \arg \max_S p(I | S) p(S). \quad (2)$$

The likelihood $p(I | S)$ says which images a scene would probably produce. The prior $p(S)$ encodes what kinds of scenes are plausible before seeing the image.

The gap between the pixel array and the human interpretation is the essential problem. A person immediately infers objects, depth, material, intent, and context, while the measured image itself only contains sampled intensities or colors.

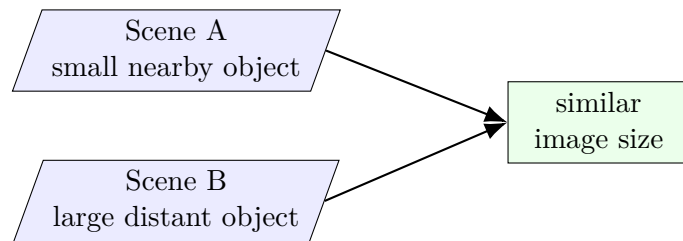


Figure 2: Ambiguity is unavoidable: different scenes can lead to similar image measurements. Priors and additional evidence help choose an explanation.

1.2 From Pixels to Interpretation

Automatic image analysis can be organized as a progression from low-level to high-level tasks.

- Low-level processing improves or transforms image measurements, for example acquisition, sampling, denoising, filtering, contrast changes, and edge extraction. Its typical mapping is image to image, or scene to image when the acquisition process is included.
- Mid-level analysis groups measurements into more meaningful structures, such as regions, contours, features, segments, motion fields, or depth estimates. It is often the bridge from image measurements to feature descriptions.
- High-level interpretation assigns semantic meaning, such as recognizing objects, estimating scene layout, detecting abnormalities, or tracking actions. Its typical mapping is from features or image evidence to object models, object detections, categories, or decisions.

These levels are useful for teaching, but real systems often mix them. A modern recognition model may learn low-level filters and high-level categories inside one network. A classical pipeline may use carefully designed features followed by statistical inference. In both cases, the goal is the same: reduce the ambiguity of image data until a useful decision or description becomes possible.

Level	Typical mapping	Typical question	Examples
Low-level	scene/image → image	What local measurements are reliable?	denoising, gradients, edges
Mid-level	image → features	Which measurements belong together?	regions, contours, segments, motion
High-level	features/image → objects	What does the image mean?	objects, scene layout, actions

Table 1: A teaching taxonomy for image-analysis tasks.

The same hierarchy can also be read as a contrast between bottom-up and top-down processing. A bottom-up system starts with the image data and builds larger hypotheses from local evidence: pixels become edges, edges become contours, contours become regions, and regions become object candidates. This is data-driven and attractive because the image itself constrains the result. Its weakness is that local evidence is often ambiguous or missing.

Top-down processing starts from a model, context, task goal, or prior expectation and asks how the image should be interpreted under that hypothesis. This is knowledge-driven: a rough object model can guide the search for parts, a scene context can make one interpretation more plausible than another, and a classifier can project learned category knowledge onto the image. Top-down processing is powerful because it uses what is already known, but it can also produce confident misinterpretations when the prior is wrong. Practical systems therefore often use a mixed mode: bottom-up evidence proposes or constrains hypotheses, while top-down models select, refine, or reject them.

Computer vision can be organized around three recurring goals: recognition, reconstruction, and reorganization.

Recognition. Recognition asks what is present and where it is. Examples include classification, detection, segmentation, reading text, identifying faces, or finding defects in industrial inspection.

Reconstruction. Reconstruction asks what spatial or physical structure produced the image. Examples include estimating depth, camera motion, three-dimensional shape, or the geometry of a scene from one or more images.

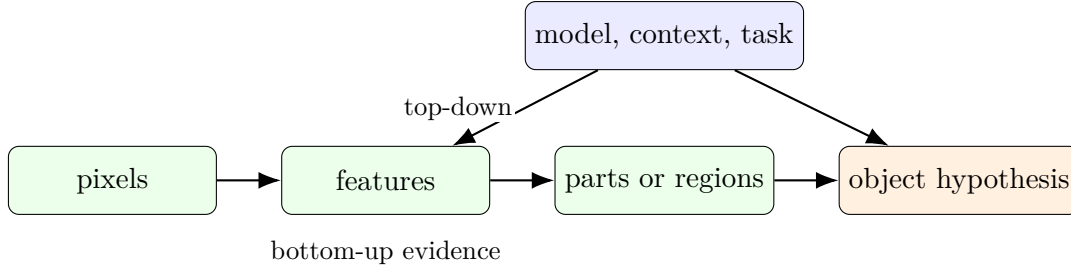


Figure 3: Bottom-up processing builds interpretations from image evidence. Top-down processing uses models and context to guide or correct the interpretation.

Reorganization. Reorganization changes the image representation so that later analysis becomes easier or the image becomes more useful to a person. Examples include registration, stitching, stabilization, denoising, super-resolution, and computational photography.

These goals are connected. Recognition benefits from good representations; reconstruction needs reliable correspondences; reorganization often depends on understanding motion, geometry, or image formation.

Area	Main transformation	Typical examples
Digital image processing	image $\rightarrow$ image	enhancement, restoration, filtering, compression
Photogrammetry and geometry	image(s) $\rightarrow$ 3D model	camera calibration, depth, surface reconstruction, mapping
Automatic image analysis	image $\rightarrow$ object model or detection	feature extraction, segmentation, object categorization, recognition

Table 2: Automatic image analysis is closest to the recognition branch of computer vision: it studies how image evidence becomes object-level descriptions, while still using processing and geometric ideas when they support that goal.

Another useful taxonomy separates the vocabulary of the field by the kind of question being asked. Some terms describe the *input* and its formation, some describe *representations* computed from the image, some describe *tasks*, and some describe the *knowledge* used to resolve ambiguity. Keeping these groups separate helps avoid a common beginner confusion: a feature, a model, a task, and an application are not the same kind of object.

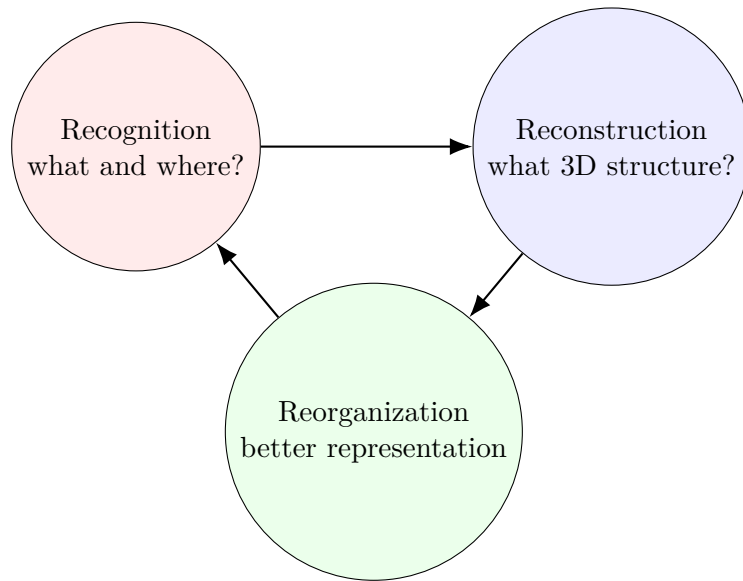


Figure 4: Recognition, reconstruction, and reorganization are not isolated topics; each goal supplies information that can support the others.

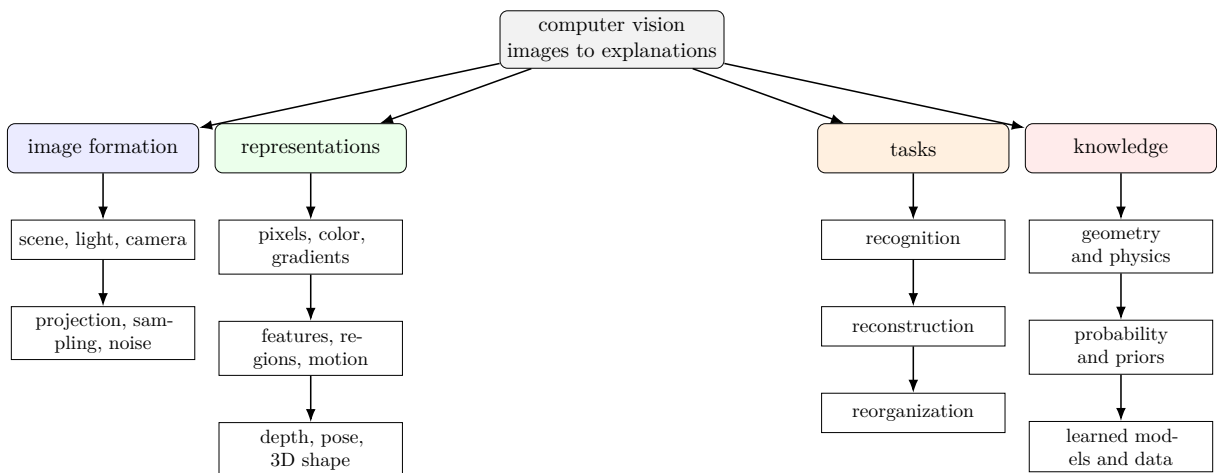


Figure 5: A compact taxonomy of introductory computer-vision terms. The field connects image formation, intermediate representations, task goals, and prior knowledge.

1.3 Data Types and Interpretation Tasks

Before listing concrete tasks, it helps to be precise about what kind of data an image is. We distinguish *structured* from *unstructured* data by whether the arrangement of the values carries meaning. A list of attributes such as (70 kg, 170 cm, 32 years) is unstructured: the entries can be permuted without changing what is described, as long as we remember which field is which. An image is structured, because the spatial layout of the pixels *is* the information. Shuffling the pixels destroys the content even though the multiset of intensity values is unchanged; sound and text are structured for the same reason. This is exactly why image analysis cannot treat a picture as an unordered bag of intensities: position and neighborhood are part of the signal, and the methods in later sections are built to exploit them.

High-level interpretation is not one task but a family of related questions, and it is worth naming them precisely because they differ in what the output looks like. *Image classification* assigns the whole image, or a given region, to one of a fixed set of categories, without saying where anything is. *Object detection* adds localization: it returns a set of objects, each with a position, usually a bounding box, together with a category, so it answers “what” and “where” at once. *Semantic segmentation* pushes localization to the pixel level, assigning a class label to every pixel and thereby partitioning the image into semantically uniform regions, though without separating individual instances of the same class. *Object recognition* is the broader goal of identifying which objects are present; in its narrow sense it means recognizing a specific identity or instance, such as a particular face or landmark, and in its broad sense it is used as an umbrella over classification and detection. Beyond these, interpretation also includes pose estimation, which recovers the spatial configuration of an object, action recognition, which labels an activity in a video, and attribute classification, which describes properties rather than category.

Task	Output	Question it answers
Image classification	one category for the whole image or region	what is in this image?
Object detection	set of located objects, e.g. boxes plus labels	what is present and where?
Semantic segmentation	a class label for every pixel	which class does each pixel belong to?
Object recognition	identity or category of the objects present	what object is this?
Pose estimation	spatial configuration, e.g. key-points or orientation	how is the object arranged?
Action recognition	activity label for a clip	what is happening over time?

Table 3: High-level interpretation tasks differ mainly in how much localization they require, from a single image-level label up to a decision at every pixel.

Classifying a real application means asking what the system has to output. A building access system that shows a face to a camera and matches it against a database of employee faces must decide *which* person this is: that is object recognition in the narrow, identity sense. It needs no pixel labels (segmentation), no free-text description (captioning), and localization is only a helper step. A system that analyzes basketball games, draws boxes around players, reads their names and numbers, and states whether a player is dribbling, throwing, or defending combines several tasks: the boxes and identities are object detection and recognition, and the verbs describe activities over time, which is action recognition. Exam questions of this kind reward naming *every* task whose output the system produces.

1.4 The Classical Processing Chain

All of these tasks share the same underlying processing chain. Image acquisition and preprocessing produce reliable low-level measurements, mid-level analysis groups them into features, and the high-level stage turns those into the task-specific output. In classical image analysis every stage is a separate, hand-designed module, and the order of the stages is fixed: Figure 6 shows it and Table 4 gives one example per stage.

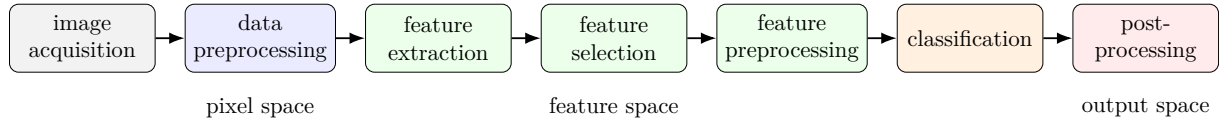


Figure 6: The classical image-analysis processing chain. Data preprocessing stays in pixel space, feature extraction maps pixels to a feature space, and only the classifier maps features to the output space of labels.

Stage	Mapping	Typical example
Image acquisition	scene $\rightarrow$ image	camera, sensor, sampling
Data preprocessing	image $\rightarrow$ image	denoising, histogram equalization to remove illumination differences
Feature extraction	image $\rightarrow$ feature vector	intensity histogram, HOG, bag of visual words
Feature selection	features $\rightarrow$ fewer features	keep only the informative dimensions
Feature preprocessing	features $\rightarrow$ features	normalization, whitening, dimensionality reduction
Classification	features $\rightarrow$ label	linear classifier, SVM, random forest, MLP
Postprocessing	labels $\rightarrow$ labels	smoothing label maps, non-maxima suppression of detections

Table 4: One example per stage of the processing chain. The stage is decided by what goes in and what comes out: an operation that returns an image is preprocessing, one that returns a feature vector is feature extraction.

The mapping column is the tool for sorting any method into its stage. Applying a transformation so that all images of a dataset have approximately the same histogram changes pixel values but still outputs images, so it is *data* preprocessing, not feature extraction. Nor is it *feature* preprocessing, although it “normalizes”: feature preprocessing acts on feature vectors after extraction, never on the images themselves. Feature extraction maps from pixel space into a *feature* space, not directly into the output space; turning features into labels is the classifier’s job.

Two classical features. The simplest image feature is the *intensity histogram*: count how many pixels take each grey value and normalize the counts. The histogram ignores where the pixels are, so rotating (or mirroring, or shuffling) the image leaves it unchanged; it is rotation invariant. It is *not* illumination invariant, because brightening the scene shifts all values and therefore the histogram. The *bag of visual words* (BoVW) transfers the same counting idea from grey values to image parts:

1. detect keypoints in all training images and describe each by a local descriptor;

2. cluster the descriptors (e.g. with k -means); each cluster center is a *visual word*, and together they form the vocabulary;
3. for an image, assign every keypoint to its nearest word and count how often each word occurs, which gives a word histogram;
4. compare this histogram with those of the database images, or feed it to a classifier.

Like the intensity histogram, BoVW records *which* parts occur, but not *where*: it describes no spatial relations between the patches. Part-based models that keep the geometry, such as the implicit shape model of Section 3, replace the plain count by votes with offset vectors.

From classifier to detector. A classifier trained on image patches of a fixed size answers only “is this patch an object?” Turning it into a detector requires four additions. First, a *sliding window* applies the classifier at every position of the image. Second, an *image pyramid* rescales the image to several resolutions so that objects of different sizes fit the fixed window. Third, each positive response gives a bounding box, whose size follows from the pyramid level and whose position follows from the window location. Fourth, neighboring windows and scales fire on the same object, so overlapping boxes are reduced to the strongest one by *non-maxima suppression*; without this step one object yields a cluster of detections.

Where deep learning fits. Deep networks do not remove the stages but merge them: a single differentiable model can learn preprocessing, feature extraction, feature selection, and classification jointly from data. A neural network can therefore play any of these roles alone, as a learned feature extractor whose activations feed another classifier, as a feature selector, or as the classifier on top of hand-crafted features.

1.5 Models, Data, and Evaluation

Because images are ambiguous, vision systems need prior knowledge. Sometimes that knowledge is explicit: camera geometry, physical image formation, smooth surfaces, rigid motion, or probabilistic noise models. Sometimes it is learned from data: neural networks discover features and decision rules from many examples.

Classical computer vision often emphasizes explicit models and engineered features. Modern deep learning often emphasizes learned representations and large datasets. The practical lesson is not that one replaces the other completely. Robust systems usually combine several kinds of knowledge: scientific models for image formation, statistical models for uncertainty, engineering choices for reliability, and data-driven learning for complex patterns that are hard to specify by hand.

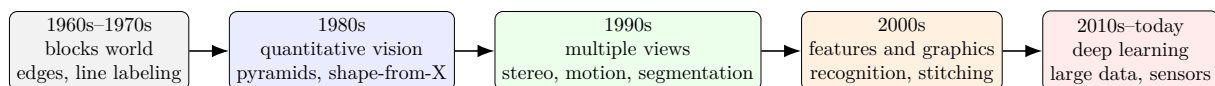


Figure 7: A brief historical timeline. The emphasis moved from hand-built geometric reasoning, through quantitative reconstruction and engineered features, toward large-scale learned representations and integrated sensing.

The timeline is not a story of one idea simply replacing another. Edge operators, camera geometry, variational estimation, probabilistic models, feature matching, and neural networks all remain useful. What changes over time is the balance between explicit modeling, available computation, available training data, and the kinds of applications that can be deployed reliably.

Computer vision is now a basic technology in many domains. It is more useful to name the application categories than to memorize a long list of products: consumer imaging and media,

Source of knowledge	What it contributes	Risk if missing
Physical model	Camera geometry, illumination, projection	Results may violate how images are formed
Statistical model	Noise, uncertainty, priors, inference	The system may overtrust ambiguous measurements
Engineering design	Robustness, speed, failure handling	The method may work only in demonstrations
Data-driven learning	Features and decision rules from examples	Complex visual variation may be underspecified

Table 5: A practical vision system usually combines several kinds of knowledge.

medicine and life sciences, industry and logistics, robotics and autonomous systems, remote sensing and mapping, document analysis, security and biometrics, and scientific measurement. The tasks differ, but the same conceptual questions recur: what image evidence is available, what assumptions make the interpretation possible, and what failure cases matter in the application context?

Application category	Typical vision output	Main reliability concern
Consumer and media	enhanced images, alignment, tracking	visual quality and robustness across devices
Medicine and life sciences	measurements, segmentations, suspected findings	uncertainty, bias, and clinical validation
Industry and logistics	defects, codes, counts, object poses	speed, repeatability, and rare failures
Robotics and autonomy	localization, obstacles, maps, actions	safety under changing environments
Remote sensing and mapping	land cover, 3D structure, change detection	scale, resolution, and registration accuracy
Documents and biometrics	text, identity cues, authenticity signals	privacy, fairness, and adversarial misuse

Table 6: Categorical application areas are easier to study than isolated product examples because they expose recurring reliability questions.

These applications also show why evaluation matters. A result that looks plausible on one example is not enough. Vision algorithms must be tested against noise, changing illumination, clutter, unusual viewpoints, occlusion, and data that differ from the training or development examples.

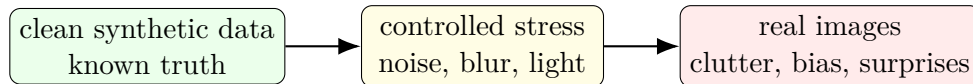


Figure 8: Evaluation should move from controlled correctness toward real-world robustness.

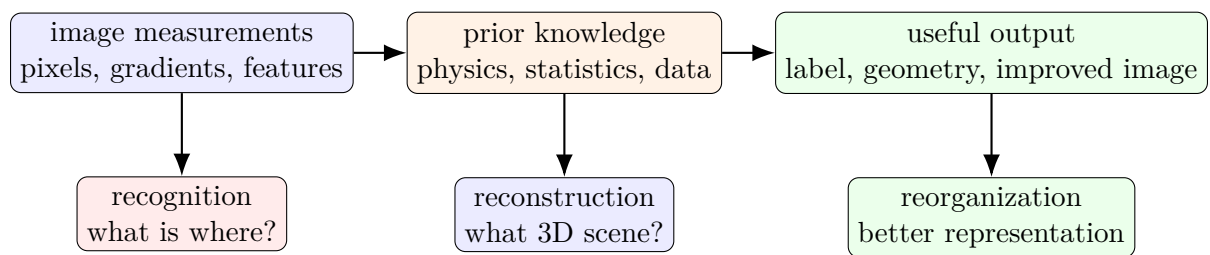


Figure 9: A compact summary of the introductory framing: measurements become useful outputs only after they are combined with assumptions, models, and data.

1.6 Topic Roadmap

Automatic image analysis is narrower than the whole field of computer vision. It uses image processing tools and needs some geometric and probabilistic reasoning, but the main line is automatic interpretation: how images are transformed into features, segments, object models, detections, categories, and evaluated decisions.

The foundations are image representation, filtering, features, transformations, and the basic language of image formation. These topics supply the bottom-up evidence used later. Segmentation, clustering, and feature extraction provide mid-level descriptions that make object-level reasoning possible. Learning-based methods, especially neural networks, show how useful representations and classifiers can be estimated from data. Object models, probabilistic estimation, and higher-level recognition tasks connect these pieces; reconstruction appears where it helps explain image formation, geometry, or the limits of recognition.

Three questions recur throughout automatic image analysis:

1. What information is actually present in the image measurements?
2. What assumptions are needed to infer the hidden scene or decision?
3. How can we test whether the algorithm works beyond the example in front of us?

Automatic image analysis starts with pixels but does not end there. Its real task is to connect images to explanations: what is in the scene, how it is arranged, how it changes, and what action or decision should follow.

2 Neural Networks and Backpropagation

Neural networks are one way to replace manually designed decision rules by learned functions. For image analysis, the input may be an image, a patch, or a feature vector, and the output may be a class score, a probability, a segmentation label, or another task-specific prediction. The central idea is simple: compose many parameterized transformations and fit their parameters from examples.

Szeliski's treatment of deep learning, especially the part on deep neural networks and backpropagation, is a useful companion for this topic. The focus here is narrower: understand the computational structure of a feed-forward network, why backpropagation is the efficient way to compute the gradients needed for training, and how to tell whether a trained network will generalize.

2.1 From Perceptrons to Multilayer Networks

A perceptron is a linear classifier. Given an input vector x , weights w , and bias b , it predicts a binary label by thresholding an affine score:

$$f(x) = \begin{cases} 1, & w^\top x + b > 0, \\ -1, & \text{otherwise.} \end{cases} \quad (3)$$

The decision boundary is the hyperplane $w^\top x + b = 0$. This makes the perceptron easy to interpret, but also limited: if the classes cannot be separated by a linear boundary, the standard perceptron cannot solve the problem. The XOR problem is the usual minimal example.

During perceptron training, an incorrectly predicted example changes the weights in the direction of the input:

$$w_i^{t+1} = w_i^t + \eta (y - f(x)) x_i, \quad (4)$$

where η is the learning rate. The rule is useful historically because it already shows the pattern of learning from prediction error, but modern neural networks need a smoother and more general gradient-based view.

Logistic regression is the smooth relative of the perceptron and a useful reference point. For C classes and a D -dimensional input it computes one affine score per class, $z = Wx + b$ with $W \in \mathbb{R}^{C \times D}$ and $b \in \mathbb{R}^C$, and turns the scores into probabilities with the softmax function (introduced below). Its parameter count, $C(D + 1)$, therefore depends on both the number of classes and the input dimensionality (for two classes one score suffices, giving $D + 1$). Despite the nonlinear softmax, the predicted class is the largest affine score, and the boundary between two classes, where $z_i = z_j$, is a hyperplane: logistic regression is a *linear* classifier, exactly like linear regression used for classification.

A multilayer perceptron (MLP) overcomes the purely linear limitation by stacking several affine maps and nonlinear activation functions. For layer l ,

$$h^l = W^l x^l + b^l, \quad y^l = \phi(h^l), \quad x^{l+1} = y^l. \quad (5)$$

Without the nonlinearity ϕ , stacking layers would still collapse into one linear transformation. The nonlinearities make it possible to build complex decision boundaries from many simple units. The entries of W^l are the *weights*, one per connection, and b^l is the *bias*. Network diagrams often draw the bias as an extra neuron with constant output 1 whose outgoing weights are the bias values; both views compute the same affine map.

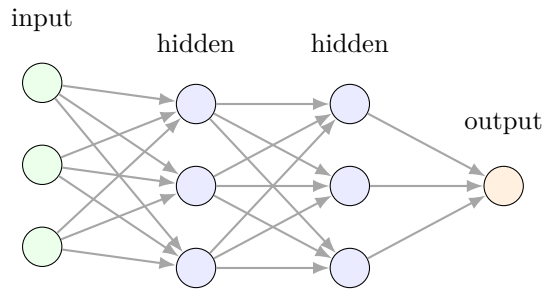


Figure 10: A feed-forward network composes affine transformations and nonlinearities. Training changes the weights on the connections.

Beyond being nonlinear, a useful transfer function is typically monotonic and smooth, so that it has a well-defined derivative everywhere; the classical logistic sigmoid and hyperbolic tangent are additionally bounded, which keeps activations in a fixed range. Common activation functions include the logistic sigmoid, hyperbolic tangent, and rectified linear unit:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad (6)$$

$$\tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z). \quad (7)$$

For ReLU, the derivative is 1 for positive inputs and 0 for negative inputs; at zero one chooses a convenient subgradient in implementations. This simple derivative is one reason ReLU-style activations are computationally attractive, and ReLU is the standard choice in convolutional networks.

Worked forward pass. Computing a small network by hand is the best check that the notation is understood. Take two inputs, one hidden layer with two ReLU units, and two ReLU output units, with the weights of Table 7. The input is $x = (2, 1)$ and the target is $y = (0, 1)$. Layer by layer, first form the weighted sum and then apply ReLU:

$$\begin{aligned} h_1 &= \max(0, 1 \cdot 2 - 1 \cdot 1 + 0) = 1, & h_2 &= \max(0, -1 \cdot 2 + 1 \cdot 1 + 0.5) = \max(0, -0.5) = 0, \\ o_1 &= \max(0, 1 \cdot 1 + 1 \cdot 0 - 1) = 0, & o_2 &= \max(0, 2 \cdot 1 - 1 \cdot 0 + 0) = 2. \end{aligned}$$

Unit	weight from 1st input	weight from 2nd input	bias
hidden h_1	1	-1	0
hidden h_2	-1	1	0.5
output o_1 (inputs h_1, h_2)	1	1	-1
output o_2 (inputs h_1, h_2)	2	-1	0

Table 7: Weights of a 2-2-2 ReLU network. Each row lists the weights of the connections entering one unit; the bias is the weight from the constant-1 neuron.

With the squared-error criterion $e = \frac{1}{2}\|o - y\|_2^2$ the error is $e = \frac{1}{2}((0 - 0)^2 + (2 - 1)^2) = \frac{1}{2}$. Two details are easy to get wrong under time pressure: ReLU clips negative sums to zero *before* they enter the next layer, and the factor $\frac{1}{2}$ belongs to the whole sum.

2.2 Losses, Optimization, and the Need for Gradients

Training starts with examples (x_i, y_i) and a network f_θ with parameters θ , meaning all weights and biases. A loss function measures how bad a prediction is. Over a dataset, the empirical objective is

$$L(\theta) = \sum_{i=1}^N \ell(f_\theta(x_i), y_i). \quad (8)$$

For regression, ℓ is typically the squared error $\frac{1}{2}\|f_\theta(x) - y\|^2$. For classification, the network outputs one score (logit) z_c per class, the softmax turns the scores into probabilities, and the loss is the cross-entropy, the negative log-probability of the correct class y :

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}, \quad \ell_{\text{CE}} = -\log p_y. \quad (9)$$

The loss is what the optimizer sees, so it must match the task; accuracy itself is not differentiable and is only used to evaluate the result. How the gradients of squared and absolute error differ, and what the universal approximation theorem does and does not promise, is summarized in Appendix E.

Even when the loss is convex, as for logistic regression, there is in general no closed-form solution for the best parameters, so the minimum has to be found iteratively. Gradient descent is therefore not a tool reserved for non-convex problems; convexity only guarantees that it will not get stuck in a worse local minimum. For deep networks the loss is non-convex, and (stochastic) gradient descent gives no guarantee of reaching the optimal solution.

Gradient descent steps against the gradient with learning rate η . The three classical variants differ only in how many examples enter one update:

$$\text{online: } \Delta w = -\eta \frac{\partial e_n}{\partial w} \quad \text{after every single example } n, \quad (10)$$

$$\text{mini-batch: } \Delta w = -\eta \sum_{n \in \mathcal{B}_t} \frac{\partial e_n}{\partial w} \quad \text{after a small batch } \mathcal{B}_t, \quad (11)$$

$$\text{offline (batch): } \Delta w = -\eta \sum_{n=1}^N \frac{\partial e_n}{\partial w} \quad \text{after a pass over all } N \text{ examples.} \quad (12)$$

Offline learning follows the exact gradient of the full objective but updates rarely; online learning updates often but with a noisy direction. Stochastic gradient descent in modern practice is the mini-batch variant. It is used for two reasons: the gradient of a random mini-batch points, on average, in the same direction as the full gradient, and the cost of one update depends only on the batch size, not on the size of the dataset. It does *not* need fewer steps than full gradient descent (its steps are noisier, so it usually needs more of them, each far cheaper), and it does not by itself reach a lower training error. Two common additions modify the update. *Momentum* (inertia) keeps part of the previous step, so that consistent directions accelerate and oscillations cancel,

$$\Delta w(t+1) = -\eta \frac{\partial e}{\partial w} + \alpha \Delta w(t), \quad 0 \leq \alpha < 1; \quad (13)$$

weight decay adds a penalty $\frac{\lambda}{2}\|w\|^2$ to the loss, which contributes $-\eta\lambda w$ to every step and pulls unused weights toward zero.

Whatever the variant, every step needs the partial derivatives of the loss with respect to all parameters. The practical difficulty is not writing the update down; it is computing these derivatives for millions of parameters quickly and without duplicating work. Backpropagation solves exactly that problem.

Component	Role in the network	Training question
Weights and biases	Store the adjustable model parameters	How should each parameter change to reduce the loss?
Activations	Introduce nonlinear transformations between layers	Which units are active and how do they transmit gradients?
Loss function	Converts predictions and targets into one scalar	What does “better” mean for this task?
Optimizer	Turns gradients into parameter updates	How large and in which direction should the next step be?

Table 8: Training a neural network requires both a model architecture and a procedure for turning prediction errors into parameter updates.

2.3 Backpropagation

Backpropagation is an efficient application of the chain rule to a computational graph. The forward pass computes the network output and stores intermediate quantities. The backward pass starts from the derivative of the loss with respect to the output and propagates sensitivity information back through the same operations in reverse order.

A graph worked by hand. The mechanism is clearest on a tiny scalar graph, $f = (x + y) \cdot z$ with $x = -2$, $y = 5$, $z = -4$ (Figure 11). The forward pass computes and stores the intermediate value $q = x + y = 3$ and the output $f = qz = -12$. The backward pass starts with $\partial f / \partial f = 1$ and applies one local rule per node. The multiplication node sends to each input the upstream gradient times the *other* input: $\partial f / \partial z = q = 3$ and $\partial f / \partial q = z = -4$. The addition node has local derivative 1 for both inputs, so it passes the upstream gradient unchanged to each of them: $\partial f / \partial x = \partial f / \partial y = -4$.

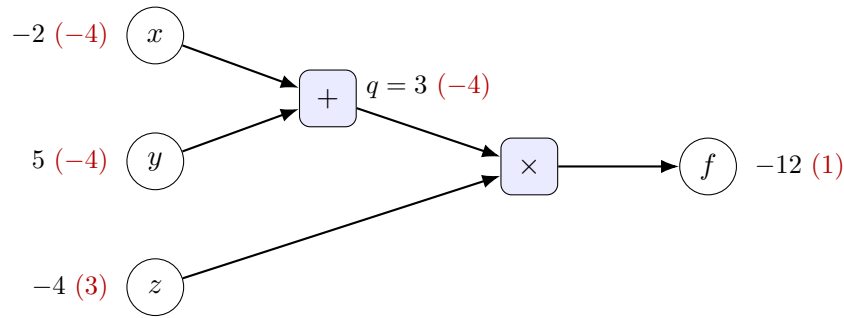


Figure 11: Backpropagation on $f = (x + y)z$. Black numbers are forward values, red numbers in parentheses are the gradients $\partial f / \partial (\cdot)$ from the backward pass. The multiplication node swaps its inputs; the addition node copies the upstream gradient to both inputs.

These local rules are worth remembering as patterns, summarized in Table 9. In particular, an addition node does *not* send its inputs backward; it distributes the upstream gradient.

Node	Local derivative	Backward behavior
Addition $a + b$	1 for each input	distributes: each input receives the upstream gradient unchanged
Multiplication ab	b for a , a for b	swaps: each input receives the upstream gradient times the other input's forward value
Maximum $\max(a, b)$	1 for the larger, 0 else	routes: only the larger input receives the gradient (ReLU is $\max(0, z)$)

Table 9: Backward rules of the elementary nodes. The multiplication rule is why forward values must be stored until the backward pass.

The layer view. It is helpful to name the quantities at one layer before writing the gradient. Let h_k be the activation vector that enters layer k . The trainable parameters of this layer are a weight matrix Ω_k and a bias vector β_k . The affine part of the layer first forms the pre-activation vector

$$f_k = \beta_k + \Omega_k h_k, \quad (14)$$

and a nonlinearity would then produce the next activation, for example $h_{k+1} = \phi(f_k)$. With this notation, the gradient with respect to the weight matrix has the structure

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \ell_i}{\partial f_k} h_k^\top. \quad (15)$$

This is the multiplication rule at matrix scale: the effect of a weight depends both on the upstream error signal and on the activation feeding into that weight. If the previous activation is doubled, the same small weight change has twice the local effect.

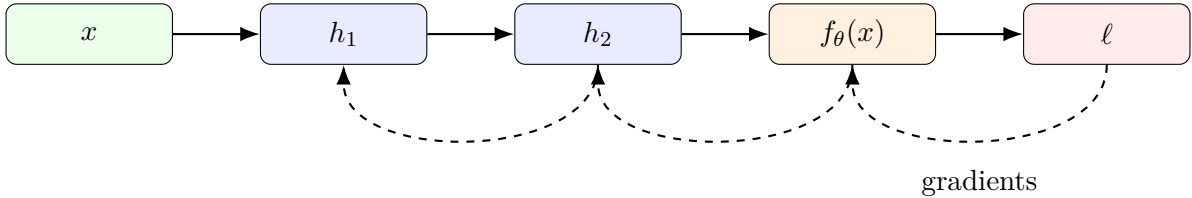


Figure 12: The forward pass evaluates the network from left to right. The backward pass propagates derivatives from the loss back through the stored intermediate quantities.

For ReLU hidden units, the backward recurrence includes the pointwise derivative of the activation:

$$\frac{\partial \ell_i}{\partial f_{k-1}} = \mathbb{I}[f_{k-1} > 0] \odot \left(\Omega_k^\top \frac{\partial \ell_i}{\partial f_k} \right), \quad (16)$$

where $\odot$ denotes elementwise multiplication. The indicator gates the gradient: inactive ReLU units do not pass local gradient information backward.

What backpropagation costs and what it does not do. The important point is not to memorize every indexed expression, but to know what must be stored and what is reused. One parameter update needs exactly *one* forward pass (to compute and store all intermediate values) and *one* backward pass (to reuse them); every derivative is computed once and shared by all parameters upstream of it. Because the forward values of every node are kept until the backward pass, the memory consumption grows with the combined output dimensionality of all nodes in the graph (the stored activations, times the batch size), not merely with the number of trainable parameters. Backpropagation is only a way to *compute* the gradient exactly and efficiently. It does not change which gradient is computed, so it neither reduces the noise of stochastic gradient descent (that comes from using mini-batches) nor guarantees that the optimizer reaches the optimal solution.

2.4 Automatic Differentiation and Practical Training

Modern deep-learning libraries implement backpropagation through automatic differentiation. A model is represented as an acyclic graph of operations. Each operation knows how to compute its output in the forward direction and how to propagate derivatives in the reverse direction, exactly as the nodes of Table 9. The user specifies the model, the loss, and the optimizer; the framework records the graph and applies the chain rule. Branches are allowed as long as the graph stays acyclic.

A typical training loop has the following structure:

1. initialize the network parameters;
2. choose a loss function and optimizer;
3. load a batch of examples;

4. run the forward pass and compute the loss;
5. run the backward pass to compute gradients;
6. update the parameters and repeat.

This is the conceptual core behind PyTorch-style code: zero old gradients, evaluate the model, call backward on the loss, and let the optimizer step. Poor initialization can make training slow or unstable. Too large a learning rate can overshoot; too small a learning rate can make progress painfully slow. On a non-convex loss, gradient descent settles in the minimum whose basin contains the starting point, so a different initialization can end in a local instead of the global minimum. Momentum tends to carry the parameters *through* shallow minima; too much of it shows as overshooting and wide oscillation, not as clean convergence into a nearby basin.

2.5 Generalization: Overfitting, Bias, Variance, and Regularization

A network that fits its training data perfectly may still be useless. The goal is low error on *new* data, and the training error cannot measure that: a flexible model can memorize the training set, noise included, which is called *overfitting*. The opposite failure, *underfitting*, occurs when the model is not complex enough to match the data and performs poorly even on the training set. Figure 13 shows the typical picture: as model complexity grows, the training error keeps falling, while the error on held-out data first falls and then rises again once the model starts fitting noise.

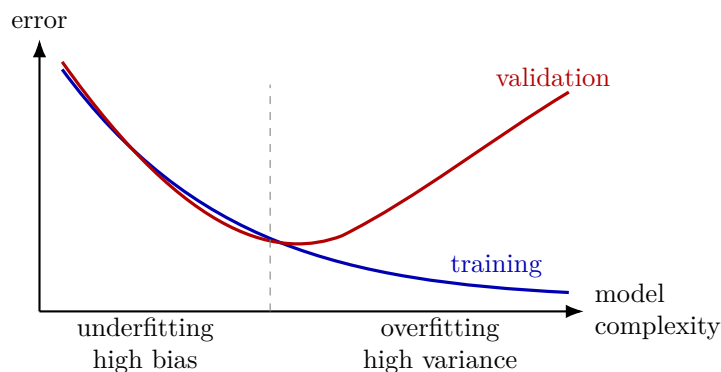


Figure 13: Training and validation error versus model complexity. The dashed line marks the complexity with the lowest validation error; to its left the model underfits, to its right it overfits.

The *bias–variance* vocabulary names the two failures. A model with high *bias* does not match the training data closely; it underfits. *Variance* describes how much the fitted model changes when it is trained on different portions of the data. It can be measured exactly that way: fit the model to several subsets and compare the results. A high-variance model follows the particular sample it saw and therefore overfits.

Training, validation, and test data. Because the training error is optimistically biased, the available data are split into three parts with different roles. The *training set* fits the parameters. The *validation set* is used for every decision about the model: hyperparameters such as learning rate, weight decay, or depth, and when to stop training. The degree of overfitting is read from the gap between training and validation error, never from the training data alone. The *test set* is touched once, at the end, for an unbiased estimate; once it influences a decision, it is no longer a test set. This separation is needed for every model, convex or not, because convexity concerns only how well the optimizer finds the minimum of the training loss, not how well that minimum generalizes. It also fixes the reading of small errors: a small training error only suggests that the model has sufficient complexity, while a small *validation* error suggests good performance.

Regularization. Regularization is any change that reduces overfitting by encoding a preference for simpler or more robust solutions. The standard tools are:

- *Weight decay* (L_2 penalty): add $\lambda\|W\|_2^2$ to the loss so that large weights are discouraged.
- *Dropout*: randomly set a fraction of activations to zero during training so that the network cannot rely on any single path; at test time all units are used.
- *Data augmentation*: enlarge the training set with transformed copies (crops, flips, small rotations, color jitter). This has a regularizing effect, because the model must produce the same answer for all variants; the transformations must respect the semantics of the task.
- *Early stopping*: monitor the validation loss after every epoch, stop when it has not improved for a fixed number p of epochs (patience), and keep the checkpoint with the best validation loss, not the last one.
- *Batch normalization*: normalize activations over the mini-batch and learn a scale and shift; mainly a stabilizer of optimization with a mild regularizing side effect (Section 6).

More data and a smaller model act in the same direction: both fight overfitting. Neither helps an underfitting model, which already fails on the data it has; only more capacity does, that is, a wider (more units per layer) or deeper (more layers) network, or longer training. Batch normalization stabilizes optimization but adds no capacity. Table 10 turns these ideas into a diagnosis checklist.

Symptom	Meaning	Typical response
High training error	The model or optimization setup is too weak (high bias)	increase capacity, train longer, adjust learning rate
Low training error but high validation error	The model overfits the training set (high variance)	use regularization, more data, or early stopping
Unstable loss	Updates are too aggressive or scales are poor	lower learning rate, improve initialization, normalize inputs

Table 10: Most neural-network training failures can be diagnosed by comparing training behavior with validation behavior.

3 Visual Object Models

An object model is a computational answer to a deceptively simple question: what does an object look like, expressed in a form an algorithm can search for? A person has no trouble saying that a coffee mug has a rounded body and a handle, but a detector needs something concrete: a parametric shape, a set of edge displacements, a collection of characteristic image patches, or a learned feature map. The shape of that representation decides which transformations the detector can tolerate (translation, scale, rotation, partial occlusion) and how the search through the image is organized.

This section develops object models through a single recurring idea that ties its parts together:

Local image evidence accumulates into global object hypotheses by voting.

Each fragment of the image (an edge, a gradient, a patch) is weak and ambiguous on its own, but it is consistent with only a limited set of global explanations. If every fragment casts a vote for the explanations it supports, the correct hypothesis collects many votes and stands out as a

peak in a parameter space, while accidental coincidences stay near the noise floor. We follow this idea through increasingly flexible models: edges first vote for *lines* (Hough transform), then for the center of a *known shape* (generalized Hough transform), and finally learned appearance *parts* vote for the center of an *object category* (implicit shape model). Saliency and descriptors sit in between, deciding *which* local evidence is worth trusting in the first place.

3.1 Detecting Lines: The Hough Transform

The simplest non-trivial object model is a straight line, possibly broken by noise or occlusion into many short edge fragments. The difficulty is that the fragments are scattered across the image, and we do not know in advance how many lines there are or where they lie. Testing every possible line against every group of pixels would be hopeless. The Hough transform sidesteps this by changing coordinates: instead of grouping pixels in the image, it lets every pixel vote in the space of all possible *lines*, and then simply looks for the lines that received many votes.

To make “the space of all lines” concrete we need a way to label each line by a few numbers. The familiar slope–intercept form $y = mx + b$ is a poor choice: a vertical line has infinite slope, so m and b cannot represent it. The *Hesse normal form* sidesteps this by describing a line through the one point it can never help but single out — the point on the line closest to the origin.

Every line has a unique closest point to the origin: the foot of the perpendicular dropped from the origin onto the line. We can reach that foot by starting at the origin and walking a fixed distance $r \geq 0$ in a single direction. Call θ the angle of that direction, so the unit vector pointing from the origin straight at the line is

$$\hat{\mathbf{n}} = (\cos \theta, \sin \theta), \quad (17)$$

so the foot of the perpendicular is the point

$$P_0 = r \hat{\mathbf{n}} = (r \cos \theta, r \sin \theta). \quad (18)$$

The vector from the origin to P_0 is r times $\hat{\mathbf{n}}$, so it is *normal* (perpendicular) to the line, because the shortest path from a point to a line always meets it at a right angle. The pair (r, θ) then pins the line down completely: θ says which way the line faces and r says how far out it lies.

To turn this picture into an equation, take any point $P = (x, y)$ on the line and look at the vector $P - P_0$ that runs from the foot to it. Because both P and P_0 lie on the line, this vector points *along* the line; and the line is perpendicular to the origin-to-foot vector P_0 . Orthogonality of $P - P_0$ and P_0 means their dot product vanishes,

$$(P - P_0) \cdot P_0 = 0 \quad \Longleftrightarrow \quad P \cdot P_0 = P_0 \cdot P_0. \quad (19)$$

Both sides are now immediate. On the left, $P \cdot P_0 = x(r \cos \theta) + y(r \sin \theta) = r(x \cos \theta + y \sin \theta)$; on the right, $P_0 \cdot P_0 = r^2(\cos^2 \theta + \sin^2 \theta) = r^2$, using $\cos^2 \theta + \sin^2 \theta = 1$. Equating the two gives $r(x \cos \theta + y \sin \theta) = r^2$, and cancelling one factor of r (every line we care about has $r > 0$) leaves the Hesse normal form of the line,

$$x \cos \theta + y \sin \theta = r. \quad (20)$$

Figure 14 shows the whole construction. What looked like an arbitrary combination of a cosine and a sine is really one clean geometric statement: from every point P on the line, the step $P - P_0$ to the foot leaves at a right angle to the origin-to-foot vector P_0 .

Here x and y are pixel coordinates, $r \geq 0$ is measured in pixels, and θ ranges over $[0, \pi)$. A quick sanity check fixes the meaning of the two numbers. A horizontal line at height $y = c$ has a vertical perpendicular, so $\theta = 90^\circ$ and Equation (20) reduces to $r = c$; a vertical line $x = d$ has $\theta = 0^\circ$ and $r = d$; a line passing through the origin has $r = 0$ at every one of its points, whatever

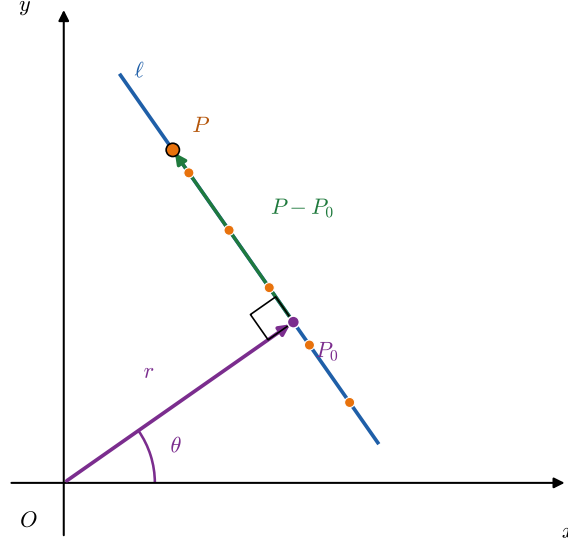


Figure 14: The Hesse normal form of a line. The foot of the perpendicular from the origin O is $P_0 = (r \cos \theta, r \sin \theta)$, reached by walking a distance r at angle θ . For any point P on the line ℓ , the vector $P - P_0$ runs along the line and is therefore orthogonal to P_0 ; the condition $(P - P_0) \cdot P_0 = 0$ is exactly $x \cos \theta + y \sin \theta = r$, which every collinear edge point (orange) satisfies with the same (r, θ) .

its angle. No line, not even a vertical one, makes the parameters blow up. That is exactly the robustness the slope–intercept form lacks.

The decisive property of Equation (20) is an *invariance*: every pixel that lies on one particular line produces the *same* pair (r, θ) , because r and θ describe the line, not the individual point. A whole line in the image therefore collapses to a single point in the (r, θ) *parameter space*. Now read the same equation the other way around. Fix a single image point (x_0, y_0) and let θ vary; the equation turns into a function of θ alone,

$$r(\theta) = x_0 \cos \theta + y_0 \sin \theta, \quad (21)$$

a fixed weighted sum of a cosine and a sine. Any such combination collapses to a single shifted sine of the same frequency, and it is worth seeing exactly why, because the shift is where the phase φ comes from. Write down the shifted sine we are aiming for and expand it with the angle-addition identity $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$:

$$R \sin(\theta + \varphi) = \underbrace{R \cos \varphi}_{\text{weight of } \sin \theta} \sin \theta + \underbrace{R \sin \varphi}_{\text{weight of } \cos \theta} \cos \theta. \quad (22)$$

For this to equal $x_0 \cos \theta + y_0 \sin \theta$ for *every* θ , the two expressions must have the same weight on $\cos \theta$ and the same weight on $\sin \theta$ separately. Matching them term by term gives

$$x_0 = R \sin \varphi, \quad y_0 = R \cos \varphi. \quad (23)$$

These two relations determine R and φ from the point alone. Squaring and adding them cancels φ (via $\sin^2 + \cos^2 = 1$) and leaves the amplitude,

$$R = \sqrt{x_0^2 + y_0^2}, \quad (24)$$

which is simply the point’s distance from the origin; dividing them instead cancels R and fixes the phase,

$$\varphi = \text{atan2}(x_0, y_0). \quad (25)$$

Here `atan2` is the two-argument arctangent, which recovers the angle in the correct quadrant from the signed pair (x_0, y_0) . So φ is not a new free parameter that appears from nowhere: it is the phase *forced* on the sinusoid by where the point sits, namely the angle of (x_0, y_0) measured from the y -axis (that is what Equation (23) says, since $(x_0, y_0) = (R \sin \varphi, R \cos \varphi)$). Reassembling the pieces,

$$r(\theta) = x_0 \cos \theta + y_0 \sin \theta = R \sin(\theta + \varphi), \quad (26)$$

which is a sinusoid in the (r, θ) plane whose amplitude R is just the point's distance from the origin and whose phase φ encodes its direction. The sinusoid enumerates *all* the lines that pass through that one point. Two pixels that happen to lie on a common line have sinusoids that cross at the (r, θ) of that shared line; many collinear pixels produce many sinusoids crossing at a single point. That crossing is the peak we are looking for. Figure 15 shows both readings side by side: collinear points in the image (one point, one sinusoid) versus the single bright accumulator cell where the sinusoids meet.

So far the “points” that vote have been abstract, but on a real image they are anything but. We do not start with a tidy list of points; we start with a grid of brightness values, and the first job is to decide which pixels are even worth a vote. That is what *edge detection* does: it flags the pixels that sit on a strong local change in brightness — the outlines where a dark region meets a light one — and leaves the smooth interior, which carries no boundary information, silent. These high-contrast *edge pixels* are the evidence the transform actually works with. A straight edge in the scene appears as a run of collinear edge pixels, and it is exactly their sinusoids that all pass through the one (r, θ) of that edge. A peak in parameter space is therefore not a geometric coincidence but a direct tally of how many real edge pixels line up: the more pixels that genuinely share a line, the taller the crossing they build, and the further it stands above the scatter left by unrelated pixels.

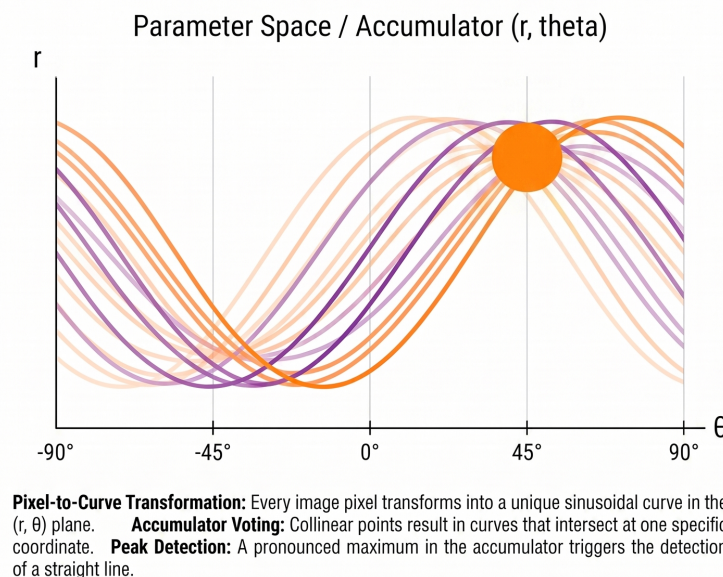
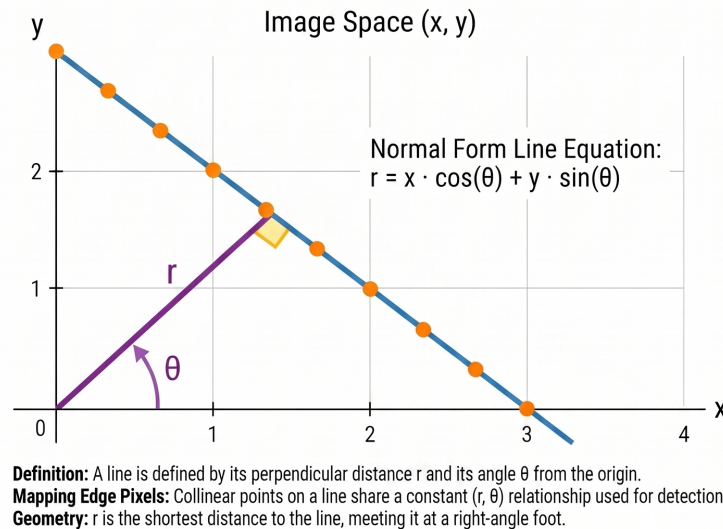
This is also where the name *Radon transform* comes from. We cannot solve for (r, θ) directly, because a single point lies on infinitely many lines. So we discretize the parameter space into a grid of cells, an *accumulator* $H_L(r, \theta)$ initialized to zero, and let each edge pixel add one vote to every cell on its sinusoid: for the pixel, sweep θ over the grid, compute the matching r from Equation (20), and increment $H_L(r, \theta)$. After all pixels have voted, a real line shows up as a pronounced maximum, while cells that received only scattered, accidental votes stay low. Detecting lines has become peak-finding in a 2D array.

It helps to picture H_L literally as an image and to set it next to the picture it came from, which is what Figure 16 does: the edge map on the left, the accumulator on the right. The horizontal axis is the angle θ and the vertical axis is the distance r — here allowed to take either sign, so that θ needs to run only over a half-turn instead of a full one. The plane is chopped into a grid of *bins*, and the number of bins along each axis sets the resolution of the search: too coarse and neighbouring lines merge into one cell, too fine and the votes for a single line scatter across several. The grid starts completely black, at zero. Each edge pixel then paints its sinusoid of votes across it, brightening exactly one bin in every θ column, so on its own a pixel leaves only a faint curve. Where many collinear pixels overlap, their sinusoids pile into the *same* bin, which flares into a concentrated bright hotspot, while the accidental votes of unrelated pixels stay dim and diffuse. Detecting lines is then nothing more than thresholding this image for its brightest cells and reading off their (r, θ) .

Using gradients. Making every edge pixel sweep over all directions θ is wasteful, because it throws away information the edge detector already handed us for free. Finding an edge means measuring, at each pixel, the direction in which the brightness changes fastest: the direction of *steepest contrast*, pointing straight across the boundary from the dark side to the light side. That direction is the image *gradient*. Now the geometric twist that makes it so useful — and it is easy to get backwards. The brightness changes fastest as you step *across* an edge and barely at all as you walk *along* it, so the gradient points *perpendicular* to the edge, not along it. The direction of

The Hough Transform: Detecting Straight Lines

The Hough Transform identifies shapes by mapping image pixels into a parameter space. For straight lines, each pixel votes for all possible lines it could belong to, represented as sinusoidal curves that intersect at a single coordinate (r, θ) corresponding to the detected line.



📖 NotebookLM

Figure 15: Hough duality for line detection. Collinear edge points in image space (top) all map to the same point in the (r, θ) accumulator (bottom); each individual point maps to a sinusoid, and the sinusoids intersect where the pixels agree. The pronounced peak triggers the detection.

maximum contrast tells us which way the boundary faces; the edge itself runs at a right angle to that. But “perpendicular to the edge” is precisely the *normal* direction of the line through the pixel — the same normal $\hat{n}$ whose angle we named θ in the Hesse form. The contrast direction therefore hands us θ directly, and with it the single line that can pass through that pixel, so the pixel casts one sharp, well-aimed vote instead of a whole sinusoid of guesses. To make this concrete, write the gradient of the image brightness as the pair $(G_x(x, y), G_y(x, y))$, the rates

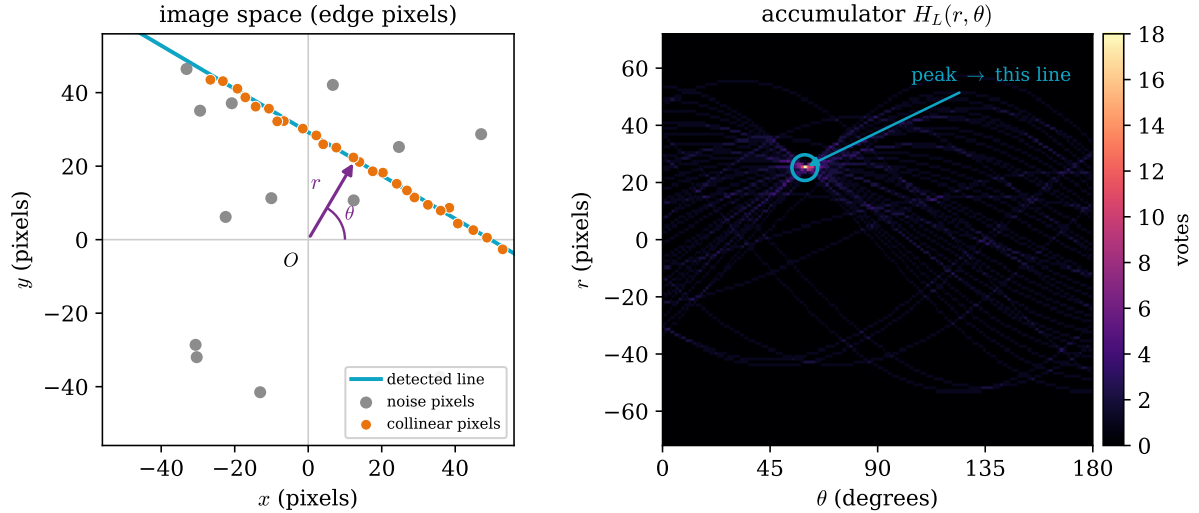


Figure 16: Image space and parameter space side by side, for a synthetic edge map of 26 collinear pixels (orange) plus 14 scattered ones (grey). *Left*: the edge pixels, with the detected line and its perpendicular parameters r and θ measured from the origin O . *Right*: the accumulator $H_L(r, \theta)$ drawn as an image. Every edge pixel casts one vote per θ column, tracing a sinusoid across the grid; the collinear pixels’ sinusoids all cross in the same bin, which lights up as the bright peak (circled) at $(\theta, r) \approx (60^\circ, 25 \text{ px})$, whereas the noise pixels leave only faint, uncrossed curves. That peak *is* the line on the left: line detection reduces to thresholding this grid for its hottest cells and reading off their (r, θ) . Brightness is the number of votes a bin has received.

of change in the x and y directions (for example the outputs of a Sobel filter). The gradient direction

$$\theta_G(x, y) = \text{atan2}(G_y(x, y), G_x(x, y)) \quad (27)$$

already fixes the angle θ of the line through that pixel, so there is no need to try every θ : we compute the single matching r from Equation (20) and update one cell instead of a whole sinusoid. (The function atan2 is the two-argument arctangent; unlike $\arctan(G_y/G_x)$ it returns the angle in the correct quadrant and does not divide by zero when $G_x = 0$.) Two conventions matter when reading exercises: gradients point from dark to bright, and angles are measured counter-clockwise from the x -axis. How G_x and G_y are computed with derivative filters is summarized in Appendix A. Finally, a sharp, high-contrast edge is stronger evidence for a line than a faint one, so instead of adding a flat $+1$ we weight each vote by the gradient magnitude,

$$H_L(r, \theta) \leftarrow H_L(r, \theta) + \sqrt{G_x(x, y)^2 + G_y(x, y)^2}, \quad (28)$$

where $\sqrt{G_x^2 + G_y^2}$ is the length of the gradient vector, that is, how steeply the brightness changes at the pixel. A line in the image then produces a pronounced maximum in H_L , found by the same simple peak detection as before.

Points versus edges. The contrast between voting with bare points and voting with oriented edges is worth keeping in mind, because it recurs in every later method. A bare point is consistent with infinitely many lines and must vote for all of them (its full sinusoid), which inflates the chance that unrelated points accidentally agree and forces a higher peak threshold. An oriented edge, by contrast, carries a direction, pins down θ through Equation (27), and votes for essentially one line. The lesson generalizes: the more informative each piece of local evidence is, the cleaner the peaks and the fewer the false alarms. The two modes carry two names. Letting a point sweep over all θ and trace its full sinusoid is the *Radon* view: a point in image space becomes

a sinusoid in parameter space. The gradient-based *Hough* line detector maps an oriented edge pixel to a single point in parameter space.

Working with the parameterization. Three kinds of exercise follow directly from Equation (20). First, to test whether a line (θ, r) passes through a point, substitute the point and compare. For the point $(x, y) = (3, 7)$:

(θ, r)	$x \cos \theta + y \sin \theta$	on the line?
$(0.5\pi, 3)$	$3 \cdot 0 + 7 \cdot 1 = 7$	no ($7 \neq 3$)
$(0, 7)$	$3 \cdot 1 + 7 \cdot 0 = 3$	no ($3 \neq 7$)
$(0, 3)$	3	yes
$(0.75\pi, -7)$	$3 \cdot (-\frac{\sqrt{2}}{2}) + 7 \cdot \frac{\sqrt{2}}{2} = 2\sqrt{2} \approx 2.83$	no

The pattern to remember: $\theta = 0$ describes the vertical line $x = r$, $\theta = \pi/2$ the horizontal line $y = r$.

Second, to count the votes a pixel casts, count the parameter cells it increments. Suppose θ is discretized in 1° steps over $0^\circ, \dots, 359^\circ$ and r over $-100, \dots, 100$. Without gradient information a point votes once per θ column, 360 votes (one r per θ , never all 201 values of r). With its gradient (G_x, G_y) the orientation θ is fixed by Equation (27), r follows from the Hesse form, and the pixel casts exactly *one* vote.

Third, to recognize the shape behind a voting space, count the peaks. Every straight segment is one line and produces one peak; two parallel sides share θ and differ only in r , so they give two peaks in the same column. A triangle therefore gives three peaks, a square or parallelogram four (in two pairs), and a circle none at all: each of its tangent lines touches only one point, so the votes spread evenly over a band $|r| \leq R$ whose edges are merely brighter where many tangents nearly coincide. Figure 17 shows these four voting spaces; a space with three maxima can only come from the triangle. By the same logic, two isolated pixels produce two sinusoids that cross in exactly one cell, the unique line through both of them. Circles are detected with a different parameterization, $(x - x_c)^2 + (y - y_c)^2 = R^2$, whose voting space is three-dimensional (x_c, y_c, R) .

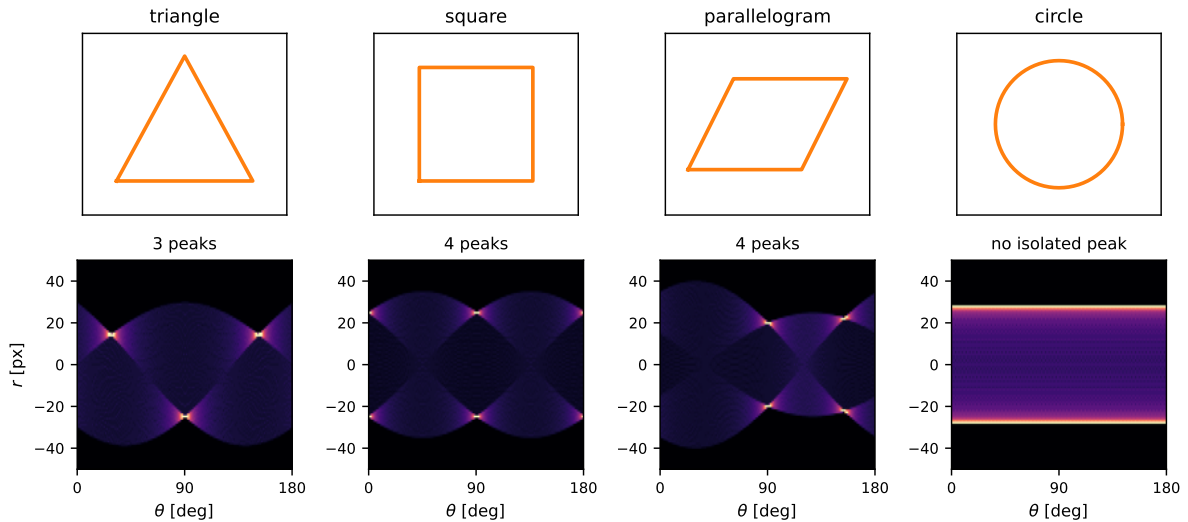


Figure 17: Line voting spaces of four outlines (every contour point votes for all $\theta \in [0^\circ, 180^\circ)$, signed r). Each straight side is one peak: three for the triangle, four for the square and the parallelogram, whose parallel sides appear as peak pairs at the same θ . A side at $\theta = 0^\circ$ reappears at the right border as $\theta = 180^\circ$ with opposite sign of r , which is the same line. The circle has no isolated peak, only a uniform band between $r = -R$ and $r = R$.

3.2 Detecting Arbitrary Shapes: The Generalized Hough Transform

Lines are special because they have a tidy two-parameter equation. Most objects do not: a wrench, a leaf, or a logo is an arbitrary 2D curve with no closed-form parameterization. Since we cannot write such a shape down as a formula, we instead *show* it to the algorithm: we supply a *template* — a single example, one image of the object or equivalently the 2D curve of its boundary — that serves as the known geometric model of exactly the shape we want to find. The *generalized Hough transform* (GHT) keeps the voting idea but, in place of a parametric equation, compiles this template into a stored table that records the shape’s geometry explicitly. The detector still asks the same question every edge can answer with a vote, only now about a shape rather than a line: *if you are part of this shape, where is its center?*

Everything then splits into two phases built around that template, laid out in Figure 18. *Training* reads the template once and turns it into the table; *detection* runs the votes on each new test image. We take the two in turn.

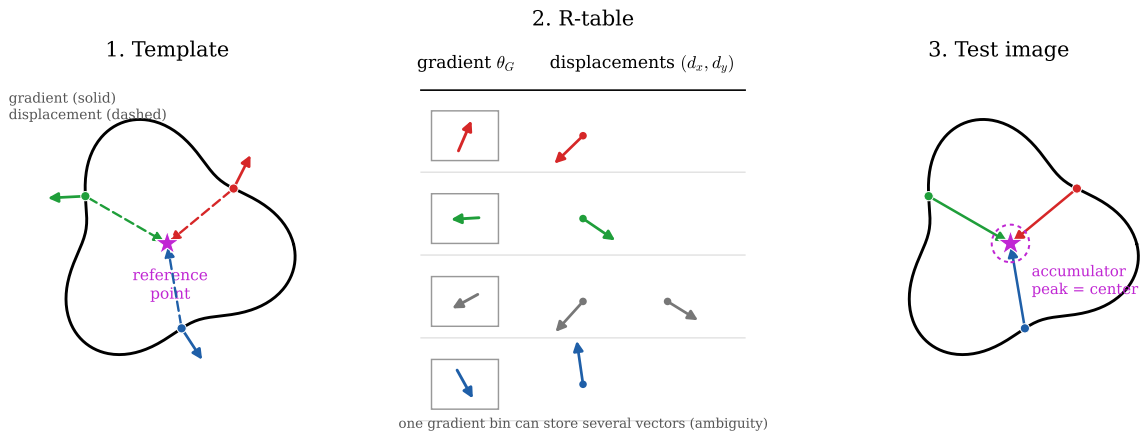


Figure 18: The generalized Hough transform, from template to detection. (1) *Template*: on the known model shape we fix a reference point (the center, magenta star); at each edge we read its gradient direction (solid arrows) and record the displacement vector pointing to the center (dashed arrows). (2) *R-table*: those displacements are filed by gradient orientation, so a gradient direction indexes straight to the vectors seen at edges facing that way; because one orientation can occur at several places on the shape, a single bin may hold more than one vector. (3) *Test image*: each observed edge looks up its gradient in the R-table and votes for a center at its position plus the stored displacement; edges of a genuine instance all point to the same spot, which wins the accumulator peak. Colours track the same edge through all three panels.

To set this up we first pick one *reference point* on the template, typically (but not necessarily) its center of mass. This reference point plays the role that (r, θ) played for lines: its position in a test image is what we are voting for. During training we walk along the template’s contour, and for each edge pixel at position (x, y) we record the *displacement vector* (d_x, d_y) that points from that edge to the reference point, so that

$$(x_{\text{ref}}, y_{\text{ref}}) = (x + d_x, y + d_y). \quad (29)$$

The logic at detection time is the mirror image of Equation (29): if an observed edge in the test image is the same part of the shape, then the center must lie at the edge’s own position plus one of the stored displacements, so the edge votes for those locations. Edges that genuinely belong to one instance of the shape all point back to the same spot, which lights up as a peak; unrelated edges scatter their votes harmlessly.

What keeps this efficient is the *R-table*, an indexed list of the displacement vectors. Just as the gradient orientation accelerated line detection, it organizes the table here. We quantize the

possible gradient directions into A equal bins (rows). Row T covers gradient orientations near $\theta \approx 2\pi T/A$ for $T \in \{0, 1, \dots, A-1\}$, and it stores every displacement vector that was observed at an edge with roughly that orientation:

$$\text{row } T : \quad (d_x^{T,1}, d_y^{T,1}), (d_x^{T,2}, d_y^{T,2}), \dots, (d_x^{T,N_T}, d_y^{T,N_T}). \quad (30)$$

Concretely, with $A = 8$ bins each row spans 45° ; an edge whose gradient points at 100° falls in the row centered near 90° , and we read off exactly that row's vectors. The superscripts in Equation (30) just enumerate the N_T different vectors stored in row T — there can be several, which is the point of the next paragraph. At test time, then, the procedure is: read each edge's gradient orientation, jump to the matching row, and cast a vote at $(x + d_x, y + d_y)$ for every displacement (d_x, d_y) in that row.

Why does one row hold several vectors? Because the local gradient is *ambiguous*: the same gradient direction occurs at several different places on a shape (think of the two opposite sides of a circle, whose outward normals are antiparallel, or the many points along a smooth curve that happen to face the same way). A given orientation is therefore consistent with several possible centers, so a single observed edge generally increments *several* accumulator cells, one per displacement in its row. This is not a defect to be engineered away; it is precisely why voting is needed. Each edge is individually uncertain about where the center is, but only the true center collects support from many edges at once. Figure 19 traces this from the model contour and its displacement vectors through the R-table to the accumulator peak.

Real instances also differ from the stored template in size and orientation. To handle this, the accumulator gains two more axes and becomes four-dimensional, $H(x, y, s, \theta)$, where (x, y) is the candidate center, s a scale factor, and θ a rotation angle. The two phases are then:

Training. Extract the edge pixels of the template; for each, compute its gradient orientation and store the displacement vector to the reference point in the corresponding R-table row (Equation (30)).

Prediction. For each scale s , multiply the stored displacement vectors by s ; for each rotation θ , rotate the vectors by θ and shift the R-table rows around by the same amount (a circular shift, since rotating the shape rotates every edge's gradient by θ); then, for each edge pixel in the test image, look up its row, cast a vote from each of its displacement vectors, and increment $H(x, y, s, \theta)$. Finally, find the peaks in the four-dimensional array.

The price of this generality is the four-dimensional search; the reward is a detector for an *arbitrary* known shape rather than a single parametric family of lines.

Transforming an R-table. The prediction loop builds a modified R-table for every candidate (s, φ) , and exercises ask for exactly this step. Three rules cover every case. A *scale* s multiplies every displacement vector and leaves the rows alone, since scaling does not change gradient directions. A counter-clockwise *rotation* by φ rotates every displacement vector,

$$\begin{pmatrix} d'_x \\ d'_y \end{pmatrix} = s \begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix} \begin{pmatrix} d_x \\ d_y \end{pmatrix}, \quad (31)$$

and rotates every gradient by φ , so the entries of the row for orientation θ move to the row for $\theta + \varphi$: a circular shift by $\varphi/\Delta\theta$ rows, where $\Delta\theta$ is the bin width. If the table only covers orientations in $[0^\circ, 180^\circ)$, orientations are taken modulo 180° . A *translation* changes nothing, because the displacements are relative to the reference point, which moves along with the shape. Three worked cases show how the rules combine:

- Rows of width 45° over $[0^\circ, 180^\circ)$, $s = 2$, $\varphi = 180^\circ$. A rotation by 180° maps (d_x, d_y) to $(-d_x, -d_y)$, and $\theta + 180^\circ \equiv \theta \pmod{180^\circ}$ keeps every entry in its row. With the scale, $(1, 2)$ becomes $(-2, -4)$, $(9, 10)$ becomes $(-18, -20)$, and the empty row stays empty.

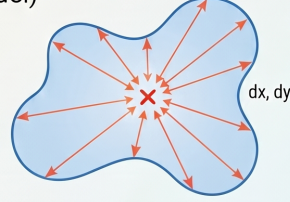
The Generalized Hough Transform: A Two-Stage Process for Shape Detection

Enables detection of arbitrary 2D shapes by mapping edge orientations to displacement vectors through a relational model (R-Table) and probabilistic voting.

Stage 1: Training (Building the Model)

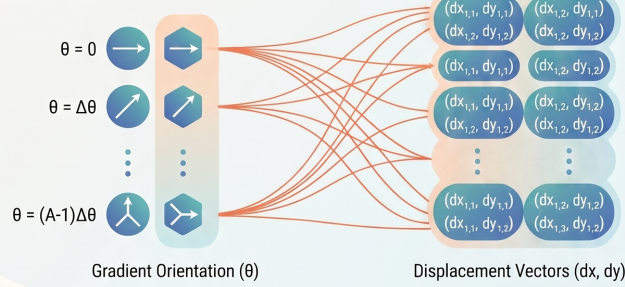
Define Reference Point and Displacement

Select a center point for the arbitrary contour and map displacement vectors from every edge point.



Construct the R-Table

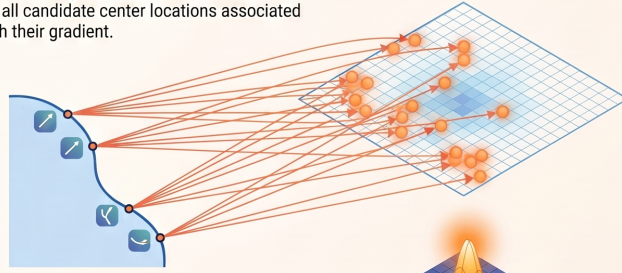
Store displacement vectors (dx, dy) in a table indexed by the local gradient orientation of the edge.



Stage 2: Detection (Object Recognition)

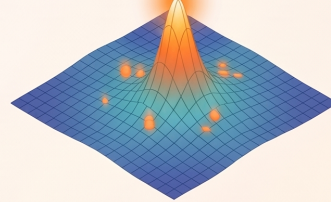
Probabilistic Voting

Observed edges in a test image cast votes for all candidate center locations associated with their gradient.



Accumulator Peak Identification

The true center of the object is identified where the highest density of votes (the pronounced peak) occurs.



NotebookLM

Figure 19: Generalized Hough transform. Each model edge stores a displacement vector to the reference point, organized in the R-table by gradient orientation. Because the same orientation occurs at several places on the shape, one observed edge casts votes for several candidate centers; the true center is where votes accumulate.

- Rows of width 90° over $[0^\circ, 360^\circ)$, $s = 1$, $\varphi = 180^\circ$, translation $t = (10, 10)$. Every vector is negated, and every row moves two places on: the entries of $[0, 90)$ go to $[180, 270)$, those of $[180, 270)$ to $[0, 90)$, those of $[270, 360)$ to $[90, 180)$. The translation is ignored.
- $s = 2$, $\varphi = 45^\circ$, rows of width 45° . With $\cos 45^\circ = \sin 45^\circ = 1/\sqrt{2}$, Equation (31) becomes $(d'_x, d'_y) = \sqrt{2}(d_x - d_y, d_x + d_y)$, for example $(2, 0) \mapsto (2\sqrt{2}, 2\sqrt{2})$, and each entry moves

one row on.

The same rules explain which invariances a single R-table has (Table 11).

Property	Holds?	Reason
Invariant to translation	yes	displacements are relative to the reference point
Invariant to scale	no	displacements must be multiplied by s
Invariant to rotation	no	vectors must be rotated and rows shifted
Robust to noise and partial occlusion	yes	each edge votes on its own; a missing part lowers the peak, noise adds scattered votes
Needs gradient orientation at (x, y)	yes	the orientation selects the row to vote with

Table 11: Properties of a single R-table. Scale and rotation are handled by the extra accumulator dimensions, not by the table itself.

The dimensionality of the voting space follows from what is unknown: a line detector votes in 2D (r, θ) , a circle detector in 3D (x_c, y_c, R) , and the generalized Hough transform in 4D (x, y, s, θ) . An alternative to explicit R-tables correlates complex gradient images of template and image, which can be computed with the Fourier transform (Appendix B).

3.3 Where to Look: Salient Regions

The generalized Hough transform still assumes we possess an exact template of the object. That assumption breaks for object *categories*: no two cars, faces, or motorbikes share one outline, so there is no single contour to store in an R-table. Before we can let parts vote for an object, we have to decide which image locations even deserve to be treated as parts. This is the problem of *saliency*: detecting distinctive regions whose local appearance stands out from their surroundings, so that the same regions can be found again on a new, previously unseen instance.

The motivation is partly biological. Saliency is a subjective perceptual quality that makes certain items capture an observer’s attention before others. A standard computational account mirrors early vision: simple features (color, edge orientation, luminance, motion) are measured in parallel across the whole field and at several scales, producing a stack of *feature maps*. Each map is processed to emphasize locations that differ from their neighbors, the maps are combined into a single saliency map, and attention — and, in a living visual system, the eyes and head — is drawn to the most salient locations. The idea we borrow for object models is this notion of “standing out from the surroundings,” made mathematically precise and, importantly, invariant to scale and rotation so that detections repeat across instances.

The Kadir/Brady detector makes “standing out” concrete by equating saliency with *rareness*, measured through Shannon entropy. The reason rareness needs a careful definition is that it is easy to get trivially wrong: if our description of a patch is so specific that every location looks unique, then everything is equally rare; if it is so generic that everything looks alike, nothing is rare. Entropy of the local intensity distribution sits between these extremes and measures the *complexity* of a patch. For a window of pixels, let $p(k)$ be the fraction of pixels whose grey value equals k , for $k = 0, \dots, 255$; this is just the normalized local histogram. The entropy is

$$H = - \sum_{k=0}^{255} p(k) \log_2 p(k), \quad \text{with the convention } 0 \log_2 0 = 0. \quad (32)$$

Each term $-p(k) \log_2 p(k)$ is the contribution of grey level k ; summing them gives the average number of *bits* needed to encode a pixel drawn from the window, which is also a measure of how

uncertain we are about that pixel's value. A couple of concrete windows fix the intuition. If every pixel has the same grey value, one $p(k)$ equals 1 and the rest are 0, so $H = 0$: a flat, predictable patch carries no information. If the window contains two grey values in equal numbers, two terms equal $-\frac{1}{2} \log_2 \frac{1}{2} = \frac{1}{2}$ and $H = 1$ bit. If all 256 levels are equally likely, $p(k) = 1/256$ for each and $H = \log_2 256 = 8$ bits, the maximum. So a *flatter* local histogram means *higher* entropy, and a textured, unpredictable patch is rare in this spatial sense, while a smooth or constant patch is not.

Because Equation (32) depends only on the *list* of histogram values, several properties follow immediately:

- A patch whose histogram is uniform over all grey levels, such as pure noise, has the highest possible entropy.
- Relabeling grey values, for example swapping two of them, only permutes the histogram bins and does not change the entropy.
- Permuting the pixels leaves the histogram, and hence the entropy, unchanged. A checkerboard and a single straight edge with the same 50/50 black-white split both have $H = 1$ bit.
- A homogeneous patch ($H = 0$) has lower entropy than a patch containing an edge ($H > 0$).

The third property also shows the weakness of the measure: the intensity distribution contains no spatial information. Ranking patches by entropy is therefore a matter of counting grey levels and their frequencies. A constant patch ($H = 0$), a patch half black and half white ($H = 1$), a patch with four equally frequent grey levels ($H = 2$), and a noise patch (H close to 8) are in ascending order of entropy, whatever the arrangement of their pixels.

High spatial entropy alone is still not enough, because pure noise also has a flat histogram yet is not distinctive: its statistics barely change as the analysis window grows, so it looks the same at every scale. We therefore add a second requirement, *self-dissimilarity in scale*: a salient region should look different at its own scale than at nearby scales. To express this we analyze each location over a range of window sizes (a Gaussian scale space, or an image pyramid that subsamples in logarithmic steps), and treat entropy as a function of the scale s . Writing $p(I, s, x)$ for the probability density of intensity value I inside a window of scale s centered at position x , the scale-dependent entropy and the scales at which it peaks are

$$H_D(s, x) = - \int p(I, s, x) \log_2 p(I, s, x) dI, \quad s_p = \left\{ s : \frac{\partial H_D}{\partial s} = 0, \frac{\partial^2 H_D}{\partial s^2} < 0 \right\}. \quad (33)$$

Equation (33) is just Equation (32) written for a continuous intensity I and made a function of scale; the condition on s_p is the textbook test for a local maximum (first derivative zero, second derivative negative), so s_p collects the scales where the entropy curve has a *peak*. Each such peak is then reweighted by how strongly the intensity distribution itself changes with scale,

$$W_D(s, x) = s \int \left| \frac{\partial}{\partial s} p(I, s, x) \right| dI, \quad y(s_p, x) = H_D(s_p, x) W_D(s_p, x). \quad (34)$$

The weight W_D is large when the distribution looks markedly different at scale s than at neighboring scales (the $|\partial p / \partial s|$ term), with the leading s making the measure comparable across scales; the final saliency $y(s_p, x)$ is the product, so a location scores highly only when it is both complex (high H_D) and tied to a distinctive scale (high W_D). This is exactly what suppresses featureless noise, whose entropy is high but flat across scale. The same holds for a regular texture such as a checkerboard: it has the highest entropy of a binary patch, but larger windows keep showing the same black/white ratio, so there is no peak. A single dark blob on a bright background has lower entropy, yet its histogram changes sharply once the window grows past the

blob, so the blob is the more salient region. The same argument ranks points within one image: the center of a blob is the most salient, because its entropy peaks at the scale where the window just encloses the blob. A point on the blob's boundary sees roughly half dark and half bright at every window size, so its entropy is high from the start but shows no peak over scale; a point in a flat region has zero entropy until the window reaches some structure. Figure 20 shows the entropy-versus-scale curve with its peak, alongside the flat-versus-peaked histogram intuition.

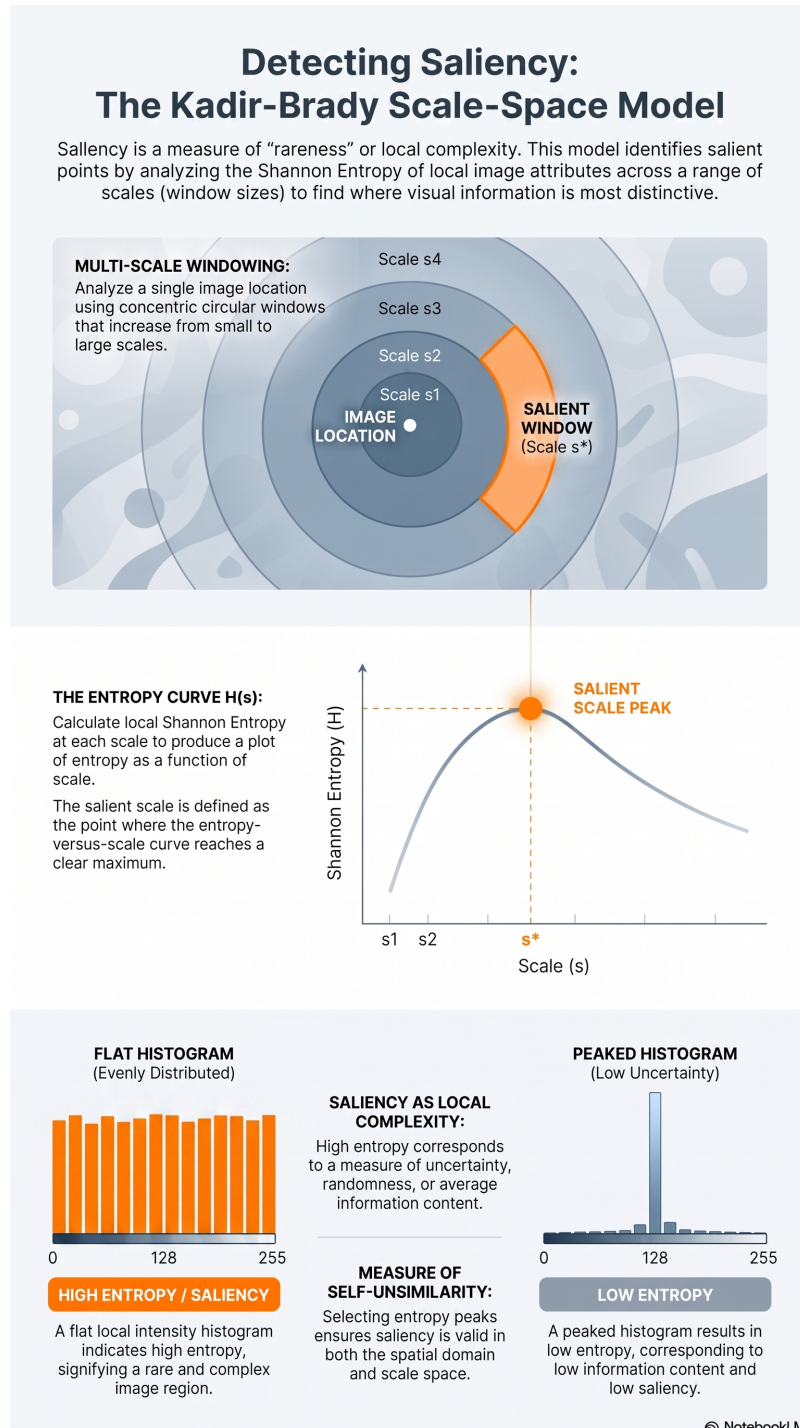


Figure 20: Scale-space saliency. The same location is analyzed over a range of window sizes; entropy as a function of scale has a peak at the salient scale. A flat local histogram (high entropy) signals a rare, informative region, while the scale-localization weight discounts patches whose statistics barely change across scale.

The two multi-scale representations mentioned above are not the same thing. An *image pyramid* subsamples the image in logarithmic steps of resolution, $R_0, R_{-1}, R_{-2}, \dots$, so each level is smaller. A *linear Gaussian scale space* keeps the full resolution and instead smooths the original signal $L(x)$ with Gaussian kernels of growing width t ,

$$L(x, t) = L(x) * G(x, t), \quad G(x, t) = (\pi t)^{-n/2} \exp\left(-\frac{|x|^2}{t}\right), \quad (35)$$

where n is the dimension of x ($n = 2$ for images). A Gaussian scale space is therefore a family of images, all derived from the original and progressively more blurred as t increases; it is not a pyramid with randomly drawn scales, nor an assumption about how object sizes are distributed.

The scale weight also decides exercises that ask for the *most salient* patch. Saliency in the sense of Kadir and Brady is $y = H_D \cdot W_D$ at a peak scale, not the entropy alone. A noise patch has the highest entropy of all, yet its statistics are the same at every scale, so W_D is small and the patch is not salient. When a task only supplies entropy values, the ranking comes from those values, with patches that are known to be scale-less noise discounted. Entropy of the intensity distribution alone is not sufficient as a saliency measure because that distribution contains no spatial information.

In practice the algorithm visits each pixel, computes the local entropy (Equation (32)) over a range of scales, keeps the scales where the entropy peaks (there may be none), and weights those entropies by the inter-scale change of the distribution. Run on a dataset such as side views of motorbikes, and averaged over many images, the detected salient points cluster on repeatable, characteristic object parts — wheels, the engine block, the seat — which is precisely the raw material the next models need.

This is the *classical* stance on saliency: hand-crafted features grounded in information theory, computed without learning, and largely task-independent (data enter only at evaluation time). The *modern* alternative is to learn saliency from data with deep networks, building on the multilayer perceptrons and convolutional networks of Section 2; a convolutional network trades hand-designed rareness for features learned from many labeled examples, at the cost of needing a great deal of such data. We stay with the classical view here because it makes explicit *why* a location is worth voting from.

3.4 What to Record: Local Features and Shape Descriptors

Saliency tells us *where* to look; a descriptor decides *what* to record there, encoding the neighborhood as a vector so that the same part can be matched again on a new image. What makes one descriptor better than another is best understood as a balancing act among four competing demands, summarized in Table 12.

Property	What it means
Invariance	Stays stable under nuisance changes: translation, rotation, scale, and illumination.
Discriminability	Separates similar classes reliably; different parts get different descriptors.
Compactness	Stores enough information in a short vector to keep matching and storage cheap.
Computational cost	Remains feasible to extract and compare on large datasets.

Table 12: The four criteria for a good local feature. Invariance and discriminability pull in opposite directions, and compactness and cost bound how far either can be pushed.

These demands genuinely conflict. Pushing invariance to the limit deliberately discards information — if a descriptor is built to ignore rotation, it can no longer tell two parts apart that differ only by rotation — which eats into discriminability. And richer, more discriminative descriptors are longer and costlier to compute and compare. Every concrete descriptor is one particular compromise among the four.

The *histogram of oriented gradients* (HOG) is the canonical compromise on the shape-and-appearance side, and it is worth seeing how it is built because it reuses the gradients from Equation (28). The region is divided into a grid of small *cells*; within each cell the gradient orientations of the pixels are collected into a histogram (for example over nine orientation bins), each pixel contributing in proportion to its gradient magnitude. Cells are then grouped into overlapping *blocks*, and the histograms in a block are normalized together before being concatenated, over all blocks, into one long feature vector. Figure 21 shows the whole construction on a simple shape. Two design choices give HOG its character. Summarizing gradient *orientations* rather than raw intensities, together with the block normalization, makes it robust to moderate lighting changes and small deformations. But because the descriptor is laid out as a fixed grid of oriented histograms, rotating the patch permutes the bins and changes the vector: standard HOG is *not* rotation invariant. In return it turns the diffuse evidence of many edges into a single classifier-ready vector, which is why it remains a standard baseline against which learned features are measured.

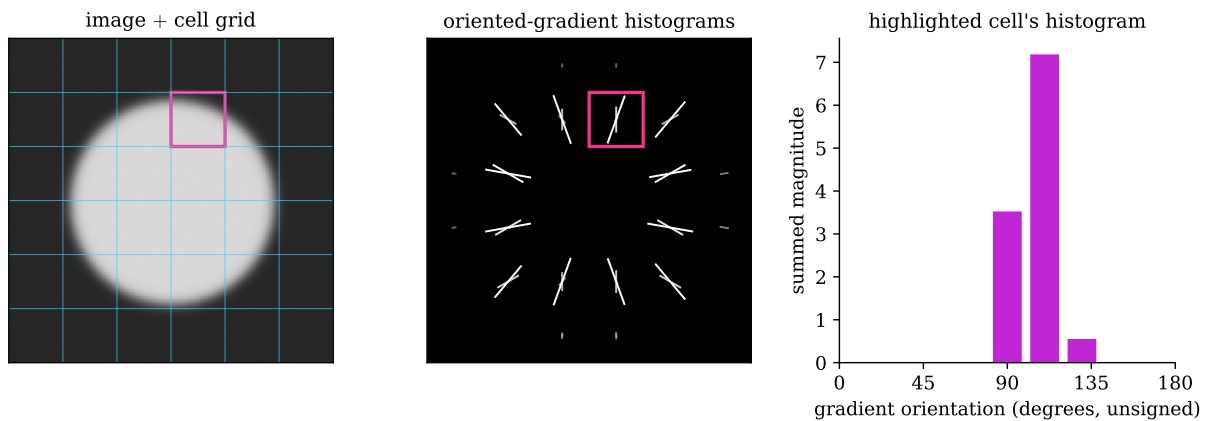


Figure 21: Building a HOG descriptor, computed on a synthetic disk. *Left:* the region is divided into a grid of cells (one highlighted in magenta). *Middle:* each cell is replaced by a small star of strokes, one per orientation bin, whose length is the total gradient magnitude falling in that orientation; since gradients run perpendicular to edges, the strokes point radially around the disk’s rim, while flat interior and exterior cells stay empty. *Right:* the highlighted cell’s nine-bin histogram of gradient orientation, each pixel weighted by its gradient magnitude, with a clear peak at the orientation of the local edge. Stacking every cell’s (block-normalized) histogram into one vector yields the descriptor.

Two further descriptor families round out the classical toolkit and contrast usefully with HOG. *Fourier descriptors* encode a closed contour by the Fourier coefficients of its boundary curve; translation, rotation, and scaling of the shape act on those coefficients in simple, separable ways (a shift of the constant term, a phase change, an overall magnitude scaling), so after normalization the coefficients become invariant shape descriptors. Their construction, normalization, and use for smoothing are worked out in Appendix B. *Hu moments* combine image moments into a handful of quantities that are likewise invariant to translation, scale, and rotation. For detecting the salient *points* in the first place, the standard keypoint operators (Harris, Förstner, SIFT) provide the geometric counterpart to the information-theoretic salient regions of the previous subsection. The common thread across all of them is the same: a descriptor converts a neighborhood into a

compact vector, and closeness between two such vectors is taken to mean that the two patches show the same thing.

3.5 Putting It Together: The Implicit Shape Model

The implicit shape model (ISM) is where the three threads of this section — voting, saliency, and description — finally meet. Its goal is object *categorization* in real scenes: given training images of a category, learn a model from the local appearance of its parts; given a new image, both label and segment the instances. The one-sentence summary is that ISM is a generalized Hough transform whose fixed geometric R-table has been replaced by a *learned appearance codebook*. Where the GHT indexed displacement vectors by an edge’s gradient orientation, ISM indexes them by a patch’s appearance; the voting machinery is otherwise the same.

The **training phase** builds that codebook. Salient points are extracted from the training images (using the salient-region or keypoint detectors above), and each one defines a *patch* at its characteristic scale. The patches are described by their local appearance and then grouped so that visually similar patches become a single *codebook* entry — a prototype part such as “front wheel” or “headlight.” The grouping is done by iterative merging: every patch starts as its own cluster, and at each step the two most similar clusters C_1 and C_2 are merged, provided their average pairwise similarity stays above a threshold t . Similarity between two patches p and q is the normalized greyscale correlation, and the similarity of two clusters averages it over all their members:

$$\text{NGC}(p, q) = \frac{\sum_i (p_i - \bar{p})(q_i - \bar{q})}{\sqrt{\sum_i (p_i - \bar{p})^2 \sum_i (q_i - \bar{q})^2}}, \quad \text{sim}(C_1, C_2) = \sum_{p \in C_1, q \in C_2} \frac{\text{NGC}(p, q)}{|C_1| |C_2|}. \quad (36)$$

In $\text{NGC}(p, q)$, the index i runs over the pixels of the patch, p_i and q_i are their grey values, and $\bar{p}, \bar{q}$ are the patch means; subtracting the means removes brightness offsets and the denominator removes contrast scaling, so NGC is a correlation coefficient between -1 and 1 that equals 1 for patches identical up to brightness and contrast and near 0 for unrelated ones. The cluster similarity $\text{sim}(C_1, C_2)$ is then just the average NGC over all cross-pairs, with $|C_i|$ the number of patches in cluster C_i . Stopping the merges at the threshold t keeps clusters compact and groups only genuinely similar patches. Finally, for each codebook entry we record, across all the training images, the offset vectors from every matching patch to the object center — the appearance analogue of the R-table’s displacement vectors, specifying where on the object each prototype part tends to appear. In order, training runs through five steps:

1. extract salient points from the training images and cut out a patch at each point’s scale;
2. describe each patch by its local appearance (grey values or a keypoint descriptor);
3. cluster similar appearances by iterative merging and store the cluster centers as the codebook;
4. match the codebook entries against all training patches and activate every entry whose similarity exceeds t ;
5. for every activation, store the offset vector from the patch to the object center with the activated entry.

The codebook itself thus holds *prototypes of object parts* with a stable appearance across images; the offsets to the reference point are attached to each entry afterwards. They play the role of the R-table rows, not of the codebook’s content.

The **prediction phase** runs the voting. Salient patches are extracted from the test image and matched against the codebook; an entry is *activated* whenever its NGC similarity to a patch exceeds the threshold t . Each activated entry then casts generalized Hough votes for the object

center, placing a vote at the patch position plus each of that entry’s stored offsets — exactly the GHT step, but driven by appearance rather than edge orientation. Unlike the discrete accumulator of the classical GHT, ISM votes in a *continuous* space, and the densest cluster of votes is located with mean-shift mode estimation (equivalently, a non-parametric density estimate); that mode is the hypothesized object center. A final *back-projection* step reverses the voting: the patches that supported the winning center are projected back into the image, which both localizes the object within a surrounding region and, through a probabilistic formulation, yields a pixel-level *segmentation* rather than a mere bounding box. Figure 22 lays out the whole pipeline, training above and prediction below.

The Implicit Shape Model (ISM) Pipeline: From Local Patches to Global Detection

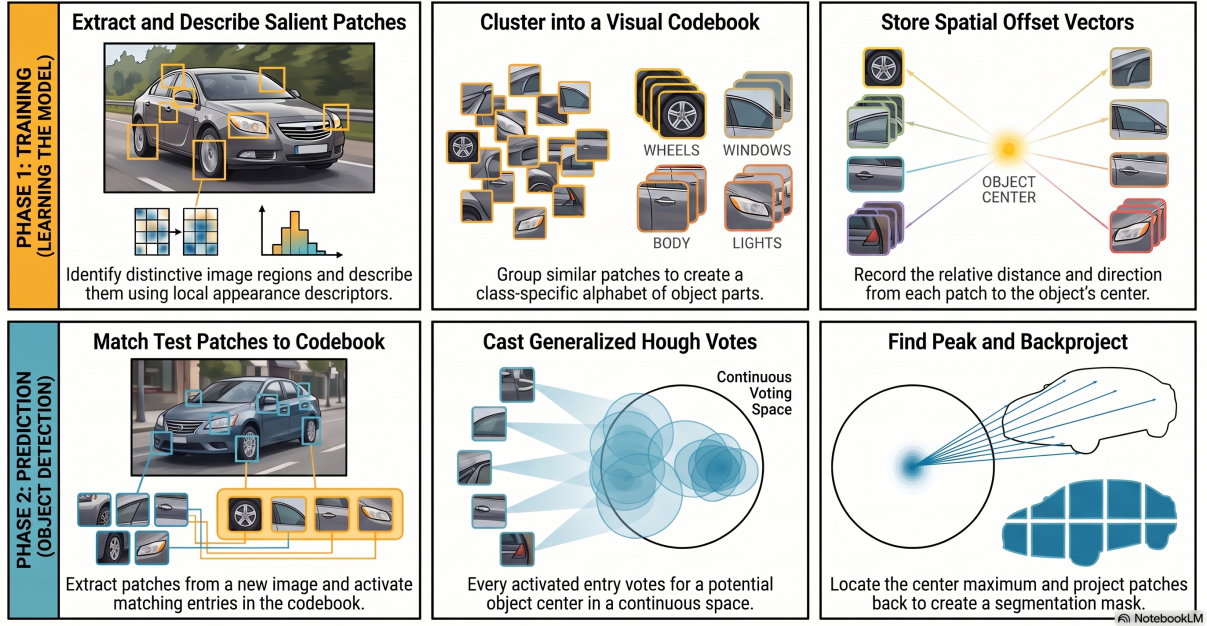


Figure 22: The implicit shape model. Training (top row): salient patches are described, clustered into a codebook, and annotated with offset vectors to the object center. Prediction (bottom row): test patches are matched to the codebook, cast generalized Hough votes for the center, and the peak is back-projected to recover and segment the object.

Because ISM grows directly out of the generalized Hough transform, it helps to see the two side by side; Table 13 lines up what changes and what stays the same.

Aspect	Generalized Hough transform	Implicit shape model
Model of a part	Fixed geometric R-table	Learned appearance codebook
Local evidence	Edge + gradient orientation	Salient patch + appearance
Indexing	Gradient-orientation row	Codebook match above threshold t
Voting space	Discrete accumulator	Continuous (mean-shift modes)
Output	Center / pose of one known shape	Category detection <i>and</i> segmentation

Table 13: From a single known shape to an object category: ISM keeps the voting machinery of the generalized Hough transform but learns the parts and their spatial distribution from data.

Two further models slot naturally into the same picture. *Hough forests* replace the codebook-

and-matching step with a random forest that maps each patch directly to a distribution of votes, learning the appearance grouping and the offsets together in one trained predictor. *Deformable part models* (DPM) take a complementary route: an object is a coarse root HOG template plus a few higher-resolution part templates attached to it by spring-like deformation costs, the whole thing scored with a latent-variable classifier. The Hough forest still votes, but learns the votes instead of looking them up in a codebook. DPM does not vote at all: it localizes objects by classifying windows of the HOG feature map at many positions and scales. Appendix C gives the details needed for both. Both keep the central bargain of this section — local parts proposing global configurations — while changing how the parts are represented and how their votes are combined. The implicit shape model itself is due to Leibe, Leonardis, and Schiele (ECCV’04 workshop; IJCV 2008).

It is worth ending where we began. Every model in this section is one instance of the same mechanism, differing only in what counts as local evidence and how much flexibility the global hypothesis is allowed; Table 14 lines them up along that single axis.

Model	What casts a vote	What it votes for
Hough line detector	An oriented edge pixel	The line (r, θ) it lies on
Generalized Hough transform	An edge, indexed by gradient orientation	The reference center of a known shape, in (x, y, s, θ)
Implicit shape model	A patch matched to a codebook entry	The object center, in a continuous space, then a segmentation

Table 14: The through-line of the section. As the local evidence grows richer — from edge to oriented edge to learned appearance patch — the same voting scheme stretches from a single line to an arbitrary shape to a whole object category.

4 Bayesian Decision Theory and Probability Estimation

Every image analysis system eventually has to commit to a decision: this pixel is road, this window contains a pedestrian, this lesion should be referred to a doctor. The evidence behind such decisions is never certain. Noise corrupts the measurements, classes overlap in feature space, and the same appearance can be produced by different causes. Probability theory is the language that turns this uncertainty into something we can compute with, and Bayesian decision theory says how to act optimally once the uncertainty has been quantified.

The chain of reasoning in this section runs in three steps. First, a decision is made by comparing *posterior* probabilities $P(\omega \mid x)$ of the possible classes ω given an observation x , possibly weighted by the cost of each mistake. Second, the posterior is assembled from a *likelihood* $p(x \mid \omega)$ and a *prior* $P(\omega)$ with Bayes’ theorem, exactly as in the scene-inference formula of the introduction. Third, the likelihood is almost never given and has to be *estimated* from training data, either without assumptions about its shape (histograms, Parzen windows, nearest neighbors) or by fitting the parameters of an assumed model (maximum likelihood and Bayesian estimation). The section closes with Markov random fields, where the prior is not a single number per class but a model of how neighboring pixels depend on each other, which lets prior knowledge repair noisy image data.

4.1 Probability Basics

A probability assigns to each possible outcome of an experiment a number that says how much we believe in it. Let Ω be the *sample space*, the set of all possible discrete outcomes ω . A function

$P : \Omega \rightarrow \mathbb{R}$ is a probability if and only if

$$P(\omega) \geq 0 \quad \forall \omega \in \Omega \quad \text{and} \quad \sum_{\omega \in \Omega} P(\omega) = 1. \quad (37)$$

Both conditions together imply $P(\omega) \leq 1$. The probability of an event $E \subseteq \Omega$, a set of outcomes, is the sum over its members, $P(E) = \sum_{\omega \in E} P(\omega)$.

Densities are not probabilities. For a continuous variable such as a grey value or a feature coordinate, a single exact value has probability zero, because there are infinitely many values competing for a total mass of one. What exists instead is the *cumulative distribution function* $F(x) = P(X \leq x)$, which increases monotonically from $\lim_{x \rightarrow -\infty} F(x) = 0$ to $\lim_{x \rightarrow +\infty} F(x) = 1$. Its derivative is the *probability density function* $p(x) = \partial F(x)/\partial x$, with

$$p(x) \geq 0 \quad \forall x, \quad \int p(x) dx = 1, \quad P(x \in E) = \int_{x \in E} p(x) dx. \quad (38)$$

Unlike a probability, a density can be larger than one: a uniform density on the interval $[0, 0.1]$ has the value 10 everywhere on it. Only its integral over a region is a probability, and that integral is at most one. Two consequences appear again and again in exercises. For a Gaussian with $\mu = 0$ and $\sigma = 1$, the density at zero is $1/\sqrt{2\pi} \approx 0.40$, but the *probability* $P(x = 0)$ is 0, because the integral over a single point vanishes. And a statement like “ $p(x) \leq 1$ ” is not a property of densities at all.

The Gaussian distribution. The most important density is the normal or Gaussian distribution. In d dimensions, with mean vector μ and covariance matrix Σ ,

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right), \quad (39)$$

and in one dimension $p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{(x-\mu)^2}{2\sigma^2})$ with $\mu = E[x] = \int x p(x) dx$ and $\sigma^2 = E[(x - \mu)^2] = \int (x - \mu)^2 p(x) dx$. It earns its central role for two reasons. It describes many real phenomena, above all sensor noise, because by the central limit theorem a sum of many small independent effects is approximately Gaussian. And it is analytically convenient: parameter estimates have closed forms, the derivative is simply $\partial p/\partial x = -\frac{x-\mu}{\sigma^2} p(x)$, and the logarithm of the density is a quadratic function. Among all distributions with a given mean and variance, the Gaussian has the largest entropy $H = -\int p(x) \ln p(x) dx$, so it is the least committal choice when only these two moments are known.

Joint, conditional, and marginal probabilities. With two random variables A and B , the *joint probability* $P(A = a, B = b)$ is the probability that both take the given values together; it lives on a two-dimensional table (or, for densities, a two-dimensional surface). The *conditional probability* $P(A = a | B = b)$ is the probability of a once b is known to have happened; it is a one-dimensional slice of the joint, renormalized so that it sums to one. Three rules connect them:

$$\text{marginalization: } P(A = a) = \sum_b P(A = a, B = b), \quad p(x) = \int p(x, y) dy, \quad (40)$$

$$\text{product rule: } P(A, B) = P(A | B) P(B) = P(B | A) P(A), \quad (41)$$

$$\text{independence: } P(A | B) = P(A) \iff P(A, B) = P(A) P(B). \quad (42)$$

Marginalization removes a variable by summing or integrating it out; for a density $p(x_1, x_2, x_3)$, both $p(x_2, x_3) = \int p(x) dx_1$ and $p(x_1) = \iint p(x) dx_2 dx_3$ are marginalizations, while rewriting $p(x) = p(x_1 | x_2, x_3) p(x_2 | x_3) p(x_3)$ is a factorization with the product rule and $p(x) = p(x_1)p(x_2)p(x_3)$ is an independence assumption. For independent variables the joint table is the outer product of the marginals: with $P(a_1) = 0.2$, $P(a_2) = 0.8$ and $P(b_1) = P(b_2) = 0.5$ every entry is $P(a) \cdot 0.5$, so both rows of the table read (0.1, 0.4).

Bayes' theorem. The two factorizations in Equation (41) describe the same joint probability, so they must be equal: $P(A | B) P(B) = P(B | A) P(A)$. Dividing by $P(B)$ gives

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}, \quad p(x | y) = \frac{p(y | x) p(x)}{p(y)}. \quad (43)$$

The theorem inverts a conditional: it computes the probability of a cause given an effect from the probability of the effect given the cause, which is usually what we can model. Its four terms have fixed names (Table 15), which are asked for directly.

Term	Name	Meaning in classification
$P(\omega x)$	posterior	belief in class ω after seeing x
$p(x \omega)$	likelihood	how probable the observation is if the class is ω ; read as a function of ω it is the likelihood <i>of the class</i>
$P(\omega)$	prior	belief in ω before any observation
$p(x)$	evidence	normalizer $\sum_j p(x \omega_j) P(\omega_j)$, the same for all classes

Table 15: The terms of Bayes' theorem $P(\omega | x) = p(x | \omega)P(\omega)/p(x)$. In $P(C = c | X = x)$ the quantity is the posterior of the class C , not of the observation X .

4.2 Bayesian Decision Theory

Does a given picture show a tiger? Without looking at it, the only available knowledge is how common tigers are. If the picture was taken in an Indian jungle the answer “tiger” is more plausible than for a picture from Antarctica. This is the *prior* $P(\omega)$: knowledge that does not come from the current observation but from experience, experts, or given information, and that is often simply assumed uniform. With the two states $\Omega = \{\omega_1, \omega_2\} = \{\text{tiger}, \text{no tiger}\}$ and $P(\omega_1) + P(\omega_2) = 1$, the best decision *before* observation is

$$\text{select } \omega_1 \text{ if } P(\omega_1) > P(\omega_2), \text{ otherwise } \omega_2. \quad (44)$$

This rule always gives the same answer, whatever the picture shows. Once the picture itself or features x extracted from it are available, the decision should be based on the *posterior* instead: select ω_1 if $P(\omega_1 | x) > P(\omega_2 | x)$. Discriminative methods, such as the neural networks of Section 2 with a softmax output, model this posterior directly. Generative methods model likelihood and prior and obtain the posterior with Bayes' theorem.

MAP, ML, and naive Bayes. For any number of classes the rule generalizes to picking the class with the largest posterior. Since the evidence $p(x)$ does not depend on ω , it can be dropped:

$$\omega^* = \arg \max_{\omega} P(\omega | x) = \arg \max_{\omega} \frac{p(x | \omega) P(\omega)}{p(x)} = \arg \max_{\omega} p(x | \omega) P(\omega). \quad (45)$$

This is the *maximum a posteriori* (MAP) estimator. If the prior is uniform, all classes are equally likely beforehand and $P(\omega)$ drops out as well, leaving the *maximum likelihood* (ML) estimator $\omega^* = \arg \max_{\omega} p(x | \omega)$. ML and MAP therefore coincide exactly when the prior is uniform. When x consists of F features, the likelihood $p(x | \omega)$ is a F -dimensional density that is hard to model. *Naive Bayes* assumes that the features are conditionally independent given the class,

$$p(x | \omega) = \prod_{i=1}^F p(x_i | \omega), \quad (46)$$

so that only F one-dimensional densities per class have to be estimated. The assumption is rarely true, but the resulting classifier often works well because the decision only needs the correct *ranking* of the classes.

Working with logarithms. Products of many small probabilities underflow numerically, and priors and likelihoods are easier to compare as sums. Because the logarithm is strictly increasing, taking it does not change the arg max. A typical exam situation: a pixel is foreground ($C = 1$) or background ($C = 0$), a model supplies $p(x | C = c)$, and foreground covers on average 10% of the image, so $P(C = 1) = 0.1$ and $P(C = 0) = 0.9$. The MAP rule labels a pixel as background if

$$\begin{aligned} 0.9 p(x | C = 0) > 0.1 p(x | C = 1) &\iff 9 p(x | C = 0) > p(x | C = 1) \\ &\iff \ln p(x | C = 0) - \ln p(x | C = 1) > \ln 0.1 - \ln 0.9. \end{aligned} \quad (47)$$

All three forms are the same rule. Comparing only the priors ignores the pixel, and comparing only the likelihoods is the ML rule, not MAP.

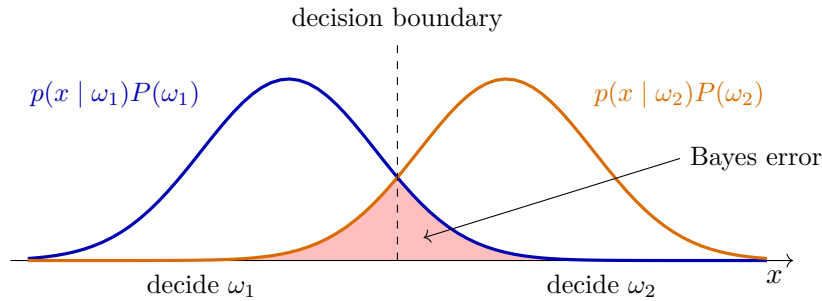


Figure 23: Two classes with equal priors and Gaussian likelihoods. The MAP rule switches class where the weighted likelihoods intersect. The shaded area is the Bayes error of Equation (50): the probability mass of each class that falls on the wrong side. Increasing $P(\omega_2)$ raises the orange curve and moves the boundary toward ω_1 .

Loss and risk. Not all mistakes are equally bad. Missing a melanoma is far worse than sending a harmless lesion for a second inspection. Bayesian decision theory captures this with a *loss* $\lambda(\alpha_i | \omega_j)$, the cost of taking action α_i when the true state is ω_j . Since the true state is unknown, each action is judged by its expected loss under the posterior, the *conditional risk*

$$R(\alpha_i | x) = \sum_j \lambda(\alpha_i | \omega_j) P(\omega_j | x), \quad \alpha^* = \arg \min_i R(\alpha_i | x). \quad (48)$$

The posterior says what is likely, the loss says what mistakes cost, and the decision rule combines both. MAP is the special case of the 0/1 loss ($\lambda = 0$ for the correct class, 1 otherwise): then $R(\alpha_i | x) = \sum_{j \neq i} P(\omega_j | x) = 1 - P(\omega_i | x)$, and minimizing the risk is maximizing the posterior.

Two worked cases show how to compute with Equation (48). First, a diagnostic system with states ω_1 (ill), ω_2 (healthy), actions α_1 (treat) and α_2 (do nothing), losses $\lambda(\alpha_1 | \omega_1) = 18$, $\lambda(\alpha_1 | \omega_2) = 27$, $\lambda(\alpha_2 | \omega_1) = 57$, $\lambda(\alpha_2 | \omega_2) = 0$, and $P(\omega_1 | x) = 0.75$. The expected risk of treating is

$$R(\alpha_1 | x) = 18 \cdot 0.75 + 27 \cdot 0.25 = 13.50 + 6.75 = 20.25, \quad (49)$$

and doing nothing would cost $57 \cdot 0.75 + 0 \cdot 0.25 = 42.75$, so treatment is chosen. The only care needed is to read the loss table correctly: fix the action (one column of the table), then sum over *all* true states, each weighted by its posterior. The risk of an action is never a single table entry times a single posterior. Second, a binary triage with classes melanoma M and benign B , zero cost for correct decisions, and a false negative five times as costly as a false positive, $\lambda(\text{FN}) = 5\lambda(\text{FP})$.

The risks are $R(\text{refer} \mid x) = \lambda(\text{FP})P(B \mid x)$ and $R(\text{dismiss} \mid x) = \lambda(\text{FN})P(M \mid x)$. Referring is better when $\lambda(\text{FP})(1 - P(M \mid x)) < 5\lambda(\text{FP})P(M \mid x)$, that is, when $P(M \mid x) > 1/6$. The asymmetric cost lowers the decision threshold from 0.5 (MAP) to 1/6 and shifts the decision boundary toward the benign class, trading more false alarms for fewer missed melanomas. The cost matrix is a policy choice made with domain experts; a classifier does not learn it from data.

The Bayes error. Even the optimal classifier makes mistakes where the classes overlap (Figure 23). For an observation x the MAP rule picks the class with the larger posterior, so the probability of being wrong at x is the smaller posterior. Averaging over all observations gives the expected error of the binary MAP classifier,

$$P(\text{error}) = \int_{-\infty}^{\infty} \min[P(\omega_1 \mid x), P(\omega_2 \mid x)] p(x) dx. \quad (50)$$

With max instead of min the same integral is the probability of a *correct* decision. If $P(\omega_1 \mid x) > P(\omega_2 \mid x)$ holds for every x , the classifier always answers ω_1 and is wrong exactly when the true class is ω_2 ; the integral becomes $\int P(\omega_2 \mid x)p(x) dx = P(\omega_2)$ by marginalization. The expression $P(\omega_2 \mid x)$ alone is the error at one particular x , not the expected error.

4.3 Discriminant Functions

Classification rules can be written in one uniform format. A classifier is given by one *discriminant function* $g_i(x)$ per class (or action), and assigns x to class ω_i if

$$g_i(x) > g_j(x) \quad \forall j \neq i. \quad (51)$$

The MAP rule is the choice $g_i(x) = P(\omega_i \mid x)$. Only the ranking of the values matters, not the values themselves. Hence any replacement $g_i \mapsto f(g_i)$ with a strictly increasing function f that is the same for all classes gives the same decisions: multiplying by a positive constant, adding a class-independent term, taking a logarithm or square root of a positive function. Dividing by the evidence $p(x)$ or dropping it is therefore allowed, and the following are equivalent MAP discriminants:

$$g_i(x) = P(\omega_i \mid x), \quad g_i(x) = p(x \mid \omega_i) P(\omega_i), \quad g_i(x) = \ln p(x \mid \omega_i) + \ln P(\omega_i). \quad (52)$$

A decreasing function reverses the ranking: $1/g_i(x)$ or $-g_i(x)$ turns the best class into the worst and is not a valid replacement. Whether a simplified function is still valid depends on which terms are really constant across classes. Table 16 collects the reasoning that the typical true/false questions require.

Gaussian likelihoods. If each class-conditional density is a Gaussian $\mathcal{N}(\mu_i, \Sigma_i)$, inserting Equation (39) into the logarithmic discriminant gives

$$g_i(x) = -\frac{1}{2}(x - \mu_i)^\top \Sigma_i^{-1}(x - \mu_i) - \frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln|\Sigma_i| + \ln P(\omega_i). \quad (53)$$

The first term is the squared *Mahalanobis distance* from x to the class mean, a distance that measures deviation in units of the class spread. The second term is the same for all classes and can always be dropped. What remains depends on how the covariances relate, and three cases are distinguished.

Case 1, $\Sigma_i = \sigma^2 I$. All classes are round blobs of equal size. The discriminant reduces to $g_i(x) = -\|x - \mu_i\|^2 / (2\sigma^2) + \ln P(\omega_i)$. Expanding the square, the term $x^\top x$ is common to all classes and cancels, so g_i is linear in x . With equal priors this is the *minimum-distance classifier*: assign x to the nearest mean. The boundary between two classes is the hyperplane perpendicular to $\mu_i - \mu_j$ through the midpoint, shifted away from the more probable class if the priors differ.

Candidate	Valid MAP discriminant?
$\frac{P(\omega_i x), p(x \omega_i)P(\omega_i)}{\sqrt{p(x \omega_i)P(\omega_i)}}$	always (monotonic transforms of the posterior)
$p(x \omega_i)$	only if all priors are equal (then it is ML = MAP)
$p(x \omega_i)/p(x)$	only if all priors are equal, since it equals $P(\omega_i x)/P(\omega_i)$
$P(\omega_i)$	only if all likelihoods are identical, $p(x \omega_i) = p(x \omega_j)$; the same mean with different covariances is not enough
constants such as $g_1 = 1, g_2 = 2$	only if they rank the classes like the posteriors for every x , e.g. identical likelihoods and $P(\omega_2) > P(\omega_1)$
$1/g_i(x)$	no, the ranking is reversed

Table 16: Checking candidate discriminant functions. A function is valid if it induces the same ranking of the classes as the posterior for every observation x .

Case 2, $\Sigma_i = \Sigma$ shared. The classes have the same, possibly elongated and tilted, shape. The quadratic term $x^\top \Sigma^{-1} x$ is again common and cancels, leaving a *linear machine*

$$g_i(x) = w_i^\top x + w_{i0}, \quad w_i = \Sigma^{-1} \mu_i, \quad w_{i0} = -\frac{1}{2} \mu_i^\top \Sigma^{-1} \mu_i + \ln P(\omega_i). \quad (54)$$

The decision boundary $g_i(x) = g_j(x)$ is the hyperplane $w^\top (x - x_0) = 0$ with $w = \Sigma^{-1}(\mu_i - \mu_j)$ and

$$x_0 = \frac{1}{2}(\mu_i + \mu_j) - \frac{\ln(P(\omega_i)/P(\omega_j))}{(\mu_i - \mu_j)^\top \Sigma^{-1}(\mu_i - \mu_j)} (\mu_i - \mu_j). \quad (55)$$

With equal priors it passes through the midpoint of the means, but it is in general not perpendicular to $\mu_i - \mu_j$, because Σ^{-1} tilts w .

Case 3, arbitrary Σ_i . Nothing cancels except the constant, and the discriminant is quadratic,

$$g_i(x) = x^\top W_i x + w_i^\top x + w_{i0}, \quad W_i = -\frac{1}{2} \Sigma_i^{-1}, \quad w_i = \Sigma_i^{-1} \mu_i, \\ w_{i0} = -\frac{1}{2} \mu_i^\top \Sigma_i^{-1} \mu_i - \frac{1}{2} \ln |\Sigma_i| + \ln P(\omega_i). \quad (56)$$

In the two-class case the boundary is a *hyperquadric*: a curve such as an ellipse, parabola, or hyperbola in 2D. Figure 24 contrasts cases 2 and 3. The logic runs only one way: Gaussians with a shared covariance imply a linear boundary, but a linear optimal boundary does not imply that the classes are Gaussian.

Everything above assumes that the class-conditional densities are known. In practice their shape is at best known in principle (for example “Gaussian”) while the parameters are not, and often considerably less is known. The rest of the section is about estimating these densities from data, and about methods that are as insensitive as possible to wrong assumptions.

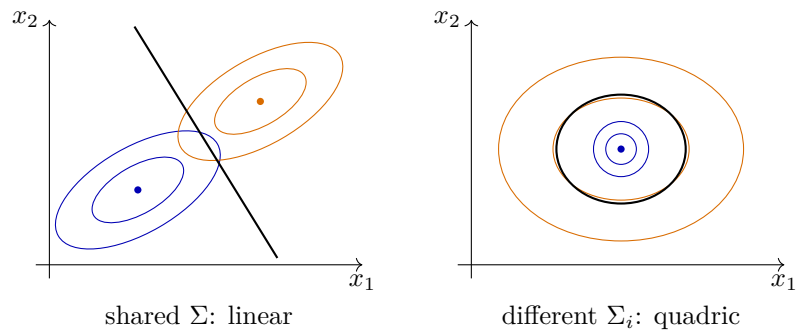


Figure 24: Decision boundaries (black) for two Gaussian classes, drawn as contours of equal density. Left: with a shared covariance the quadratic terms cancel and the boundary is a straight line, tilted by Σ^{-1} . Right: a narrow class inside a wide one with the same mean gives a closed, elliptic boundary; the narrow class wins only near the center.

4.4 Non-Parametric Density Estimation

A Bayes-optimal classifier is possible if the prior and the likelihood are known. Priors are usually given or assumed uniform. Likelihoods must be estimated from *training data*: measurements for which the target value, for example the class label, is known. The question the estimate answers is: how often does a feature value occur given the class? There are two ways to answer it. Non-parametric methods estimate the density directly from the samples, with no assumption about its shape. Parametric methods assume a functional form with a few parameters and fit those.

Histograms. The simplest estimate divides the feature range into bins and counts. The probability of a bin is the fraction of samples that fall into it, a frequentist interpretation of probability as the ratio of positive events to possible events; dividing by the bin volume turns this into a density. The estimate is *consistent*: as the number of samples $|D| \rightarrow \infty$ (with suitably shrinking bins) it converges to the true distribution. Two problems limit it. The bin width must be chosen: many narrow bins give a fine resolution but need a lot of data, otherwise most bins are empty or noisy; few wide bins are stable but blur the shape. And the number of cells explodes with the dimension, the *curse of dimensionality*: with n bins per axis, d dimensions need n^d cells, so the number of samples required to fill them grows exponentially. Ten bins in ten dimensions are already 10^{10} cells. Empty bins produce zero probabilities, which a product of likelihoods cannot recover from; the Laplace correction against this is covered in Appendix D.

Parzen windows. A histogram fixes the bins in advance and asks which bin a sample falls into. Parzen windows turn this around and center a window on the query point x itself. Let the window be a hypercube of side length h_n and volume $V_n = h_n^d$, and let

$$\varphi(u) = \begin{cases} 1 & |u_j| \leq 0.5 \quad (j = 1, \dots, d), \\ 0 & \text{otherwise} \end{cases} \quad (57)$$

be the unit-cube indicator. Then $k_n = \sum_{i=1}^n \varphi(\frac{x-x_i}{h_n})$ counts the samples inside the window around x , and the density estimate is “fraction of samples per volume”:

$$p_n(x) = \frac{k_n/n}{V_n} = \frac{1}{n} \sum_{i=1}^n \frac{1}{h_n^d} \varphi\left(\frac{x-x_i}{h_n}\right). \quad (58)$$

Read from the right, the formula says: put a small bump of volume one around every sample and average the bumps. If the window function is itself a density, $\varphi(u) \geq 0$ and $\int \varphi(u) du = 1$, then p_n is a valid density. The box is only one choice; a Gaussian window gives a smooth estimate (Figure 25). The width h_n plays the role of the bin width: too small and the estimate is a spiky sum of isolated peaks, too large and all structure is smoothed away. A one-dimensional example with the box window: the samples are $\{1, 2, 2.5, 6\}$, $h = 2$, and we evaluate at $x = 2$. The window covers $|x - x_i| \leq 1$, which contains the three samples 1, 2 and 2.5, so $p(2) = 3/(4 \cdot 2) = 0.375$.

k -nearest-neighbor estimate. Parzen windows fix the volume and count the samples inside. The k -nearest-neighbor (k-NN) estimate fixes the count instead: grow the window around x until it contains exactly k samples, and use its volume $V_k(x)$,

$$p(x) \approx \frac{k}{n V_k(x)}. \quad (59)$$

In dense regions the window stays small, in sparse regions it grows, so the resolution adapts to the data. If the k neighbors contain k_i samples of class ω_i , the same reasoning gives the posterior estimate $P(\omega_i | x) \approx k_i/k$, which is the k-NN classifier: vote among the k nearest training samples.

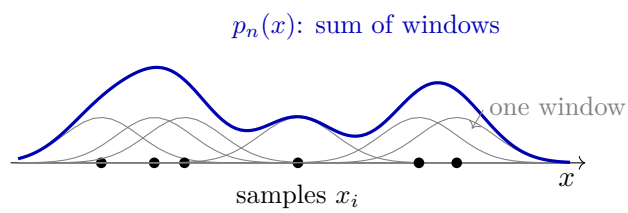


Figure 25: Parzen window estimate with a Gaussian window. Each sample (dot) contributes one bump of width h (gray); their average is the density estimate (blue, drawn unnormalized). Where samples cluster the bumps add up to a high density.

4.5 Parametric Estimation

Non-parametric methods are very general and make no assumption about the true distribution, but they need a lot of data and must store all of it. Often the general form of the density is known, or can be approximated well enough, and can be parameterized, for example as a Gaussian. This reformulates a hopeless problem, estimating the density at every point, into an easy one, estimating a few numbers such as a mean and a variance. Table 17 contrasts the families.

Method	Type	Key choice and weakness
Histogram	non-parametric	bin width; curse of dimensionality, empty bins
Parzen windows	non-parametric	window width h ; stores all samples
k -NN	non-parametric	k ; estimate is not smooth
Gaussian (ML)	parametric	assumed shape; wrong if data are multimodal
Gaussian mixture	parametric	number of components K

Table 17: Density estimators. Parametric methods have a fixed functional form with a fixed number of parameters. A piecewise-constant model $P(4i \leq x < 4(i+1)) = \theta_i$ with its 25 parameters θ_i is written in this parametric form, while histograms and Parzen windows are the standard non-parametric examples. Both Parzen windows and ML estimation yield a distribution over the data.

Maximum-likelihood estimation. Assume the class-conditional density has a known parametric form, for example $p(x | \omega_j) = p(x | \omega_j, \theta_j)$ with $\theta_j = (\mu_j, \Sigma_j)$ for a Gaussian. The training samples of each class are used to estimate its unknown parameter vector. If the samples $D = \{x_1, \dots, x_n\}$ are drawn independently, the probability of the whole training set under the model θ is the product

$$p(D | \theta) = \prod_{k=1}^n p(x_k | \theta), \quad \hat{\theta} = \arg \max_{\theta} p(D | \theta). \quad (60)$$

Viewed as a function of θ , $p(D | \theta)$ is the *likelihood of the model*, and the ML estimate is the model that makes the observed data most probable: the one that best explains them. In practice one maximizes the *log-likelihood* $\ell(\theta) = \sum_k \ln p(x_k | \theta)$, which has the same maximum and turns the product into a sum, by setting its derivative to zero. More generally, the maximizer stays the same under any strictly increasing transformation (such as $\ln$), when a term that does not depend on θ is added or dropped, and when the function is multiplied by a positive constant. Dropping a factor that *does* depend on θ moves the maximizer. The recipe for every exercise of this kind is: write down $\ell(\theta)$, differentiate, set to zero, solve.

For a Gaussian, $\ell(\mu) = \sum_k [-\frac{1}{2} \ln(2\pi\sigma^2) - \frac{(x_k - \mu)^2}{2\sigma^2}]$ and $\partial\ell/\partial\mu = \sum_k (x_k - \mu)/\sigma^2 = 0$ gives the sample mean. The same computation for the covariance gives the sample covariance:

$$\hat{\mu} = \frac{1}{n} \sum_{k=1}^n x_k, \quad \hat{C} = \frac{1}{n-1} \sum_{k=1}^n (x_k - \hat{\mu})(x_k - \hat{\mu})^\top. \quad (61)$$

The exact ML solution divides by n ; dividing by $n-1$ removes its small bias, because $\hat{\mu}$ was fitted to the same data. The difference vanishes for large n . With $n=2$ samples the estimate is essentially arbitrary; with 10, 100, and 1000 samples the fitted Gaussian approaches the true one. A million samples give a very precise estimate, but only of the assumed model: if the data are

not Gaussian (for example bimodal), more samples do not fix the wrong shape. How bias and variance of an estimator combine into its mean squared error is shown in Appendix E.

The same recipe works for discrete models. For a count of errors per time interval modeled as Poisson, $P(k) = \lambda^k e^{-\lambda}/k!$, with measurements $X = \{2, 3, 8, 6, 4\}$:

$$\ell(\lambda) = \sum_i [k_i \ln \lambda - \lambda - \ln k_i!], \quad \frac{d\ell}{d\lambda} = \frac{\sum_i k_i}{\lambda} - n = 0 \Rightarrow \hat{\lambda} = \frac{1}{n} \sum_i k_i = \frac{23}{5} = 4.60. \quad (62)$$

For the day k of the first production failure modeled as geometric, $P(k) = (1-p)^{k-1}p$, with $D = \{8, 6, 10, 9, 4\}$:

$$\ell(p) = \sum_i [(k_i - 1) \ln(1-p) + \ln p], \quad \frac{d\ell}{dp} = -\frac{\sum_i (k_i - 1)}{1-p} + \frac{n}{p} = 0 \Rightarrow \hat{p} = \frac{n}{\sum_i k_i} = \frac{5}{37} \approx 0.14. \quad (63)$$

The step from the derivative to the result is $n(1-p) = p \sum_i (k_i - 1) = p \sum_i k_i - pn$, so $n = p \sum_i k_i$. The estimate is one over the mean waiting time, which matches intuition: failures every 7.4 days on average mean a daily failure probability of about $1/7.4$. A quick plausibility check catches the most common slip: p is a probability, so an answer outside $[0, 1]$, such as the mean 7.4 itself, cannot be right. Always insert the model exactly as stated; the formula for $\hat{p}$ changes if k is defined differently.

Bayesian estimation. ML treats the parameter as a fixed unknown number and picks one value. Bayesian estimation treats the parameter itself as a random variable. Prior knowledge enters in three forms: the shape of the unknown density $p(x | \theta)$, a prior density $p(\theta)$ that expresses what is known about the parameter values beforehand, and the training samples. The training data turn the prior into a posterior with Bayes' theorem,

$$p(\theta | D) = \frac{p(D | \theta) p(\theta)}{p(D)}, \quad p(x | D) = \int p(x | \theta) p(\theta | D) d\theta, \quad (64)$$

and the density used for classification is the *predictive* density $p(x | D)$: an average over all models, each weighted by how well it is supported by the data. The class posterior becomes $P(\omega_i | x, D) = p(x | \omega_i, D)P(\omega_i) / \sum_j p(x | \omega_j, D)P(\omega_j)$. The hope is that $p(\theta | D)$ has a sharp maximum at the true parameter value. When it does, the integral is dominated by that single value and the Bayesian result equals the ML (or MAP) plug-in estimate; this happens with many training samples, and with a flat prior the peak lies exactly at the ML estimate.

A discrete example shows the three strategies side by side. Two candidate models θ_1, θ_2 have likelihoods $P(D | \theta_1) = 2 \cdot 10^{-6}$ and $P(D | \theta_2) = 10^{-6}$ on the data, priors $P(\theta_1) = 1/4$ and $P(\theta_2) = 3/4$, and predict a new sample with $P(x | \theta_1) = 1/4$, $P(x | \theta_2) = 1/3$.

- ML selects the model with the larger likelihood, θ_1 , so $P(x | D) = 1/4$.
- MAP compares $P(D | \theta)P(\theta)$: $2 \cdot 10^{-6} \cdot \frac{1}{4} = 0.5 \cdot 10^{-6}$ against $10^{-6} \cdot \frac{3}{4} = 0.75 \cdot 10^{-6}$. It selects θ_2 , so $P(x | D) = 1/3$.
- Bayesian estimation normalizes these two numbers into the posteriors $P(\theta_1 | D) = 0.5/1.25 = 0.4$ and $P(\theta_2 | D) = 0.6$, and averages the predictions: $P(x | D) = 0.4 \cdot \frac{1}{4} + 0.6 \cdot \frac{1}{3} = 0.1 + 0.2 = 3/10$.

The same normalization answers the plain posterior question: with $P(D | \Omega_1) = 0.9$, $P(D | \Omega_2) = 0.43$ and priors 0.72, 0.28,

$$P(\Omega_1 | D) = \frac{0.9 \cdot 0.72}{0.9 \cdot 0.72 + 0.43 \cdot 0.28} = \frac{0.648}{0.7684} \approx 0.8433. \quad (65)$$

	ML	MAP	Bayesian
Parameter is	fixed unknown	fixed unknown	random variable
Maximizes	$p(D \mid \theta)$	$p(D \mid \theta) p(\theta)$	nothing; integrates over θ
Result	one $\hat{\theta}$	one $\hat{\theta}$	posterior $p(\theta \mid D)$
Prediction	$p(x \mid \hat{\theta})$	$p(x \mid \hat{\theta})$	$\int p(x \mid \theta) p(\theta \mid D) d\theta$
Coincides with ML	—	uniform prior	sharp posterior at $\hat{\theta}_{\text{ML}}$

Table 18: Three ways of estimating parameters from training data D . ML explains the data, MAP adds prior knowledge, and Bayesian estimation keeps the full uncertainty about the parameters.

Table 18 summarizes the differences. Instead of the density function itself, all parametric methods search for its parameters. When one Gaussian is too rigid, a mixture of several Gaussians fitted with the EM algorithm is the usual parametric extension; it is summarized together with probability calibration in Appendix D.

4.6 Markov Random Fields

So far the prior was one number per class, the same for every pixel. Images contain much more prior knowledge than that. In the real world, dependencies occur between neighboring objects, not between objects far apart: buildings are parallel to the street next to them, pixels of one land-use class form larger coherent areas, and road segments connect to long roads and networks. A noisy pixel that looks like background in the middle of a road is probably road. A *Markov random field* (MRF) turns this kind of knowledge into a prior over whole label images, and Bayes' theorem combines it with the noisy observations.

The name describes the model. A *field* is a spatial (2D or 3D) arrangement of variables. *Random* means that every variable takes its value randomly. In a general random field each variable may depend on all others; in a *Markov* random field a variable depends only on its neighbors. Given the values of its neighbors, a variable is independent of the rest of the field. The variables, called primitives, can be nodes of a general graph (graph-based MRF, for example regions or objects) or the pixels of a raster (raster-based MRF). Table 19 lists the vocabulary.

Term	Meaning
Neighborhood relation	a line connecting two primitives
Neighborhood of s	all primitives with a neighborhood relation to s , written ∂s
Neighborhood system	set of all neighborhoods of the field, e.g. the 4-neighborhood on a raster
Clique	subset of primitives that are all neighbors of each other, e.g. single pixels and horizontal or vertical pixel pairs
Maximal clique	clique that cannot be enlarged by another primitive
Potential H_c	energy as a function of the states of the primitives of clique c ; low energy means a likely configuration

Table 19: Vocabulary of Markov random fields.

From potentials to probabilities. An MRF is specified by its potentials. The *energy* of a complete configuration x is the sum of the potentials of all cliques C , and the probability follows the Gibbs distribution:

$$H(x) = \sum_{c \in C} H_c(x), \quad p(x) = \frac{1}{Z} \exp\left\{-\frac{H(x)}{T}\right\}, \quad Z = \sum_{x \in \Omega} \exp\left\{-\frac{H(x)}{T}\right\}. \quad (66)$$

Low energy means high probability, and a sum of energies becomes a product of factors, one per clique. The temperature T controls how peaked the distribution is and is used in simulated annealing. The normalization constant Z sums over the whole configuration space Ω , which for an image of N binary pixels has 2^N elements and cannot be computed. The Markov property rescues us: the *local* conditional probability of one primitive s given its neighbors involves only the cliques C_s that contain s ,

$$H_s(x_s \mid x_{\partial s}) = \sum_{c \in C_s} H_c(x_s, x_{\partial s}), \quad p(x_s \mid x_{\partial s}) = \frac{1}{Z_s} \exp\left\{-\frac{H_s(x_s \mid x_{\partial s})}{T}\right\}, \quad (67)$$

where the local normalizer Z_s sums only over the few states Λ_s of the single primitive s .

Example: horizontal lines. Suppose the world contains preferably horizontal one-pixel-wide lines, and line interruptions and line ends are rare. The model uses the pixels as primitives of a raster MRF, each with state L (line, black) or N (no line, white), the 4-neighborhood system,

and horizontal and vertical two-pixel cliques. The potentials in Figure 26 make the unwanted configurations expensive: a horizontal pair with one line and one non-line pixel, which is a line end or an interruption, costs α ; a vertical pair of two line pixels, which would make the line thicker than one pixel, costs β ; every other pair costs nothing.

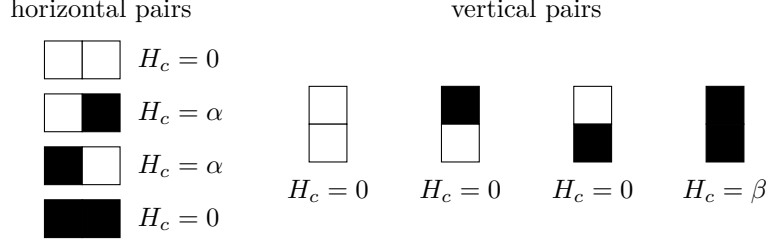


Figure 26: Potentials of the two-pixel cliques for the horizontal-line model (black: line L , white: no line N), for example with $\alpha = \beta = 1$. Horizontal changes between L and N (line ends) and vertically stacked line pixels (thick lines) are penalized.

The local conditional probability follows by adding the potentials of the four cliques that contain the pixel s , once for each of its possible states. Take $\alpha = \beta = T = 1$.

- Left and right neighbors are L , top and bottom are N (a gap in a line). If $s = N$, both horizontal pairs are L – N changes, so $H_s(N) = 2\alpha = 2$; the vertical pairs cost nothing. If $s = L$, the horizontal pairs are L – L and the vertical pairs have one L , so $H_s(L) = 0$. Hence $p(L | \partial s) = 1/(1 + e^{-2}) \approx 0.88$: the gap is probably filled.
- Only the top neighbor is L , the others are N . If $s = N$ nothing is penalized, $H_s(N) = 0$. If $s = L$, two horizontal changes and one vertical L – L pair give $H_s(L) = 2\alpha + \beta = 3$, so $p(L | \partial s) = e^{-3}/(1 + e^{-3}) \approx 0.05$.

Estimating object parameters. The observations y are the noisy grey values, and the object parameters to be estimated are the hidden states ε (L or N) of all pixels. Bayes' theorem $p(\varepsilon | y) \propto p(y | \varepsilon)p(\varepsilon)$ applies locally, with the MRF as prior:

$$p(\varepsilon_s | \varepsilon_{\partial s}, y_s) \propto p(y_s | \varepsilon_s) p(\varepsilon_s | \varepsilon_{\partial s}), \quad p(y_s | \varepsilon_s) = \frac{1}{\sqrt{2\pi} \sigma_\varepsilon} \exp\left\{-\frac{(y_s - \mu_\varepsilon)^2}{2\sigma_\varepsilon^2}\right\}, \quad (68)$$

with, for example, $\mu_L = 90$, $\sigma_L = 10$ (lines are dark) and $\mu_N = 110$, $\sigma_N = 10$. The ML estimate uses only the likelihood and labels each pixel by its own grey value. The MAP estimate includes the MRF prior. Taking the negative logarithm turns Equation (68) into a sum of energies: a *unary* (association) term $(y_s - \mu_{\varepsilon_s})^2/(2\sigma^2)$ that ties each label to its own observation, and *pairwise* (interaction) terms H_c/T that tie neighboring labels together. This is the typical MRF of image processing: a grid of hidden labels, each connected to its observation and to its four neighbors. A worked pixel: $y_s = 105$ in the gap of case one above. Its unary energies are $15^2/200 = 1.125$ for L and $5^2/200 = 0.125$ for N , so ML says N . Adding the prior energies 0 and 2 gives 1.125 for L against 2.125 for N , so MAP says L and closes the gap.

Because every label depends on its neighbors, which are themselves being estimated, the MAP configuration is found iteratively. Simulated annealing visits the pixels repeatedly, draws each new state from its local posterior at temperature T , and lowers the temperature slowly, for example as $T_i = 0.7/\ln(i)$ over the iterations i . At high temperature the states change freely and the search can escape poor configurations; as $T \rightarrow 0$ the local posteriors become deterministic and the field freezes into a low-energy, high-posterior labeling. On noisy observations of horizontal lines, the ML result is a scatter of isolated line pixels and holes, while the MAP result recovers mostly continuous horizontal lines with few defects.

Estimating model parameters. The prior parameters α and β need not be guessed. Given training images with known “true” object parameters, the model can be adapted to a dataset: run the MAP estimation with the current parameters, measure the difference between estimate and truth, and let this evaluation control a simulated annealing search over the parameters themselves. In the line example, optimizing the parameters from $\alpha = \beta = 1.0$ to $\alpha = 2.2$, $\beta = 1.0$ penalizes interruptions more strongly and yields visibly cleaner lines. Conditional random fields, which condition the pairwise terms on the observations, and the factorization rules of graphical models in general are summarized in Appendix F.

The whole section rests on one principle. The best decision minimizes the expected risk, and for the error probability this means choosing the class of maximum posterior. The difficulty is almost never the decision rule but the probabilities it needs: likelihoods have to be estimated from limited data, either freely or through a parametric model, and priors, from class frequencies up to Markov random fields, supply the knowledge that the data alone cannot.

5 Convolutional Neural Networks

The multilayer networks of Section 2 accept any input vector, and that generality is exactly their weakness on images. An image is not an arbitrary list of numbers: neighboring pixels belong together, and the same pattern means the same thing wherever it appears. Convolutional neural networks (ConvNets, CNNs) build these two facts into the architecture. Each unit looks only at a small local patch, and the same weights are reused at every position. The result is a network with far fewer parameters that learns much more from the same amount of data.

This section first explains why fully connected networks are a poor fit for images and which property a good image network should have. It then builds the convolutional layer step by step in one dimension, where every quantity can be drawn and computed by hand, and extends it with channels, receptive fields, two-dimensional kernels, and down- and upsampling. A small experiment shows how much the convolutional structure helps. The section closes with the three standard tasks, image classification, object detection, and semantic segmentation, and the architectures that solve them.

5.1 Why Images Need Convolution

Three problems appear as soon as a fully connected network is applied to a raw image.

Size. A 224×224 RGB image has $224 \cdot 224 \cdot 3 = 150,528$ input values. Hidden layers are usually at least as large as the input, so a single fully connected hidden layer of the same size needs $150,528^2 \approx 2.3 \cdot 10^{10}$, about 22 billion, weights. Such a model is impossible to store comfortably and hopeless to train from a realistic number of examples.

Spatial structure is ignored. Nearby pixels are statistically related, but a fully connected network does not know which pixels are neighbors. If the pixels of every training and test image were permuted with one fixed random permutation, the network could be retrained and would reach exactly the same performance. All knowledge about image neighborhoods has to be learned from scratch, which wastes data.

No stability under transformations. An eye, a wheel, or an edge looks the same in the top-left corner and in the center. A fully connected network has separate weights for every position, so it must learn the appearance of each pattern again at every location where it can occur.

Convolutional networks answer all three problems with two design decisions.

- *Local connectivity:* each hidden unit is connected only to a small patch of the input, its *receptive field*.
- *Weight sharing:* the same weights are applied to every patch, so a pattern detector learned at one position works at all positions.

The number of parameters then depends only on the patch size, not on the image size, and the network processes every location in the same way.

What does “processes every location in the same way” mean precisely? Two notions describe how the output of a function f reacts when its input x is transformed by some t , for example a translation of the image. A function is *invariant* to t if the transformation does not change the output at all, and *equivariant* to t if the output undergoes the same transformation as the input:

$$\text{invariance: } f[t[x]] = f[x], \quad \text{equivariance: } f[t[x]] = t[f[x]]. \quad (69)$$

Which of the two is wanted depends on the task (Table 20). An image classifier should say “cat” whether the cat sits left or right, so the class should be invariant to translation. A segmentation, a label for every pixel, should instead move along with the object: if the input is shifted by ten pixels, the label map should shift by ten pixels as well, so segmentation should be equivariant.

Property	Definition	Typical task	Where it comes from
Equivariance	output shifts with the input	semantic segmentation, detection maps	convolution itself (shared weights at every position)
Invariance	output unchanged by the shift	image classification	pooling and global aggregation on top of equivariant layers

Table 20: Translation invariance and equivariance. Convolutional layers are equivariant; invariance is obtained by summarizing their outputs over positions.

Convolution is translation-equivariant by construction: since the same weights are applied everywhere, shifting the input shifts the output by the same amount (apart from effects at the border). Invariance is added later by operations that throw away position, such as pooling and a final global aggregation. A typical classification network is therefore equivariant in its early layers and increasingly invariant toward its output.

5.2 The Convolutional Layer

The convolutional layer is easiest to understand on a one-dimensional signal, for example one row of an image or a sampled time series. Every idea carries over to two dimensions unchanged. Let the input be a vector $x = (x_1, \dots, x_D)$. Each output z_i is a weighted sum of x_i and its immediate neighbors. With a *kernel* (or *filter*) $\omega = (\omega_1, \omega_2, \omega_3)^\top$ of size three,

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1} = \sum_{j=1}^3 \omega_j x_{i+j-2}. \quad (70)$$

The kernel slides along the input, and at every position the same three weights are used. Strictly speaking, Eq. (70) is a cross-correlation, because the kernel is not flipped as in the signal-processing definition of convolution (Appendix B). Since the weights are learned, the flip makes no difference, and deep learning calls both operations convolution. A kernel such as $\omega = (-1, 0, 1)$ computes a central difference, the same derivative filter used for edge detection (Appendix A); a convolutional network learns kernels like this from data instead of having them designed by hand.

At the borders the operation needs a convention: at the first and last position, Eq. (70) needs inputs x_0 and x_{D+1} that do not exist. Two conventions handle this. *Zero padding* treats positions beyond the end of the input as zero, so the output has the same length as the input. A *valid* convolution only computes outputs where the kernel lies completely inside the input, so a kernel of size K produces $D - K + 1$ outputs; the signal shrinks by $K - 1$ positions per layer.

Worked example. Take $x = (2, 1, 0, 3, 1)$, the kernel $\omega = (-1, 0, 1)$, and zero padding (Figure 27). Every output is $z_i = x_{i+1} - x_{i-1}$:

$$\begin{aligned} z_1 &= -0 + 1 = 1, & z_2 &= -2 + 0 = -2, & z_3 &= -1 + 3 = 2, \\ z_4 &= -0 + 1 = 1, & z_5 &= -3 + 0 = -3. \end{aligned}$$

The valid convolution keeps only $z_2, z_3, z_4 = (-2, 2, 1)$, three outputs from five inputs. Shifting the input one position to the right, $x' = (0, 2, 1, 0, 3)$, shifts the interior outputs by one position as well; this is the equivariance of the previous subsection in numbers.

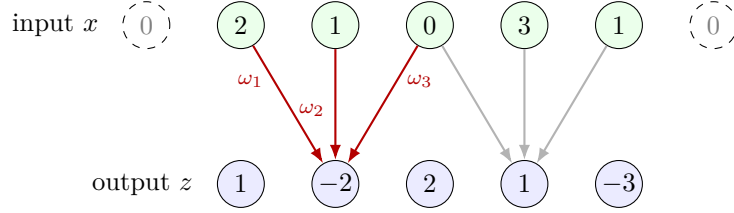


Figure 27: One-dimensional convolution with the kernel $(-1, 0, 1)$ and zero padding. The same three weights $\omega_1 = -1$, $\omega_2 = 0$, $\omega_3 = 1$ (red) produce z_2 and, shifted by two positions, z_4 (gray). Dashed units are the padded zeros.

Stride, kernel size, and dilation. Three hyperparameters change how the kernel is placed.

- The *stride* S moves the kernel by S positions between consecutive outputs. Stride 2 computes only every second output and halves the length of the signal. In the example above, stride 2 with zero padding keeps $z_1, z_3, z_5 = (1, 2, -3)$.
- The *kernel size* K sets how many inputs are combined. A larger kernel integrates information over a larger area, but a kernel of size 5 also needs five weights.
- The *dilation* d spaces the kernel entries d positions apart, which is the same as interspersing $d - 1$ zeros between the weights. A kernel of size 3 with dilation 2 covers five inputs with only three weights. Its effective size is $K_{\text{eff}} = d(K - 1) + 1$.

Together with the padding P (the number of zeros added on each side) they determine the output length. For input length W ,

$$W_{\text{out}} = \left\lfloor \frac{W - K_{\text{eff}} + 2P}{S} \right\rfloor + 1. \quad (71)$$

The same formula holds for the width and the height of an image separately. For example, a 3×3 kernel with $P = 1$ and $S = 1$ preserves a 224×224 image ($\frac{224-3+2}{1} + 1 = 224$); this “same” setting is the standard choice in modern networks. A 7×7 kernel with $P = 3$ and $S = 2$ gives $\left\lfloor \frac{224-7+6}{2} \right\rfloor + 1 = 111 + 1 = 112$, roughly halving the image. With $P = 0$, Eq. (71) reduces to the valid size $W - K + 1$ for $S = 1$.

So far the operation is purely linear. A convolutional layer turns it into a network layer in the same way as in Section 2: add a bias β and pass the result through an activation function $a[\cdot]$, usually ReLU:

$$h_i = a\left[\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}\right] = a\left[\beta + \sum_{j=1}^3 \omega_j x_{i+j-2}\right]. \quad (72)$$

In the worked example with $\beta = 0$, ReLU turns $z = (1, -2, 2, 1, -3)$ into $h = (1, 0, 2, 1, 0)$.

This layer is nothing new in principle. A fully connected layer computes $h = a[\beta + \Omega x]$ with a full weight matrix $\Omega \in \mathbb{R}^{D \times D}$ and a bias per output: D^2 weights and D biases. A convolutional layer computes exactly the same expression, but with a very special Ω . Most entries are zero, because each output only sees K inputs, and the nonzero entries repeat along the diagonal, because every output uses the same kernel. The whole layer has K weights and one bias, independent of D . Convolution is therefore a fully connected layer with two constraints: *sparse* connections and *tied* weights. The constraints are what make it efficient, and they are exactly what the exam answer “ConvNets use shared weights and units with local receptive fields” refers to.

Reading the hyperparameters off a weight matrix. The structure of Ω reveals all hyperparameters of a layer (Figure 28). The *number of nonzero entries per row* is the kernel size. The *horizontal shift from one row to the next* is the stride. The *spacing between nonzero entries within a row* is the dilation. The *number of rows* tells the padding: as many outputs as inputs (with the first row missing its left taps) means zero padding, fewer outputs and every row complete means a valid convolution.

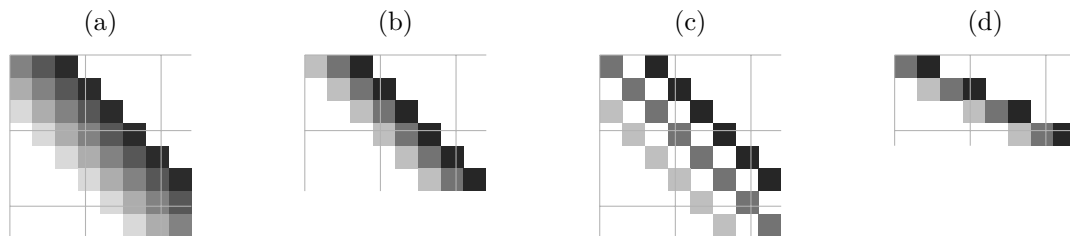


Figure 28: Weight matrices Ω of four one-dimensional convolutional layers with eight inputs (columns) and one row per output; shading marks the kernel weights $\omega_1, \omega_2, \dots$ from light to dark, white entries are zero. (a) Kernel size 5, stride 1, dilation 1, zero padding. (b) Kernel size 3, stride 1, dilation 1, valid (six outputs). (c) Kernel size 3, stride 1, dilation 2, zero padding. (d) Kernel size 3, stride 2, dilation 1, zero padding (four outputs).

The same reading works on a connectivity diagram in which every output is drawn with lines to its inputs: count the lines entering one output (kernel size), check how far the pattern moves between neighboring outputs (stride), check whether the connected inputs are adjacent or spaced (dilation), and compare the number of outputs with the number of inputs (padding or valid). In Figure 28(c), for example, h_1 is connected to x_1 and x_3 only, because its left tap would need x_{-1} ; the missing tap is the signature of zero padding, and the gap of one position between taps is the signature of dilation 2.

5.3 Channels, Receptive Fields, and Why Convolution Helps

One kernel per layer is not enough, and one layer is not enough either. The layer is first widened with channels, then the network is deepened until its units see large parts of the input, and finally the complete design is tested against a fully connected network.

A single convolution followed by ReLU averages its inputs and then discards all negative responses, so one such output necessarily loses information about the input. The remedy is to apply *several* convolutions with different kernels in parallel and to keep all their outputs. Each output signal is a *channel*, also called a *feature map*. With two kernels, one might respond to rising and the other to falling intensity; together they keep what a single one would lose.

The next layer then receives a multi-channel input, and its kernels must span all input channels. A kernel for an input with C_i channels has size $C_i \times K$: it takes a weighted sum over K neighboring positions *and* over all C_i channels, and produces one output channel. A layer with C_o output channels uses C_o such kernels, so its weights form a tensor $\Omega \in \mathbb{R}^{C_i \times C_o \times K}$ and

its biases a vector $\beta \in \mathbb{R}^{C_o}$. For a multi-channel input $X \in \mathbb{R}^{D \times C_i}$ the output of channel k at position i is

$$h_i^{(k)} = a \left[\beta_k + \sum_{c=1}^{C_i} \sum_{j=1}^K \omega_{c,k,j} x_{i+j-2}^{(c)} \right]. \quad (73)$$

Every output position therefore sees a small neighborhood of the input in *all* channels. In two dimensions this is the familiar picture: an input tensor of size $H \times W \times C$ is mapped to an output of size $H \times W \times C_o$ (with same padding), one feature map per kernel.

The parameter counts follow directly from the tensor shapes (Table 21). One output channel of a one-dimensional layer needs $C_i K$ weights and one bias; C_o output channels need $C_i C_o K$ weights and C_o biases. None of these numbers depends on the length of the input.

Layer	Weights	Biases
1D convolution, C_i inputs, one output channel	$C_i K$	1
1D convolution, $C_i \rightarrow C_o$ channels	$C_i C_o K$	C_o
2D convolution, $K \times K$ kernel, $C_i \rightarrow C_o$	$C_i C_o K^2$	C_o
1×1 convolution, $C_i \rightarrow C_o$	$C_i C_o$	C_o
Fully connected, $D_i \rightarrow D_o$ units	$D_i D_o$	D_o

Table 21: Parameter counts of convolutional and fully connected layers. The convolutional counts are independent of the spatial size of the input.

Two quick examples fix the formulas. A 3×3 convolution from an RGB image ($C_i = 3$) to $C_o = 64$ feature maps has $3 \cdot 64 \cdot 9 + 64 = 1,792$ parameters, whether the image has 32×32 or 1000×1000 pixels. A fully connected layer between the same $224 \times 224 \times 3$ input and a $224 \times 224 \times 64$ output would need about $150,528 \cdot 3.2 \cdot 10^6 \approx 4.8 \cdot 10^{11}$ weights. The difference of more than eight orders of magnitude is the whole point of convolution. The formulas also show that parameters and features are decoupled: a larger kernel K adds weights without adding a single output channel. In a fully connected layer with a fixed input, extra weights can only come with extra output units.

Channels widen what a layer computes at one position; depth widens how much of the input a unit sees. More filters in one layer therefore raise its output dimensionality and the number of distinct features it can respond to, but they do not make those features more abstract; abstraction grows with depth. A hidden unit in the first layer sees K inputs. A unit in the second layer sees K units of the first layer, each of which sees K inputs, so the region of the input that can influence it has grown. The *receptive field* of a unit is this region: the set of input positions that can change its value. Figure 29 shows the growth for kernel size 3 and stride 1: the receptive field is 3 after one layer, 5 after two, 7 after three, and in general $2L + 1$ after L layers.

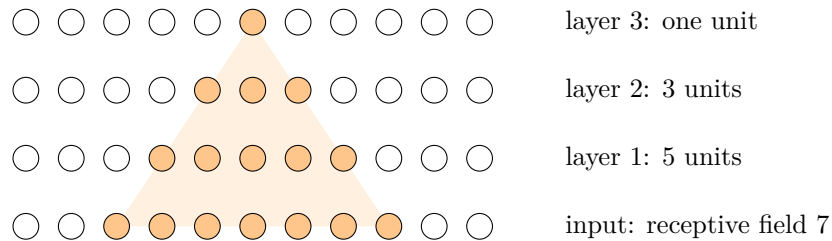


Figure 29: Receptive field of one unit in the third layer of a network with kernel size 3 and stride 1. Each layer adds $K - 1 = 2$ positions, so the unit depends on seven input values.

Stride and dilation make the growth much faster. After a layer with stride S , neighboring units are S input positions apart, so every later kernel tap reaches S times further. Let r_l be the

receptive field after layer l and j_l the distance, measured in input positions, between neighboring units of layer l (with $r_0 = 1$, $j_0 = 1$). Then

$$r_l = r_{l-1} + (K_{\text{eff},l} - 1) j_{l-1}, \quad j_l = j_{l-1} S_l. \quad (74)$$

For three layers with kernel size 3 and stride 2 this gives $r = 3, 7, 15$ with $j = 2, 4, 8$: the receptive field roughly doubles per layer instead of growing by two. Pooling layers count like convolutions in this recursion, with their window size as K .

Growing receptive fields are why depth matters for images. Early units see a few pixels and can only respond to local patterns such as edges and color blobs. Units a few layers deeper combine these into corners, textures, and repeated motifs; the deepest units see large parts of the image and respond to object parts and whole objects. Resolution, kernel size, stride, and pooling together decide which spatial scale a layer can capture. A useful side effect concerns kernel size: two stacked 3×3 layers have the same 5×5 receptive field as one 5×5 layer, but need $2 \cdot 9 C^2 = 18 C^2$ instead of $25 C^2$ weights for C channels, and they add a second nonlinearity. This is the design principle of the VGG network discussed below.

Putting it together: MNIST-1D. How much does the convolutional structure actually help? MNIST-1D is a small benchmark built for this question. Each of ten digit templates is a one-dimensional curve; a training example is a randomly transformed template with added noise, sampled at 40 positions. The class does not depend on where along the signal the pattern appears, which is exactly the kind of structure a convolutional network is designed for.

The convolutional network for this task has three convolutional layers and one fully connected layer, followed by softmax. All convolutions are valid, with kernel size 3, stride 2, and 15 channels. The output sizes follow from Eq. (71), and the parameter counts from Table 21:

Layer	Output size	Computation	Parameters
input	40×1		
conv 1 ($1 \rightarrow 15$)	19×15	$\lfloor 37/2 \rfloor + 1 = 19$	$1 \cdot 15 \cdot 3 + 15 = 60$
conv 2 ($15 \rightarrow 15$)	9×15	$\lfloor 16/2 \rfloor + 1 = 9$	$15 \cdot 15 \cdot 3 + 15 = 690$
conv 3 ($15 \rightarrow 15$)	4×15	$\lfloor 6/2 \rfloor + 1 = 4$	690
fully connected ($60 \rightarrow 10$)	10	$4 \cdot 15 = 60$ inputs	$60 \cdot 10 + 10 = 610$
total			2,050

By Eq. (74), the units of the three convolutional layers have receptive fields of 3, 7, and 15 input samples; the fully connected layer then combines all of them. The network is trained with SGD for 100,000 steps with learning rate 0.01 and batch size 100.

For comparison, a fully connected network with exactly the same number of layers and hidden units has 150,185 parameters, more than seventy times as many. Its training error drops to zero within a small fraction of the training steps, but its test error stays at roughly 40%. The convolutional network fits the training data more slowly and still reaches a test error of only about 17%. The fully connected network overfits (Section 2): it has enough freedom to memorize the training set and nothing that guides it toward rules that transfer to new examples.

The reason is the *inductive bias* of convolution, the prior assumption built into the architecture. The convolutional network is forced to process every position in the same way, so what it learns from a pattern at one position is immediately available at all other positions: information is shared across locations. It searches through a much smaller family of input–output mappings, and all members of that family are plausible for signals in which position does not matter. A fully connected network could in principle represent the same function, but it would have to discover weight sharing from data alone.

5.4 Convolutional Networks for Image Classification

Images are two-dimensional, and the convolutional layer extends to them without new ideas. The kernel becomes a $K \times K$ grid of weights, and each output is a weighted sum over a $K \times K$ neighborhood, plus a bias, passed through the activation:

$$h_{ij} = a\left[\beta + \sum_{m=1}^K \sum_{n=1}^K \omega_{mn} x_{i+m-\lceil K/2 \rceil, j+n-\lceil K/2 \rceil}\right]. \quad (75)$$

With C_i input channels the kernel has size $C_i \times K \times K$ and the sum also runs over the channels; C_o such kernels produce C_o output channels with $C_i C_o K^2$ weights and C_o biases (Table 21). Stride, dilation, and padding act separately along the rows and the columns, and Eq. (71) gives the output height and width.

Downsampling. Classification networks gradually reduce the spatial resolution. This saves computation, enlarges the receptive fields of later layers, and aggregates evidence over larger regions. Three standard operations halve the resolution of a feature map:

- *subsampling*: keep every other position in each direction, which is equivalent to a convolution with stride 2;
- *max pooling*: keep the maximum of each 2×2 block; small shifts of a feature inside a block do not change the result, which gives partial invariance to translation (not equivariance, which comes from the convolution itself) at the price of losing the exact position;
- *mean pooling*: keep the average of each 2×2 block.

Pooling acts on each channel separately and has no parameters. For the 4×4 feature map below, split into four 2×2 blocks,

$$\begin{pmatrix} 1 & 3 & 2 & 0 \\ 4 & 2 & 1 & 1 \\ 0 & 1 & 5 & 6 \\ 2 & 0 & 7 & 8 \end{pmatrix} \xrightarrow{\max} \begin{pmatrix} 4 & 2 \\ 2 & 8 \end{pmatrix}, \quad \xrightarrow{\text{mean}} \begin{pmatrix} 2.5 & 1 \\ 0.75 & 6.5 \end{pmatrix}, \quad \xrightarrow{\text{subsample}} \begin{pmatrix} 1 & 2 \\ 0 & 5 \end{pmatrix}.$$

In the backward pass, max pooling behaves like the maximum node of Table 9: the gradient is routed only to the position that held the maximum. Mean pooling distributes it equally over the block.

Networks that must output an image-sized result, such as a segmentation, need the reverse operation. The simplest is to *duplicate* every value into a 2×2 block. *Max-upsampling* (max-unpooling) remembers where each maximum came from during max pooling and writes the value back to exactly that position, filling the rest with zeros. *Bilinear interpolation* fills the new positions with weighted averages of their neighbors. A learned alternative is the transposed convolution, which spreads each input value over a neighborhood with learned weights.

A last special case is the 1×1 convolution. A kernel of spatial size 1×1 looks at a single position but at all C_i channels. It computes, at every pixel independently, a weighted combination of the channels. This is equivalent to running the same small fully connected network at every position. It mixes information between channels and changes their number, from C_i to C_o , at the cost of only $C_i C_o + C_o$ parameters. It is often used to reduce the number of channels before an expensive 3×3 layer.

Image classification and the ConvNet layout. With these building blocks, full image classifiers can be assembled. Image classification is a multiclass problem: the network receives an image and outputs a probability for each of more than two discrete classes. Deep networks won

this task on the ImageNet benchmark, a database of millions of labeled images in 1,000 classes, typically resized to 224×224 pixels. Results are reported as *top-1 error* (the highest-scoring class is wrong) and *top-5 error* (the correct class is not among the five highest-scoring ones).

Almost every classification network follows the layout of Figure 30. A stack of blocks, each made of convolution and ReLU, often followed by max pooling, reduces the resolution step by step while increasing the number of channels: spatial detail is traded for more abstract features. At the end, the feature maps are flattened (or averaged over all positions) into one vector, and one or more fully connected layers followed by a softmax produce the class probabilities. Training minimizes the cross-entropy loss with SGD and backpropagation exactly as in Section 2. ReLU is the standard activation function in ConvNets because it is cheap and does not saturate for positive inputs.

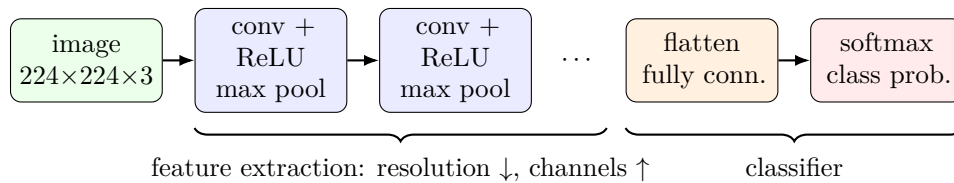


Figure 30: Basic layout of a convolutional network for classification. Convolution–ReLU–pooling blocks extract increasingly abstract features at decreasing resolution; fully connected layers and a softmax turn them into class probabilities.

AlexNet (2012). AlexNet was the first convolutional network to win ImageNet, by a large margin. It has five convolutional layers, some followed by max pooling, and three fully connected layers, with ReLU activations throughout. Of its roughly 60 million parameters, almost all sit in the fully connected layers; the convolutional layers are cheap by Table 21. Such a model overfits easily, and AlexNet introduced or popularized the regularization tools that are still standard (Section 2):

- *Data augmentation* enlarged the training set by a factor of 2048, using spatial transformations (random crops and horizontal mirroring) and modifications of the input intensities. The network sees a slightly different version of every image in every epoch, and it must give all of them the same label.
- *Dropout* in the fully connected layers sets the output of a random subset of neurons to zero, only during training. No unit can rely on a particular other unit being present, which reduces overfitting; at test time all units are used.
- *Test-time averaging*: the prediction is the average over five crops of the image and their mirrored versions, a form of augmentation at test time.
- SGD with momentum 0.9, batch size 128, and an L_2 weight-decay penalty on large weights.

AlexNet reached a top-5 error of 16.4% and a top-1 error of 38.1%.

VGG (2015). The VGG networks pushed depth further with a deliberately simple design: only 3×3 convolutions (stride 1, same padding) and 2×2 max pooling, with the number of channels doubling after each pooling step. As shown above for receptive fields, stacks of small kernels reach large receptive fields with fewer weights and more nonlinearities than one large kernel. The largest variant has 19 weight layers and 144 million parameters and reached a top-5 error of 6.8% and a top-1 error of 23.7%. Table 22 compares the two networks. Over the following years the ImageNet error kept falling as networks grew deeper, until residual networks (Section 6) made training of very deep networks routine and surpassed human-level top-5 performance.

	AlexNet (2012)	VGG-19 (2015)
Depth	5 convolutional + 3 fully connected	16 convolutional + 3 fully connected
Kernels	large in the first layer, then smaller	3×3 everywhere
Parameters	about 60 million, mostly fully connected	144 million
Regularization	augmentation, dropout, weight decay	augmentation, dropout, weight decay
Top-5 / top-1 error	16.4% / 38.1%	6.8% / 23.7%

Table 22: Two landmark ImageNet classifiers. VGG replaces large kernels by deep stacks of 3×3 convolutions and roughly halves the error.

Pre-training and fine-tuning. Networks of this size need millions of labeled images, which most applications do not have. *Pre-training* solves this by *transfer learning*. The network is first trained on a large dataset such as ImageNet. Its early layers then contain generic feature detectors (edges, color blobs, textures, simple parts) that are useful for almost any image task. For the new task, the final classification layer is replaced by a new one with the right number of classes. With little data, the pre-trained layers are *frozen* and only the new head is trained, so the network acts as a fixed feature extractor. With more data, all layers are *fine-tuned* with a small learning rate, starting from the pre-trained weights instead of a random initialization. Either way the network starts from a good representation and needs far fewer labeled examples than training from scratch.

5.5 Object Detection and Semantic Segmentation

Classification produces one label for the whole image. Many applications need more: *where* the objects are, and which pixels belong to them. Both tasks reuse the convolutional building blocks, but change what the network outputs.

Object detection with YOLO. An object detector outputs a set of bounding boxes, each with a class. YOLO (“You Only Look Once”) does this with a single forward pass of a network similar to VGG, applied to a 448×448 input. The final feature map is interpreted as a 7×7 grid of cells over the image (Figure 31). Each cell predicts

- class probabilities for the object whose center falls into the cell, and
- two bounding boxes, each described by five numbers: the center (x, y) , the width and height, and a confidence that the box contains an object.

For C classes the output therefore has $7 \times 7 \times (2 \cdot 5 + C)$ entries; with the $C = 20$ classes of the PASCAL VOC dataset this is $7 \cdot 7 \cdot 30 = 1,470$ numbers. All 98 candidate boxes are produced at once, which is what makes YOLO fast. A heuristic at the end discards boxes with low confidence and removes duplicates that overlap strongly with a more confident box of the same class (non-maximum suppression). Training uses the same toolbox as for classification: momentum, weight decay, dropout, and data augmentation.

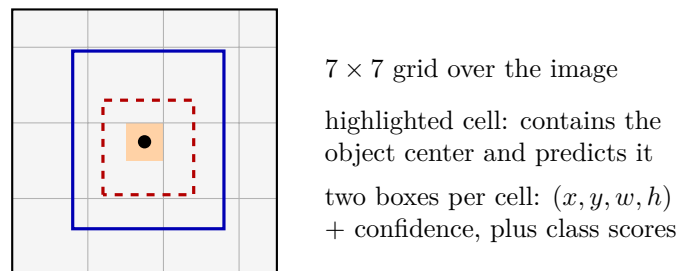


Figure 31: YOLO divides the image into a 7×7 grid. The cell containing an object’s center predicts class scores and two candidate boxes (solid and dashed); a final thresholding step keeps the confident, non-duplicate boxes.

Semantic segmentation. *Semantic segmentation* assigns a class label to every pixel of the image, for example road, car, pedestrian, or background. It does not distinguish between two objects of the same class; all car pixels simply receive the label “car”. Formally it is a multivariate classification problem: one output per pixel, each a binary (object versus background) or multiclass decision. The output must be as large as the input and, by the argument on invariance and equivariance, should move with the image content.

The standard solution is a convolutional *encoder–decoder* network (Figure 32). The *encoder* is a classification-style network, for example the convolutional part of VGG: it reduces the resolution and builds features with large receptive fields that know *what* is in the image. The *decoder* mirrors it with upsampling layers (for example max-unpooling at the positions stored by the encoder’s max pooling) and convolutions, recovering *where* each class lies at full resolution. The final layer is a 1×1 convolution with one channel per class and a softmax at every pixel, and the loss is the sum of the per-pixel cross-entropies. Because every layer is convolutional, the whole network is equivariant, and the per-pixel classifier shares its weights across the image. The U-Net of Section 6 improves this design by adding skip connections that pass fine details from the encoder directly to the decoder.

Table 23 summarizes how the three tasks differ in their output and in the network that produces it.

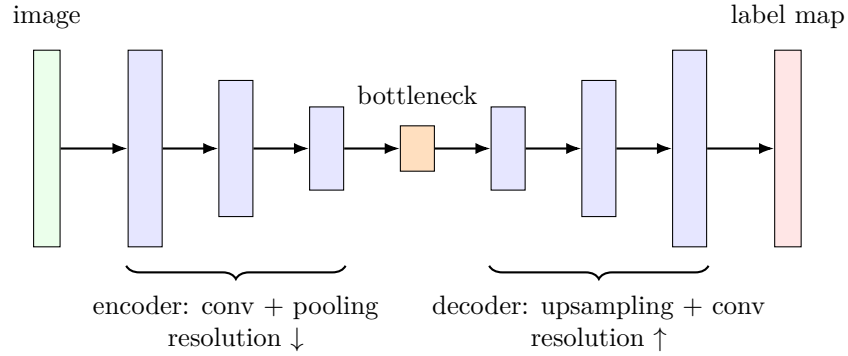


Figure 32: Convolutional encoder–decoder for semantic segmentation. Box heights indicate spatial resolution. The encoder compresses the image into features with large receptive fields; the decoder upsamples them back to one class distribution per pixel.

Task	Output	Desired property	Network
Classification	one class distribution per image	translation invariance	conv–pool stack, fully connected layers, softmax
Object detection	boxes with class and confidence	box positions move with the objects	conv backbone with a grid of box predictions (YOLO)
Semantic segmentation	one class per pixel	translation equivariance	encoder–decoder, per-pixel softmax

Table 23: Three visual tasks solved with convolutional networks. They share the convolutional feature extractor and differ in the output layer and in whether spatial position is discarded or kept.

6 Residual Networks

The convolutional networks of Section 5 became better with every added layer, from the eight layers of AlexNet to the nineteen of VGG. It is tempting to continue: if 19 layers beat 8, then 50 or 200 layers should be better still. In practice, simply stacking more layers soon stops working. Signals shrink or explode on their way through the network, gradients become erratic, and beyond a certain depth even the *training* error goes up.

Three techniques together make very deep networks trainable, and all modern image networks use them. *Careful parameter initialization* keeps the size of activations and gradients constant from layer to layer at the start of training. *Residual connections* add the input of each block to its output, which gives the gradient a direct path through the network. *Batch normalization* rescales activations during training so that they stay in a well-behaved range. This section explains each technique, starting with the problem it solves, and then presents the architectures built from them: ResNet, DenseNet, U-Net, and stacked hourglass networks.

6.1 Initialization and the Variance of Activations

Training starts from randomly chosen parameters. A common choice sets all biases to zero and draws every weight from a normal distribution with mean zero and variance σ_Ω^2 . The value of σ_Ω^2 looks like a minor detail, but in a deep network it decides whether training can start at all. Every layer multiplies the signal by a weight matrix. If the weights are small, the activations of the next layer are likely to be smaller than those of the current layer; after fifty layers they have practically vanished. If the weights are large, the activations grow from layer to layer and eventually overflow. The backward pass also multiplies by the (transposed) weight matrices, so the same happens to the gradients. This is the *vanishing gradient* and the *exploding gradient* problem: the early layers receive gradients that are either far too small to change anything or so large that every update destroys what was learned.

How the variance propagates. The right value of σ_Ω^2 follows from tracking the variance of the signal through one layer. Let f_j be the pre-activations of layer k , with mean zero and variance σ_f^2 . The layer applies ReLU, $h_j = \max(0, f_j)$, and the next layer computes the pre-activations

$$f'_i = \beta_i + \sum_{j=1}^{D_h} \Omega_{ij} h_j, \quad (76)$$

where D_h is the number of units in layer k . With zero biases and weights that are independent of the activations, the variance of f'_i is the sum of the variances of the D_h products. Each product has variance $\sigma_\Omega^2 \mathbb{E}[h_j^2]$. Because f_j is symmetric around zero, ReLU sets half of the values to zero and keeps the other half, so $\mathbb{E}[h_j^2] = \sigma_f^2/2$. Together,

$$\sigma_{f'}^2 = \sigma_\Omega^2 \sum_{j=1}^{D_h} \frac{\sigma_f^2}{2} = \frac{1}{2} D_h \sigma_\Omega^2 \sigma_f^2. \quad (77)$$

Each layer therefore multiplies the variance by the factor $\frac{1}{2} D_h \sigma_\Omega^2$. The variance stays constant exactly when this factor is one, that is, when

$$\sigma_\Omega^2 = \frac{2}{D_h}. \quad (78)$$

He initialization. The backward pass leads to the same argument with the roles of the layers swapped. The gradient with respect to the activations of layer k is a sum over the $D_{h'}$ units of layer $k+1$, so keeping the gradient variance constant requires $\sigma_\Omega^2 = 2/D_{h'}$. When the two layers

have different sizes, the two conditions cannot both hold, and the usual compromise uses the mean of D_h and $D_{h'}$:

$$\sigma_\Omega^2 = \frac{2}{(D_h + D_{h'})/2} = \frac{4}{D_h + D_{h'}}. \quad (79)$$

This rule is called *He* or *Kaiming initialization*. It assumes ReLU activations; the factor 2 compensates for the half of the signal that ReLU removes. For convolutional layers, D_h is the number of inputs of one output unit, $C_i K^2$ for kernel size K and C_i input channels.

Worked example: fifty layers of width 100. Consider a network with 50 hidden layers of $D_h = 100$ units each. By Eq. (77) the variance is multiplied by $\frac{1}{2} \cdot 100 \cdot \sigma_\Omega^2 = 50 \sigma_\Omega^2$ per layer, so after 50 layers by $(50 \sigma_\Omega^2)^{50}$. He initialization gives $\sigma_\Omega^2 = 2/100 = 0.02$ and a factor of exactly 1. Choosing $\sigma_\Omega^2 = 0.1$ looks harmless, but gives a factor of 5 per layer and $5^{50} \approx 10^{35}$ overall; $\sigma_\Omega^2 = 0.01$ gives $0.5^{50} \approx 10^{-15}$. Figure 33 shows these growth curves. The backward pass behaves in the same way, with the gradient variance growing or shrinking from the output back to the input. With 32-bit floating point numbers (range about $10^{\pm 38}$), the settings far from 0.02 make training impossible before it begins.

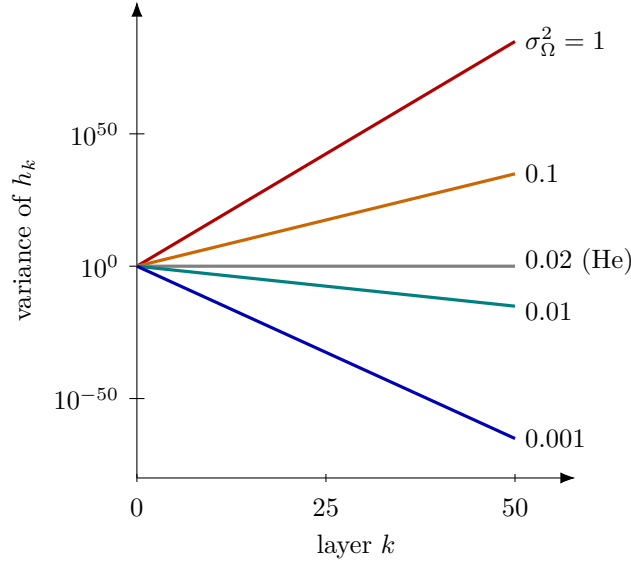


Figure 33: Variance of the activations in a network with 50 layers of 100 ReLU units, on a logarithmic scale, for five weight variances. Only He initialization, $\sigma_\Omega^2 = 2/D_h = 0.02$, keeps the variance constant; all other choices change it by many orders of magnitude.

Initialization only controls the network at the start of training; once the weights change, the variances drift again. Pre-trained weights, discussed for transfer learning in Section 5, are a different kind of initialization: they start from weights that already encode useful features instead of from random numbers with the right scale.

6.2 Why Deeper Plain Networks Fail

A network processes its input sequentially. Each layer receives the output of the previous layer, so a network with four layers is a chain of nested functions,

$$\mathbf{h}_1 = \mathbf{f}_1[\mathbf{x}, \phi_1], \quad \mathbf{h}_2 = \mathbf{f}_2[\mathbf{h}_1, \phi_2], \quad \mathbf{h}_3 = \mathbf{f}_3[\mathbf{h}_2, \phi_3], \quad \mathbf{y} = \mathbf{f}_4[\mathbf{h}_3, \phi_4], \quad (80)$$

or in one line $\mathbf{y} = \mathbf{f}_4[\mathbf{f}_3[\mathbf{f}_2[\mathbf{f}_1[\mathbf{x}]]]]$. Here $\mathbf{f}_k$ may be a single layer or a group of layers, and ϕ_k are its parameters.

Degradation: training error rises with depth. Even with good initialization, adding layers helps only up to a point. On the CIFAR-10 dataset, a plain convolutional network with 56 layers has a higher error than one with 20 layers, and this holds not only for the test error but already for the *training* error. This is not overfitting. An overfitting model fits the training set too well; here the deeper model fits it worse, although it could in principle copy the 20-layer network and set its extra layers to the identity. The deeper network is harder to *optimize*.

Shattered gradients. A likely reason is that the gradients of deep networks are *shattered*. Gradient descent takes finite steps and relies on the gradient being roughly constant over the length of one step. In a shallow network, the gradient with respect to an early-layer parameter changes smoothly as the parameter changes. In a deep network it changes quickly and unpredictably, even for tiny parameter changes, and looks almost like noise; a step along the gradient then no longer reliably reduces the loss. The chain rule shows where this comes from. The derivative of the output with respect to the first layer is

$$\frac{\partial \mathbf{y}}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{f}_4}{\partial \mathbf{f}_3} \frac{\partial \mathbf{f}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_2}{\partial \mathbf{f}_1}. \quad (81)$$

Every factor is evaluated at activations that are computed from $\mathbf{f}_1$. When the parameters of the first layer change, all factors in the product change at once, and the product of many changing matrices varies in an increasingly complex way the deeper the network is. Changes in early layers therefore modify the output in a way that becomes harder and harder to predict.

6.3 Residual Connections

Residual connections, also called skip connections, fix this by changing what each block computes. Instead of replacing its input, each block adds a correction to it: the input of the block is added to the block's output.

$$\begin{aligned} \mathbf{h}_1 &= \mathbf{x} + \mathbf{f}_1[\mathbf{x}, \phi_1], & \mathbf{h}_2 &= \mathbf{h}_1 + \mathbf{f}_2[\mathbf{h}_1, \phi_2], \\ \mathbf{h}_3 &= \mathbf{h}_2 + \mathbf{f}_3[\mathbf{h}_2, \phi_3], & \mathbf{y} &= \mathbf{h}_3 + \mathbf{f}_4[\mathbf{h}_3, \phi_4]. \end{aligned} \quad (82)$$

The block $\mathbf{f}_k$ now only has to learn the *residual*, the difference between the desired output and the input. A block that should do nothing only needs weights close to zero, which is easy to reach, whereas a plain block would have to learn the identity map exactly. Input and output of a residual block must have the same size so that they can be added. Figure 34 shows the resulting network.

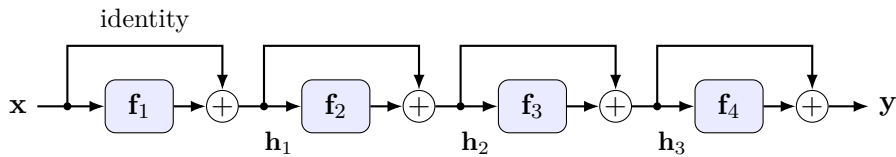


Figure 34: A residual network with four blocks. Each skip connection carries the block input unchanged around the block and adds it to the block output, so every block computes a correction to its input.

A residual network is a sum of many paths. Substituting the equations (82) into each other unravels the network:

$$\begin{aligned} \mathbf{y} &= \mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] + \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]] \\ &\quad + \mathbf{f}_4[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] + \mathbf{f}_3[\dots]]. \end{aligned} \quad (83)$$

The output is the input plus the contributions of networks of every depth from one to four. Each block can be passed through or skipped, so K blocks give 2^K paths from input to output: for $K = 4$, one path of length 0 (the pure identity), four of length 1, six of length 2, four of length 3, and one of length 4. A residual network behaves like an ensemble of shallower networks, and most of its paths are much shorter than the full depth.

Gradients through the skip connection. The same structure appears in the backward pass. The Jacobian of one residual block is

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \mathbf{I} + \frac{\partial \mathbf{f}_k}{\partial \mathbf{h}_{k-1}}, \quad (84)$$

so the derivative of the output with respect to the input is the product $\prod_k (\mathbf{I} + \partial \mathbf{f}_k / \partial \mathbf{h}_{k-1})$. Multiplying it out gives again 2^K terms, one per path, and the first term is the identity. Unlike Eq. (81), the gradient is no longer a single long product: the short paths contribute terms with few factors, which change slowly and predictably when the early layers change. In the language of computational graphs, the skip connection ends in an addition node, and an addition node distributes the upstream gradient unchanged to both of its inputs (Table 9). It does not send its inputs backward. The gradient of the loss therefore reaches every block directly through the chain of skip connections, however deep the network is.

Order of operations inside a block. Where the nonlinearity sits in a residual block matters:

- If a block ends with ReLU, as in linear layer followed by ReLU, then $\mathbf{f}_k \geq 0$ and each block can only *increase* the values of its input. The ReLU is therefore placed at the beginning of the block: ReLU, then linear layer.
- If every block starts with ReLU, a block does nothing for negative inputs. The network therefore begins with a plain linear transformation before the first residual block, so that the signal entering the residual chain is a learned mixture rather than the raw input.
- A block usually contains several layers, for example ReLU, linear, ReLU, linear, before the skip connection adds the input back.

Variant	$\mathbf{x}$	$\mathbf{h}_1$	$\mathbf{h}_2$	$\mathbf{h}_3$	after K blocks
plain residual blocks	1	2	4	8	2^K
output scaled by $1/\sqrt{2}$	1	1	1	1	1
batch normalization at block start	1	2	3	4	$K + 1$

Table 24: Variance of the signal after each residual block when input and block output both have unit variance. Plain addition doubles the variance per block; scaling by $1/\sqrt{2}$ keeps it constant; batch normalization inside the residual branch lets it grow only linearly.

Variance in residual networks. Residual connections solve the shattered gradients, but they create a new problem for the forward pass. Suppose the input of a block has variance 1 and He initialization makes the block output $\mathbf{f}_k$ also have variance 1. If the two are uncorrelated, their sum has variance $1 + 1 = 2$, so the variance *doubles* with every block (Table 24), and the network again suffers from unstable forward propagation and exploding gradients. One remedy combines He initialization with a multiplication of each block’s combined output by $1/\sqrt{2}$, which divides the variance by two and restores it to 1. More common is batch normalization, discussed next. With these measures, residual connections roughly double the depth that can be trained before performance degrades; together with normalization, networks with hundreds of layers become routine.

6.4 Batch Normalization

Batch normalization (BatchNorm, BN) attacks the variance problem directly. Instead of hoping that the initialization keeps activations in a good range, it measures their statistics during training and rescales them. For every hidden unit, the activations are standardized across the training examples of the current mini-batch $\mathcal{B}$. First the empirical mean and standard deviation of the unit's activation h over the batch are computed,

$$m_h = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} h_i, \quad s_h = \sqrt{\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (h_i - m_h)^2}, \quad (85)$$

then each activation is standardized to mean 0 and variance 1,

$$h_i \leftarrow \frac{h_i - m_h}{s_h + \epsilon} \quad \forall i \in \mathcal{B}, \quad (86)$$

where the small constant ϵ prevents division by zero. Finally a learned scale γ and offset δ give the network back the freedom to choose the mean and standard deviation itself:

$$h_i \leftarrow \gamma h_i + \delta \quad \forall i \in \mathcal{B}. \quad (87)$$

The same layer is often written as $\hat{x} = (x - \mu_{\mathcal{B}}) / \sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}$, $y = \gamma \hat{x} + \beta$, with batch mean $\mu_{\mathcal{B}}$, batch variance $\sigma_{\mathcal{B}}^2$, and offset β instead of δ ; the two forms differ only in where ϵ enters. Without γ and δ , the layer would force every unit to zero mean and unit variance forever. With them, the network can learn any scale, but the scale is now an explicit parameter of the layer instead of an accident of the weights of all preceding layers.

Where the statistics are computed. Batch normalization is applied independently to each hidden unit, and each unit gets its own γ and δ . A fully connected network with K layers of D hidden units therefore learns KD offsets and KD scales. In a convolutional network, the statistics are computed not only across the batch but also across all spatial positions of a channel, in keeping with weight sharing: every position of a feature map is treated in the same way. A convolutional network with K layers of C channels thus learns KC offsets and KC scales. For example, a layer with 64 channels, a batch of 32 images, and 56×56 feature maps computes each of its 64 means over $32 \cdot 56 \cdot 56 = 100,352$ values and adds 128 learned parameters.

Training mode and evaluation mode. At test time there may be only a single image, and the prediction for one image should not depend on which other images happen to be in the same batch. Batch normalization therefore behaves differently in the two phases. During training it uses the statistics of the current batch and additionally keeps running averages of the means and variances. During evaluation it uses these stored averages as fixed constants, so the layer becomes a fixed affine map. Forgetting to switch a model into evaluation mode for validation and testing is a classic error: the outputs then depend on the batch composition.

Invariance to the scale of the weights. If all weights and biases that feed a batch-normalized unit are multiplied by some $\alpha > 0$, the mean and standard deviation over the batch are multiplied by α as well, and the standardized value does not change. Batch normalization thus makes the network invariant to rescaling weights and biases, because it compensates for it. This removes exactly the sensitivity to scale that made initialization so critical.

Other normalizations. Batch normalization depends on the batch, which is a disadvantage for small batches and for sequence models. Other normalizations use the same standardize-then-scale recipe but compute the statistics over different axes of the activation tensor with batch size N , channels C , and spatial size $H \times W$ (Table 25). Layer normalization, instance normalization, and group normalization compute their statistics within a single example and therefore behave identically in training and evaluation.

Method	Statistics computed over	One mean per
Batch norm	N, H, W	channel (shared by all examples)
Layer norm	C, H, W	example
Instance norm	H, W	example and channel
Group norm	H, W , and a group of channels	example and channel group

Table 25: Normalization layers differ only in the axes of the $N \times C \times H \times W$ activation tensor over which mean and variance are computed. Only batch normalization mixes different examples.

Costs and benefits. Batch normalization has three main effects:

- *Stable forward propagation.* With the initialization $\gamma = 1$, $\delta = 0$, every normalized activation has unit variance. In a network without residual connections this keeps the variance constant through all layers. In a residual network with batch normalization at the start of each residual branch, each block adds variance 1 to the signal, so the variance grows as 1, 2, 3, 4, ... (Table 24). Later blocks then contribute a smaller share to the sum, and the network is effectively less deep at the start of training. As training proceeds, γ can grow; the network thereby controls its own effective depth.
- *Higher learning rates.* Batch normalization makes the loss surface and its gradients change more smoothly, which allows larger learning rates and faster training.
- *Regularization.* The normalization depends on the other examples in the batch, so every activation receives a small amount of random noise during training. Such noise can improve generalization, similar to dropout.

The costs are the dependence on the batch: small batches give noisy statistics, and training and evaluation behave differently.

6.5 Common Residual Architectures

Residual connections, He initialization, and batch normalization are the building blocks of nearly all deep image networks. Four architectures show how they are combined: ResNet for classification, DenseNet as a variant with concatenation, U-Net for segmentation, and stacked hourglass networks for pose estimation.

ResNet blocks. The standard ResNet block for images applies batch normalization, then ReLU, then a 3×3 convolution, and repeats this sequence once before the skip connection adds the input. This follows the order of operations above: normalization and nonlinearity come first, and the block ends with a linear operation. Deep ResNets use the *bottleneck block* instead (Figure 35). A 1×1 convolution first reduces the number of channels by a factor of four, the expensive 3×3 convolution then works on the reduced representation, and a second 1×1 convolution increases the number of channels again so that the input can be added.

The saving is large. With 256 input and output channels and ignoring biases, a standard block with two 3×3 convolutions needs $2 \cdot 9 \cdot 256^2 \approx 1.18$ million weights. The bottleneck block needs

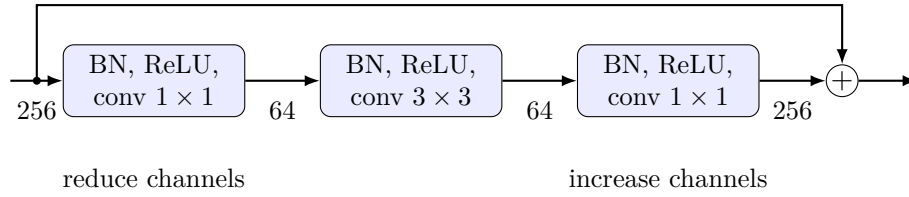


Figure 35: Bottleneck ResNet block. Numbers below the arrows are channel counts. The two 1×1 convolutions reduce the channels by a factor of four and restore them, so the 3×3 convolution runs on only 64 channels.

$256 \cdot 64 + 9 \cdot 64^2 + 64 \cdot 256 = 16,384 + 36,864 + 16,384 = 69,632$ weights, about seventeen times fewer, and it contains three nonlinear layers instead of two.

ResNet-200. The ResNet-200 network for ImageNet starts with a 7×7 convolution with stride 2 and a max pooling step, reducing the $224 \times 224 \times 3$ image to 56×56 with 64 channels. Four stages of bottleneck blocks follow, with $56 \times 56 \times 256$, $28 \times 28 \times 512$, $14 \times 14 \times 1024$, and $7 \times 7 \times 2048$ outputs; inside the blocks the channels are reduced to 64, 128, 256, and 512. Between the stages the resolution is halved, and the skip connection uses a strided 1×1 convolution so that input and output shapes match again. The two middle stages contain 24 and 36 blocks; with three blocks each in the first and last stage this gives 66 blocks of three convolutions, plus the first convolution and the final layer, 200 layers in total. Global average pooling, a fully connected layer, and a softmax over the 1000 classes produce the output. ResNet-200 reached a top-5 error of 4.8% on ImageNet, the first network to exceed human performance on this benchmark, and continued the trend of Table 22 from 16.4% (AlexNet) and 6.8% (VGG).

DenseNet. Instead of adding the input activations to the output, they can also be *concatenated* with it. In a DenseNet, the input of every layer consists of the concatenated outputs of *all* previous layers. Early layers then contribute directly to the output, which, as with residual connections, makes the loss surface behave reasonably. Concatenation lets the number of channels grow with every layer, so a 1×1 convolution reduces it from time to time. Concatenation across a downsampling step is impossible, because the representations have different spatial sizes. The chain of concatenations is therefore broken at each downsampling step, and the smaller representation starts a new chain. With the same number of parameters, DenseNet can perform better than ResNet.

U-Net. The encoder–decoder networks for semantic segmentation in Section 5 must squeeze the whole image through a low-resolution bottleneck. The decoder then has to recover the exact position of every boundary from coarse features, as if it had to “remember” the high-resolution detail. The U-Net adds skip connections from each encoder level to the decoder level of the same resolution (Figure 36). The decoder receives the fine details directly and only has to combine them with the semantic information from the bottleneck. The U-Net uses valid 3×3 convolutions, so the encoder feature maps are slightly larger than the corresponding decoder maps; they are *cropped* to the same size and then *concatenated*, not added. Upsampling uses transposed 2×2 convolutions, and a final 1×1 convolution produces one score per class and pixel. The input of 572×572 pixels therefore gives a label map of 388×388 pixels.

The U-Net was designed for segmenting medical images. A typical example is the classification of voxels in three-dimensional volumes of mouse cortex recorded with a scanning electron microscope, where a single U-Net already gives a good labeling and an ensemble of five U-Nets a better one. Because its output has the size of its input and preserves fine detail, the U-Net is also the standard backbone for other image-to-image tasks, including the denoising networks of diffusion

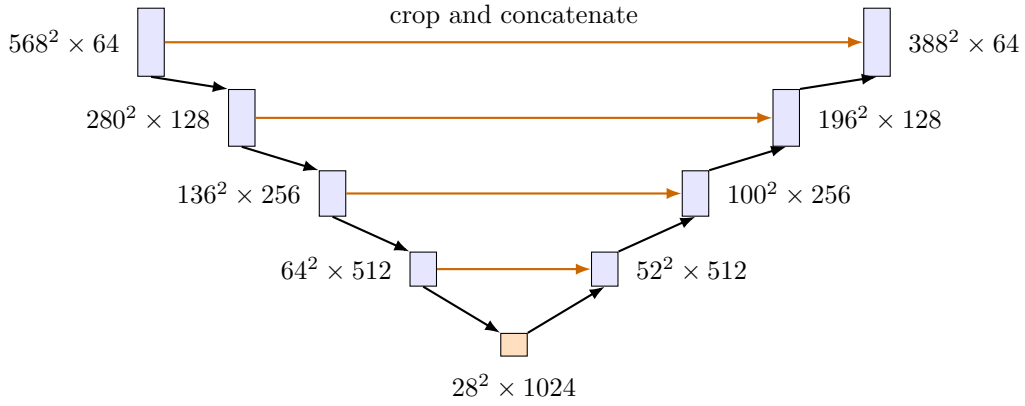


Figure 36: U-Net. The encoder (left) halves the resolution with max pooling, the decoder (right) doubles it with transposed convolutions. Horizontal skip connections copy each encoder map, crop it, and concatenate it with the decoder map of the same level. Labels give spatial size and channels.

models.

Stacked hourglass networks. A stacked hourglass network repeats several U-Net-like encoder-decoder blocks one after the other. Each block alternates between looking at the image at a local and at a global level, and the next block refines the result of the previous one. Inside the blocks, skip connections *add* the results rather than concatenating them. The network was developed for human pose estimation. It produces one *heatmap* per body joint, whose value at each pixel indicates how likely the joint is to lie there, and the estimated joint position is the maximum of its heatmap. Training compares the predicted heatmaps with target heatmaps, so pose estimation is formulated as a *regression* problem, in contrast to the classification problems considered so far.

Why residual networks work so well. Visualizations of the loss as a function of the parameters make the difference visible. For a deep plain network, the loss surface is rugged and full of local structure, matching the shattered gradients. With residual connections, the loss surface around the solution is much smoother and nearly convex, so gradient descent with a finite step size makes steady progress. Table 26 summarizes the four architectures.

Architecture	Skip operation	Typical task	Key idea
ResNet	addition	classification	bottleneck blocks with BN and ReLU; hundreds of layers
DenseNet	concatenation	classification	every layer sees all previous outputs; 1×1 convolutions limit channels
U-Net	crop and concatenate	segmentation	skips from encoder to decoder preserve high-resolution detail
Stacked hourglass	addition	pose estimation	repeated U-Net-like blocks; one heatmap per joint (regression)

Table 26: Four common residual architectures. They differ in how the skip connection combines its signals and in the task the network is built for.

7 Transformers

A convolution uses the same filter at every image location, but a single filter only sees a local neighborhood. In a sentence, the useful context for a word may be many positions away: in “the food was delicious, but the service was slow,” the two adjectives refer to different nouns. A transformer lets each position combine information from every position. The strength of each connection is computed from the current input rather than fixed in advance. Its learned parameters are shared across positions, so the same layer can process sequences of different lengths.

7.1 Tokens and Embeddings

Before a network can process text, a tokenizer divides it into discrete units. Using complete words alone makes the vocabulary enormous and leaves new names or spellings without a representation. Subword tokenization instead builds a vocabulary of frequent character sequences. Repeatedly merging common adjacent pieces can turn characters into a mixture of fragments and whole words. Related forms such as “walk,” “walked,” and “walking” can then share pieces, while an unfamiliar word can still be assembled from smaller units.

Give each of the $|\mathcal{V}|$ vocabulary entries an integer index. A learned embedding table $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ maps token t_i to $x_i = E_{t_i, \cdot} \in \mathbb{R}^d$. The same table is used at every position. This vector is a starting representation, not a meaning fixed for all contexts: the token “bank” has the same input vector in “river bank” and “bank account.” Attention can give those occurrences different outputs by combining them with their neighbors.

Token order must also be supplied. Without position information, permuting the input tokens merely permutes the outputs of self-attention: it cannot tell “the man ate the fish” from “the fish ate the man” at a given token position. An *absolute positional encoding* adds a vector p_i to each embedding, giving the layer input $\tilde{x}_i = x_i + p_i$. One fixed choice uses different sinusoidal frequencies in successive coordinate pairs:

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right). \quad (88)$$

Positions may instead have learned vectors. A relative position scheme can add a learned bias b_{i-j} to the attention score from position i to position j . Both approaches make order available; the first labels individual positions, while the second labels their separation.

7.2 Self-Attention

Suppose $X \in \mathbb{R}^{N \times d}$ holds N token vectors as rows, including their position information. Each token plays three learned roles. Its *query* describes what information it seeks, its *key* describes how it can be found, and its *value* carries the information to contribute. Shared linear projections produce all three:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (89)$$

where $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ and $W^V \in \mathbb{R}^{d \times d_v}$. Query q_i compares with every key k_j . The resulting scores pass through a softmax over j :

$$s_{ij} = \frac{q_i k_j^T}{\sqrt{d_k}}, \quad a_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^N \exp(s_{i\ell})}, \quad y_i = \sum_{j=1}^N a_{ij} v_j. \quad (90)$$

The nonnegative weights in each row sum to one. Thus y_i is a weighted combination of values, with weights chosen separately for query i . The $1/\sqrt{d_k}$ factor keeps dot products from growing with key dimension when their coordinates have roughly unit variance; otherwise softmax can become so peaked that its gradients are small.

For a small calculation, take $d_k = 2$, $q = (1, 0)$, keys $k_1 = (1, 0)$ and $k_2 = (0, 1)$, and values $v_1 = (2, 0)$ and $v_2 = (0, 2)$. The scores are $(1/\sqrt{2}, 0)$, so the softmax weights are approximately $(0.670, 0.330)$. The output is $0.670(2, 0) + 0.330(0, 2) = (1.340, 0.660)$. Changing the query changes this mixture even when the two values stay the same.

All queries can be processed together. With softmax applied separately to each row, Equation 90 becomes

$$A = \text{softmax}_{\text{row}}\left(\frac{QK^T}{\sqrt{d_k}}\right), \quad Y = AV. \quad (91)$$

Here $A \in \mathbb{R}^{N \times N}$ and $Y \in \mathbb{R}^{N \times d_v}$. Every output can use every input after one layer. Forming all pairwise scores takes $O(N^2 d_k)$ arithmetic and $O(N^2)$ attention storage, which becomes costly for long sequences. Unlike a recurrent network, the score calculations for different positions can run in parallel.

One attention pattern need not serve every relationship. *Multihead attention* learns separate projections for h heads, computes $H_r = \text{Attention}(XW_r^Q, XW_r^K, XW_r^V)$ for each head r , and concatenates their outputs:

$$\text{MHA}(X) = \text{Concat}(H_1, \dots, H_h)W^O. \quad (92)$$

Each head can weight a different set of tokens. If its value width is d_v , then $W^O \in \mathbb{R}^{hd_v \times d}$ returns to the model width d . Multiple heads do not change the quadratic dependence on sequence length.

7.3 Transformer Layers

A transformer layer combines multihead attention with a small network applied independently to each token. Both parts have residual connections, as in Section 6, and layer normalization. The attention part mixes information *between* tokens; the feed forward part transforms features *within* each token. Figure 37 shows one layer.

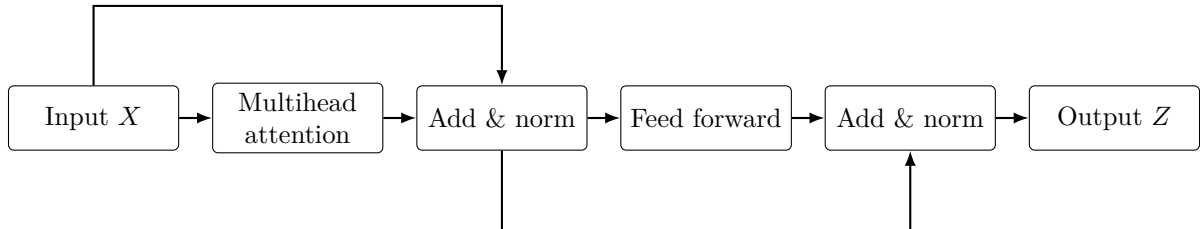


Figure 37: One transformer layer. The upper and lower paths add each sublayer’s input to its output before normalization. The feed forward network acts on each token separately.

For the layout shown, the operations can be written

$$H = \text{LN}(X + \text{MHA}(X)), \quad Z = \text{LN}(H + \text{FFN}(H)). \quad (93)$$

Layer normalization computes a mean and variance across the d features of *each token*, then applies learned scale and shift parameters. It does not require batch statistics. The same two dense layers and biases are reused at every position; for token row h_i an example is

$$\text{FFN}(h_i) = \text{ReLU}(h_i W_1 + b_1) W_2 + b_2, \quad W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}, \quad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}. \quad (94)$$

Its input and output both have width d , so the second residual addition is valid. Stacking layers lets each token repeatedly gather context and transform what it gathered. A transformer *encoder* uses such layers to represent an entire input sequence. A *decoder* uses transformer layers to produce an output sequence, restricting attention when it must predict tokens in order.

7.4 Encoders and Transfer Learning

An encoder reads all input tokens at once. Its attention is bidirectional: an output token may use words both before and after its position. This makes an encoder useful when the whole input is available and the goal is to understand it. BERT is an encoder example. During masked-language-model pretraining, some input tokens are hidden and the model predicts their identities from the surrounding tokens. A classifier applied to the output at a masked position produces a probability for each vocabulary entry; training increases the probability of the original entry. The target is a *token identity*, not the hidden token’s embedding vector. Because the same layers must recover many different missing tokens, their outputs learn contextual representations.

These representations can support several tasks. For sentence classification, a special [CLS] token is inserted and its final output is passed to a small classification head. Sentiment classification is one example. A pair of sentences can be encoded together for a relation decision. For token classification, a head reads each token’s final output, as in named-entity recognition, where words are labeled as person, place, organization, or none. Span annotation marks a contiguous range of tokens, for example by predicting its start and end. The output used by the head therefore depends on whether the target describes a whole input, each token, or a span.

Pretraining first learns reusable features on a large source dataset; a labeled target dataset then adapts the model. In *fine-tuning*, the new task’s loss updates some or all pretrained weights as well as the task head. Alternatively, a *linear probe* freezes the encoder and trains only a linear head. Freezing reduces the number of trained parameters, but does not remove the encoder at test time. Fine-tuning does not require freezing all layers except the head. Transfer can help even when source and target tasks differ because early visual filters or general contextual features are useful across tasks; the final decision rule may still need to be learned for the new labels. This is why a pretrained initialization can be better than random weights without promising that every feature or class decision transfers.

7.5 Decoders and Cross-Attention

A decoder generates one token at a time. For tokens $t_1, \dots, t_N$, an autoregressive model factors their joint probability as

$$p(t_1, \dots, t_N) = \prod_{i=1}^N p(t_i \mid t_1, \dots, t_{i-1}). \quad (95)$$

A GPT-style decoder implements these conditional distributions. The output at position i predicts the next token from a softmax over the vocabulary. *Causal masking* prevents a query at position i from attending to future positions $j > i$: their attention scores are set to $-\infty$ before softmax. Without that mask, training could read the answer it is meant to predict. A start token and a shifted target sequence align each output with its next-token target. Known target tokens can be processed in parallel during training because the mask hides later tokens. At inference, each new token must be chosen before the next step, so generation remains sequential. Providing a few examples in the input prompt can guide a pretrained decoder without changing its weights; this is in-context or few-shot learning, not fine-tuning.

For translation, the source sentence is already known, but the target sentence must be generated in order. An encoder first forms source representations $E \in \mathbb{R}^{N_e \times d}$. The decoder uses masked self-attention over its target prefix and then *cross-attention*: its target states $D \in \mathbb{R}^{N_d \times d}$ supply queries, while the encoder states supply keys and values,

$$Q = DW^Q, \quad K = EW^K, \quad V = EV^V, \quad C = \text{softmax}_{\text{row}} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (96)$$

The cross-attention weights have shape $N_d \times N_e$: each target position can select information from every source position. The causal mask applies to target self-attention, not to these known source

states. Translation training again predicts each next target token; at inference, the decoder uses its previously generated target tokens. Table 27 separates the three common layouts.

Layout	Information available	Typical output
Encoder	all input tokens	sequence or token labels
Decoder	past target tokens only	next-token probabilities
Encoder–decoder	source and past target tokens	translated sequence

Table 27: Which tokens each transformer layout can read when forming an output. The attention mask determines whether future target tokens are hidden.

Full attention over N tokens uses the $N \times N$ weight matrix from Equation 91. Its quadratic storage and compute become expensive for long text or dense images. Restricting a token to a local neighborhood, using sparse connections, or introducing selected global tokens reduces the number of interactions, but also changes which pairs can exchange information directly. A decoder’s triangular causal mask prevents future leakage; by itself it does not remove the quadratic cost of full attention over the allowed prefix.

7.6 Vision and Multiscale Transformers

An image has many more pixels than a typical sentence has words. Treating each pixel as a token makes global attention expensive: a 224×224 image has 50,176 pixel positions, so its attention matrix has over 2.5 billion entries per head. Early pixel-generating transformer decoders such as ImageGPT therefore operated on small images. A Vision Transformer (ViT) reduces the sequence length by dividing an $H \times W \times C$ image into nonoverlapping $P \times P$ patches. Assuming P divides both dimensions, the number of patches is

$$N = \frac{H}{P} \frac{W}{P} = \frac{HW}{P^2}, \quad f_i \in \mathbb{R}^{P^2 C}, \quad z_i = f_i W^E + p_i \in \mathbb{R}^d, \quad (97)$$

where f_i is a flattened patch, $W^E \in \mathbb{R}^{P^2 C \times d}$ is shared across patches, and p_i is a learned positional vector. The linear patch projection is equivalent to a convolution with kernel size and stride P , but the following encoder uses attention rather than local convolutions. A learned [CLS] vector is prepended to the N patch tokens, so the encoder processes $N + 1$ tokens. Its final class-token output passes through a classification head and softmax. Patch positions must be encoded: without them, rearranging patches would only rearrange their outputs and the class token could not recover where image content appeared.

For a 224×224 RGB image with $P = 16$, each flattened patch has $16^2 \cdot 3 = 768$ values and there are $(224/16)^2 = 196$ patches, or 197 tokens with the class token. A head’s attention matrix is then 197×197 , instead of $50,176 \times 50,176$ for pixel tokens. This calculation explains why patch size matters: smaller patches retain finer detail but increase attention cost quadratically with their count.

Unlike a convolutional network, a plain ViT has no architectural requirement that nearby pixels interact first or that shifting an image shifts its feature map in the same way. Any patch can attend to any other patch in the first layer. Convolutions instead start with local neighborhoods and build a larger receptive field across layers. This built-in locality and translation equivariance help CNNs learn from limited images. ViT must learn useful spatial relationships from data, but large-scale pretraining can make its global interactions effective. Attention distance makes the contrast visible: the average distance between a query patch and its attended patches can be large even in early ViT layers, while a small convolution initially sees only its local neighborhood. For a new classification task, a pretrained ViT can be fine-tuned, or its encoder can be frozen and a new head trained.

Plain ViT keeps one patch scale throughout. A shifted-window transformer (Swin) instead computes attention within small windows of the patch grid. The next block shifts the window partition so patches previously separated by a window boundary can exchange information. Periodic *patch merging* combines each 2×2 group of neighboring patch features and projects it to a new channel width. Spatial resolution falls while the receptive field grows, giving a hierarchy more like a CNN. If each attention window contains at most M tokens, its attention storage over all windows is $O(NM)$ rather than $O(N^2)$ for global attention, when M is fixed. The shift is essential: without it, repeated window attention cannot pass information across window boundaries at the same scale. The self-supervised ViT case DINOv2 is explained in Appendix G.

8 Generative Adversarial Networks

The networks so far learned a mapping from an image x to a label y that a human supplied. A generative model reverses the direction of interest: it should produce *new* images that look as if they came from the training set. No one can label what a “correct” new face or bedroom is, so the model must learn from unlabeled images alone. A generative adversarial network (GAN) solves this with a trick. A second network, the discriminator, tries to tell generated images from real ones, and its mistakes and successes become the training signal for the generator. This section first places GANs among learning settings without labels, then derives the adversarial loss, explains why GAN training is difficult and how the Wasserstein loss helps, and finally shows conditional GANs and image translation models such as Pix2Pix, CycleGAN, and StyleGAN.

8.1 Learning Without Human Labels

Learning settings differ in where the targets come from. In *supervised learning*, each training input x comes with a human-provided label y , and classification is the standard task. The CNNs, ResNets, and transformer classifiers of the previous sections are supervised. In *unsupervised learning*, the data have no labels; clustering, density estimation, and generative modeling are typical tasks, as in the density estimates of Section 4. *Self-supervised learning* lies in between: it uses a supervised loss, but its labels are generated automatically from the data or from the way the data were acquired. No annotator is needed, so large unlabeled collections become usable.

Several acquisition setups deliver such labels for free. A video provides the next frame a short time later, so predicting future frames has a known target. A stereo camera provides a second view; predicting it from the first view can be checked against the image the second camera actually recorded. A high-frame-rate camera can generate sharp frames and, by averaging them, realistic motion-blurred images, giving training pairs for recovering video from motion blur. Even a segmentation can be checked without labels if one assumes that a correct segmentation produces regions with typical low-level features, while a wrong segmentation produces unusual ones.

Related settings relax the labels in other ways. *Weakly supervised learning* uses labels that are incomplete (only some examples labeled), inexact (coarse, e.g. an image-level tag instead of a pixel mask), or inaccurate (noisy). *Domain adaptation* transfers a model to data with a different appearance, and *continual learning* keeps learning new tasks without forgetting earlier ones. *Pretext task learning* solves an artificial task only as preparation for the actual *downstream task*: the network trained on the pretext task provides features, and a small labeled dataset suffices for the downstream classifier.

Pretext tasks. Table 28 lists common pretext tasks for images. The first four remove or damage information and train an encoder–decoder network to restore it. An autoencoder must squeeze the image through a narrow coarse feature layer and reconstruct it; a denoising autoencoder additionally applies a distortion $d(I)$, such as noise, before the encoder. Inpainting predicts

a hidden image region from its surroundings, and colorization predicts color from a grayscale image. The last two are *classification* tasks. For a jigsaw puzzle, the image is cut into n tiles and shuffled, and the network predicts which of the $n!$ permutations was applied (in practice a subset of them). For context prediction, the network sees two patches and predicts their discrete spatial relation, e.g. “top right.” Self-supervised objectives are therefore not necessarily regression tasks: whether the target is a pixel value or a class index depends on how the pretext task is phrased. After pretraining, only the encoder is kept and reused for the downstream task.

Pretext task	Input	Target	Loss type
Autoencoding	image	same image	regression
Denoising	distorted image $d(I)$	clean image I	regression
Inpainting	image with hidden region	hidden region	regression
Colorization	grayscale image	color channels	regression or classification
Jigsaw puzzle	shuffled tiles	index of permutation	classification
Context prediction	two patches	relative position	classification
Frame order	shuffled video frames	correct order	classification

Table 28: Pretext tasks create targets from unlabeled data. Some predict pixels (regression); others predict a discrete arrangement (classification).

Two further self-supervised families, contrastive learning and energy-based self-supervision, compare views of images instead of reconstructing them; they are summarized in Appendix H.

8.2 Latent Variables and Generative Models

Most unsupervised models relate a data example x to an unobserved *latent variable* z . Usually z is much smaller than x and can be seen as a compressed description of it. Some models map from data to latent space: k -means maps x to a cluster index z , and the encoder of an autoencoder maps an image to an embedding vector z . Other models map from latent space to data space. If z has a simple distribution $Pr(z)$, for example a standard normal distribution, one can draw z and map it to a new example x . Models of this kind are called *generative models*. Figure 38 shows how these groups overlap.

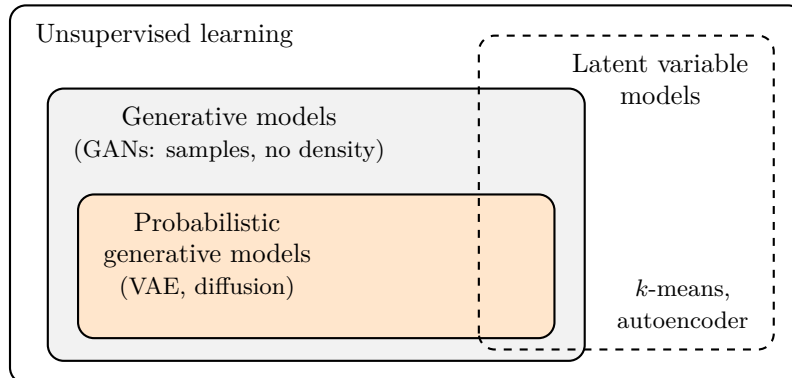


Figure 38: Taxonomy of unsupervised models. Generative models can create new examples; probabilistic generative models additionally assign a probability to every example. Latent variable models (dashed) cut across both groups.

A *probabilistic* generative model, such as a variational autoencoder (VAE) or a diffusion model, defines a probability $Pr(x_i | \phi)$ for every data point given its parameters ϕ . Training maximizes

the probability of the observed data, i.e. it minimizes the negative log-likelihood

$$L[\phi] = - \sum_{i=1}^I \log \Pr(x_i | \phi), \quad (98)$$

exactly as in maximum likelihood estimation (Equation 60). As training proceeds, the density rises where the training examples lie. A VAE thereby learns an approximation of the data distribution. A GAN is different. It is *not* probabilistic: it provides a mechanism that turns random z into samples, but it cannot say how probable a given image is. Whether a method *estimates a distribution over the data* thus separates the methods in Table 29. Parzen windows and maximum likelihood fit a density; a (restricted) Boltzmann machine defines one through an energy, $p(x) \propto \exp(-E(x))$; a GAN does not.

Method	Explicit density?	What it provides
Parzen windows	yes	$\hat{p}(x)$ as sum of windows around samples
Maximum likelihood	yes	parameters of a chosen density model
(Restricted) Boltzmann machine	yes (unnormalized)	energy $E(x)$, $p(x) \propto e^{-E(x)}$
VAE, diffusion model	yes (approximate)	likelihood bound and samples
GAN	no	samples $x^* = g[z, \theta]$ only

Table 29: Density estimators versus samplers. A GAN generator produces examples whose distribution should match the data, but it never represents that distribution as a function that can be evaluated.

Measuring sample quality. Without a likelihood, a GAN must be judged by its samples. A pretrained classifier gives a practical test with two parts. First, if a generated image x^* is realistic, the classifier should be confident: $\Pr(y | x^*)$ should be peaked at one class. Second, if the model generates all classes equally often, the marginal class distribution averaged over I samples,

$$\Pr(y) = \frac{1}{I} \sum_{i=1}^I \Pr(y | x_i^*), \quad (99)$$

should be flat. The *inception score* combines both by measuring how far each peaked $\Pr(y | x_i^*)$ lies from the flat $\Pr(y)$ with the KL divergence of Equation 102, averaged over samples: it is large only for images that are both recognizable and varied. A generator that always draws the same convincing dog passes the first test and fails the second.

8.3 Adversarial Training

A GAN consists of two networks. The *generator* $g[z, \theta]$ maps a latent vector z , drawn from a simple base distribution such as a standard normal, to a sample $x^* = g[z, \theta]$ in image space. The *discriminator* $f[x, \phi]$ receives either a real training image x_i or a generated image x_j^* and outputs the probability that its input is real. If the discriminator cannot distinguish the two, the generator produces plausible samples. If it can, the way it distinguishes them tells the generator how to improve. Figure 39 shows the arrangement.

A one-dimensional example. Let the real data be scalars clustered around some value, and let the generator merely shift standard normal noise, $x_j^* = g[z_j, \theta] = z_j + \theta$. The single parameter θ moves all generated samples along the axis. Suppose $\theta = 3$ while the real samples lie around 7. A discriminator in the form of a logistic curve over x easily separates the two groups: $\Pr(\text{real})$

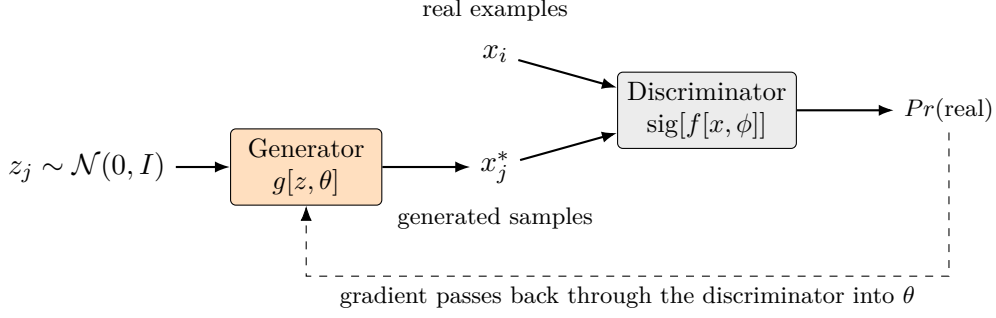


Figure 39: GAN training. The discriminator sees real images x_i and generated images x_j^* and outputs the probability of “real.” Its output is differentiated with respect to the generator parameters θ by backpropagating through the discriminator (dashed path).

is near 0 for the synthesized samples and near 1 for the real ones. The generator improves by increasing θ , because that moves its samples up the logistic curve. At $\theta = 4.9$ the groups overlap partly; at $\theta \approx 7$ they coincide and the best the discriminator can do is a nearly flat curve. Training is a race in which each network keeps adapting to the other.

The discriminator as a binary classifier. The discriminator solves a two-class problem with label $y = 1$ for real and $y = 0$ for generated. A Bernoulli distribution with parameter $\lambda \in [0, 1]$ models such a label:

$$Pr(y | \lambda) = (1 - \lambda)^{1-y} \lambda^y. \quad (100)$$

The network output $f[x, \phi]$ is an arbitrary real number, so the logistic sigmoid $\text{sig}[a] = 1/(1 + e^{-a})$ maps it into $[0, 1]$ and gives $\lambda = \text{sig}[f[x, \phi]]$. The negative log-likelihood of I labeled examples is the *binary cross-entropy*

$$L[\phi] = \sum_{i=1}^I -(1 - y_i) \log(1 - \text{sig}[f[x_i, \phi]]) - y_i \log \text{sig}[f[x_i, \phi]]. \quad (101)$$

Why cross-entropy measures a distance. The name “cross-entropy” comes from comparing two distributions. The Kullback–Leibler divergence between the empirical distribution q of the observed labels and the model distribution $Pr(y | \theta)$ is

$$D_{KL}[q \| p] = \int q(y) \log q(y) dy - \int q(y) \log Pr(y | \theta) dy. \quad (102)$$

The first term does not depend on θ . Placing the empirical distribution at the observed data, $q(y) = \frac{1}{I} \sum_i \delta[y - y_i]$, turns the second term into $-\frac{1}{I} \sum_i \log Pr(y_i | \theta)$. Minimizing the KL divergence therefore equals minimizing the negative log-likelihood, and with a Bernoulli model this is exactly Equation 101. The GAN loss can thus be read as a distance between probability distributions.

The GAN loss. Insert $y = 1$ for the real examples x_i and $y = 0$ for the generated samples x_j^* . The discriminator minimizes

$$L[\phi] = \sum_j -\log(1 - \text{sig}[f[x_j^*, \phi]]) - \sum_i \log \text{sig}[f[x_i, \phi]]. \quad (103)$$

The generator wants its samples to be misclassified, so it *maximizes* the same loss with respect to θ , after substituting $x_j^* = g[z_j, \theta]$:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\min_{\phi} \left[\sum_j -\log(1 - \text{sig}[f[g[z_j, \theta], \phi]]) - \sum_i \log \text{sig}[f[x_i, \phi]] \right] \right]. \quad (104)$$

This is a *minimax game*: the discriminator parameters minimize the loss, the generator parameters maximize it. The real-data term does not depend on θ , so the generator’s part reduces to minimizing

$$L[\theta] = \sum_j \log(1 - \text{sig}[f[g[z_j, \theta], \phi]]). \quad (105)$$

Training alternates: a gradient step on ϕ with a minibatch of real and generated images, then a gradient step on θ with new samples, keeping the other network fixed each time. The gradient of $L[\theta]$ flows from the discriminator output back through the discriminator’s layers, into the image x^* , and from there into the generator. Generator backpropagation therefore *depends on the discriminator*; the generator never sees a real image directly and learns only through the discriminator’s judgment. At the ideal end point, the generator draws samples from the real data distribution and the discriminator can only guess, outputting 0.5 everywhere.

For a numerical check, let the discriminator output 0.9 for a real image and 0.2 for a generated one. Their contributions to Equation 103 are $-\log 0.9 \approx 0.105$ and $-\log(1 - 0.2) \approx 0.223$, so $L[\phi] \approx 0.33$. The generator’s term is $\log(1 - 0.2) \approx -0.223$; it decreases if the discriminator’s output for the generated image rises toward 1. At the equilibrium with outputs of 0.5, each example contributes $\log 2 \approx 0.693$. In practice the generator often minimizes $-\log \text{sig}[f[g[z, \theta], \phi]]$ instead of Equation 105. Both push in the same direction, but the alternative gives large gradients early in training, when the discriminator confidently rejects all samples and $\log(1 - \text{sig}[\cdot])$ is almost flat.

Two statements summarize what a GAN does and does not learn. It needs no labeled pairs (x, y) : the labels real and generated are created by the training procedure itself, so an unconditional GAN is unsupervised. And although the distribution of its samples should approach the data distribution, the generator never learns an explicit probability function of the data; it learns a mapping from z to images. Even when all generated samples are indistinguishable from real ones, this does not guarantee that every plausible image can be generated.

8.4 DCGAN and Training Difficulties

The deep convolutional GAN (DCGAN) uses convolutional networks for both roles (Table 30). The generator projects a 100-dimensional latent vector to a $4 \times 4 \times 1024$ tensor and reshapes it. Four fractionally strided (transposed) convolutions each double the resolution and halve the channels until a $64 \times 64 \times 3$ image emerges. A final tanh maps pixel values to $[-1, 1]$. The discriminator mirrors this path with strided convolutions that halve the resolution, a final 4×4 convolution to a single number, and a sigmoid. DCGANs produce convincing faces and bedroom scenes, while samples trained on the diverse ImageNet classes remain blurry object fragments.

Generator stage	Output size	Operation
latent z	100×1	sampled from $\mathcal{N}(0, I)$
project and reshape	$4 \times 4 \times 1024$	linear layer
upsampling 1–3	$8 \times 8 \times 512$ to $32 \times 32 \times 128$	fractionally strided conv.
output	$64 \times 64 \times 3$	fractionally strided conv., tanh
Discriminator stage	Output size	Operation
input x or x^*	$64 \times 64 \times 3$	
downsampling	$32 \times 32 \times 128$ to $4 \times 4 \times 1024$	strided conv.
output	1×1	4×4 conv., sigmoid

Table 30: DCGAN architecture. The generator doubles the resolution and halves the channels at each stage; the discriminator mirrors it.

The latent space of a trained DCGAN has meaningful directions. Averaging the latent vectors

of several “man with glasses” images, subtracting the average for “man without glasses,” and adding that for “woman without glasses” yields vectors that generate women with glasses. The discriminator’s features are also useful for image classification.

Why GANs are hard to train. DCGAN needed special measures to train at all: strided convolutions instead of pooling for down- and upsampling, batch normalization in both networks, leaky ReLU activations, and the Adam optimizer. An MLP generator with a similar number of parameters and layers produced repetitive, low-quality bedrooms. Two failure patterns are typical. In *mode dropping*, the samples cover only a subset of the data, for example only some face types. In *mode collapse*, the generator ignores its latent input and maps every z to nearly the same output. Both are rewarded by the discriminator as long as the few produced images are realistic, because the loss of Equation 105 judges each sample separately.

An analysis of the loss as a distance between distributions shows two effects. Regions with generated samples but no real examples are penalized, so the loss enforces *quality*. Regions with real examples but no samples are also penalized, so it enforces *coverage*. The problem lies in the gradient. If generated and real distributions do not overlap, as for $\theta = 3$ in the one-dimensional example, an optimal discriminator is almost exactly 0 on one group and 1 on the other. Its output is then flat around the samples, and small changes of the generator distribution toward the real one hardly change the loss. The generator receives little guidance about which way to move.

8.5 Wasserstein GANs

A better loss should decrease smoothly as the generated distribution moves toward the real one, even when the two do not overlap. The *Wasserstein distance*, also called the *earth mover’s distance*, has this property. It views each distribution as a pile of earth and measures the minimal cost of reshaping the first pile into the second, where moving a unit of mass costs the distance it travels. For discrete distributions $Pr(x = i)$ and $q(x = j)$, the mass moved from bin i to bin j is stored in a *transport plan* matrix P :

$$D_w[Pr(x) \parallel q(x)] = \min_P \sum_{i,j} P_{ij} |i - j| \quad \text{s.t.} \quad \sum_j P_{ij} = Pr(x = i), \quad \sum_i P_{ij} = q(x = j), \quad P_{ij} \geq 0. \quad (106)$$

The row sums use up the initial distribution, the column sums build the target distribution, and masses cannot be negative. The objective and all constraints are linear in P , so computing D_w is a *linear programming* problem.

For a worked example, take three bins with $Pr = (0.5, 0.5, 0)$ and $q = (0, 0.5, 0.5)$. One optimal plan moves 0.5 from bin 1 to bin 2 and 0.5 from bin 2 to bin 3, with cost $0.5 \cdot 1 + 0.5 \cdot 1 = 1$. Moving 0.5 directly from bin 1 to bin 3 costs $0.5 \cdot 2 = 1$ as well. In one dimension, the result equals the summed absolute difference of the cumulative distributions, $|0.5 - 0| + |1 - 0.5| + |1 - 1| = 1$. Shifting q one bin further right raises the distance to 2. The KL divergence, in contrast, is infinite for both shifted versions because q puts mass where Pr has none; it cannot tell “close” from “far.”

For continuous distributions, the plan becomes a joint density $\pi(x_1, x_2)$ describing the mass moved from x_1 to x_2 :

$$\begin{aligned} D_w[Pr(x), q(x)] &= \min_{\pi} \iint \pi(x_1, x_2) \|x_1 - x_2\| dx_1 dx_2 \\ &= \max_{f: |\partial f / \partial x| \leq 1} \left[\int Pr(x) f[x] dx - \int q(x) f[x] dx \right]. \end{aligned} \quad (107)$$

The right-hand form is the dual linear program. It searches over functions $f[x]$ whose slope is at most 1 (*1-Lipschitz*) for the one that best separates the two distributions by their average values.

A neural network $f[x, \phi]$ can approximate this function. Replacing the integrals with averages over samples gives the *Wasserstein GAN* (WGAN) loss for the network, often called the critic:

$$L[\phi] = \sum_j f[x_j^*, \phi] - \sum_i f[x_i, \phi] = \sum_j f[g[z_j, \theta], \phi] - \sum_i f[x_i, \phi], \quad \left| \frac{\partial f[x, \phi]}{\partial x} \right| < 1. \quad (108)$$

The critic is minimized with respect to ϕ and the generator maximizes it, as before. Three differences from the cross-entropy GAN matter. The critic has *no sigmoid*: its output is a scalar score, not a probability. The Lipschitz constraint bounds its slope to at most 45° , so it cannot become an infinitely steep step between the groups. And because of this bounded slope, the critic keeps a useful gradient even when the two distributions are far apart. In the one-dimensional example, D_w equals the distance between the two clusters, $|7 - \theta|$, and its gradient with respect to θ never vanishes.

In practice, the Wasserstein loss always provides meaningful gradients, reduces the likelihood of mode collapse, and correlates with the visual quality of the samples, so it can be used to monitor training. It does not solve everything. The ground distance $\|x_1 - x_2\|$ is measured between pixel vectors, and Euclidean pixel distance differs from human perceptual similarity. WGAN losses therefore do not perfectly track visual quality, and training still requires care.

8.6 Improving Quality and Diversity

Several further techniques, often combined, allow GANs to synthesize varied, realistic images at high resolution. Table 31 summarizes their purpose.

Technique	Change	Main effect
Wasserstein loss	critic without sigmoid, 1-Lipschitz	useful gradients, less mode collapse
Progressive growing	add layers for higher resolution	stable high-resolution training
Minibatch discrimination	batch statistics in discriminator	diversity
Truncation τ	reject far-out latent values	quality at the cost of diversity
Precision / recall	compare real and synthetic manifolds	measure quality and coverage separately

Table 31: Improvements to GAN training and evaluation. Most trade off or separately control sample quality and diversity.

Progressive growing. Training starts with a generator and discriminator for 4×4 images. Once they are stable, both receive additional layers for the next resolution, 8×8 , and so on up to 1024×1024 . The new higher-resolution layers *fade in* over several iterations, mixed with an upsampled version of the previous output, so they do not disturb what the lower layers already learned. Coarse structure is thus settled before fine detail is learned.

Minibatch discrimination. Mode collapse goes unnoticed if the discriminator judges images one at a time. With minibatch discrimination, the discriminator also computes feature statistics across the whole minibatch, separately for real and synthesized data, and uses them in its decision. A batch of near-identical samples then looks unlike a batch of real images, and the generator is pushed to produce the same variety as the real data.

Truncation. Samples from the tails of the latent distribution tend to be less realistic. Rejecting latent values that fall further than τ standard deviations from the mean trades diversity for quality. With $\tau = 2$, generated dogs vary strongly in pose and background; with $\tau = 0.04$, they all look nearly the same, but each is very clean.

Manifold precision and recall. Quality and diversity can be measured separately. Approximate the real manifold by the union of small spheres around real examples, and the synthetic manifold likewise around generated samples, usually in a feature space rather than in pixel space. *Precision* is the fraction of generated samples that fall inside the real manifold and measures quality. *Recall* is the fraction of real examples that fall inside the synthetic manifold and measures coverage. Truncation raises precision and lowers recall.

Latent interpolation. Moving smoothly along a straight line between two latent vectors produces realistic intermediate images, e.g. one car gradually turning into another. Smooth interpolation shows that the generator has learned a continuous representation rather than memorizing the individual training images.

8.7 Conditional Generation

The GANs discussed so far cannot be asked for an image of a specified kind; a random z yields a random class. Three variants add an attribute c , for example a class, to control the output. Table 32 compares them.

A *conditional GAN* passes c to both networks, $g[z, c, \theta]$ and $f[x, c, \phi]$. For the generator, c is simply appended to z . For the discriminator, c is first linearly transformed to a 2D map and added as an extra channel of the input image. The discriminator then judges whether the *pair* (x, c) is real. A generator that ignores c is caught, because a realistic cat paired with the label “dog” is not a real pair.

The *auxiliary classifier GAN* (ACGAN) keeps the discriminator input image-only but requires it to predict the attribute. For a discrete attribute with C categories, the discriminator has $C + 1$ outputs: one sigmoid output for real versus generated and a softmax over C classes. The generator is trained so that its samples are both judged real and classified as the class it was asked for. ACGANs trained on ImageNet synthesize recognizable examples of many classes, such as butterflies, birds, and flowers.

InfoGAN tries to discover attributes automatically, without labels. The generator takes random noise z together with a random attribute vector c , whose components may be discrete, like a class, or continuous. Besides the real/generated output, the discriminator has a second head that estimates c from the generated image. To make this estimate possible, the generator must make c visible in its output, and c becomes a meaningful factor of variation. On handwritten digits, a discrete c with ten values ends up selecting the digit, and continuous components control stroke slant and thickness, even though no labels were used.

Model	Attribute c	Discriminator input	Discriminator output
Conditional GAN	given (labels)	image and c	pair is real
ACGAN	given class	image	real, and class softmax ($C + 1$)
InfoGAN	random, no labels	image	real, and estimate of c

Table 32: Three ways to condition a GAN. They differ in whether the attribute is labeled and in how the discriminator checks it.

8.8 Image Translation

The adversarial discriminator was introduced to generate random samples, but it also serves as a learned prior that favors *realism* when one image is translated into another. The generator then receives an image c instead of, or in addition to, noise, and produces an output image. A content loss keeps the output related to its input or to a target, and an adversarial loss makes it look real.

Pix2Pix. Pix2Pix maps an image c to an image x of a different style with a U-Net $g[c, \theta]$ (Section 6). Examples are colorization, map to aerial photo, sketch to product photo, and facade label map to facade photo. Training requires *paired* examples: for each input, the true output image is known. Two losses combine. The content loss $\|x - g[c, \theta]\|_1$ compares the prediction with the ground truth. The ℓ_1 norm alone gives blurry results, because averaging several plausible outputs minimizes it. The adversarial discriminator $f[c, x, \phi]$ therefore receives the before image and an after image and learns to distinguish real before/after pairs from before/synthesized pairs; its feedback trains the generator as usual. The input c plays the role of the condition in a conditional GAN.

Transcoding radar to optical images. Image translation also produces features for downstream tasks. Radar satellites such as Sentinel-1 see through clouds, but their speckled images are hard to interpret; optical Sentinel-2 images are easy to read but often cloudy. A GAN trained on co-registered pairs *transcodes* radar tiles into synthetic optical tiles; the optical images are needed only for training. The generator’s learned radar features are then reused for land-cover classification. With only a few hand-labeled samples, a classifier built on these pretrained features, retraining only its last layers, reached about 0.85 accuracy even with 1/16 of the labeled data. Random forests and CNNs trained from scratch reached only about 0.4 to 0.7, and their accuracy dropped as labeled data decreased. The GAN acts here as a pretext task in the sense of the first subsection.

Super-resolution GAN. SRGAN upsamples a low-resolution image by a factor of four. The content loss compares the prediction with the true high-resolution image, often in the feature space of a pretrained VGG network rather than per pixel. The discriminator differs from Pix2Pix: it does not judge before/after pairs, but distinguishes the synthesized image from *other* real high-resolution images. The adversarial loss thus penalizes any output that looks synthetic. Compared with bicubic upsampling, SRGAN produces sharp textures such as individual branches and fabric patterns instead of blur.

CycleGAN. Often two image sets with distinct styles exist, but no matching pairs: photos and Monet paintings, or horses and zebras. Pix2Pix cannot be trained without pairs. CycleGAN relies on a consistency idea: translating a photo into a painting and then back into a photo should recover the original. It trains *two generators*, g from domain A to B and g' from B to A , and *two discriminators*, one per domain, all simultaneously (Figure 40). For an input c from domain A , its loss is a weighted sum of three terms. A content loss encourages the translated image $g[c]$ to agree with c , so the scene is preserved. An adversarial loss encourages $g[c]$ to be indistinguishable from real images of domain B . The cycle-consistency loss $\|g'[g[c]] - c\|_1$ encourages the mapping to be reversible. The same three terms are applied in the other direction, so there are two content losses and two adversarial losses.

StyleGAN. A usual GAN feeds the latent variable only at the input of the generator. StyleGAN instead injects latent information at *various points* of the architecture and separates two kinds of variation. The *style* carries variation salient to humans. A fully connected mapping network

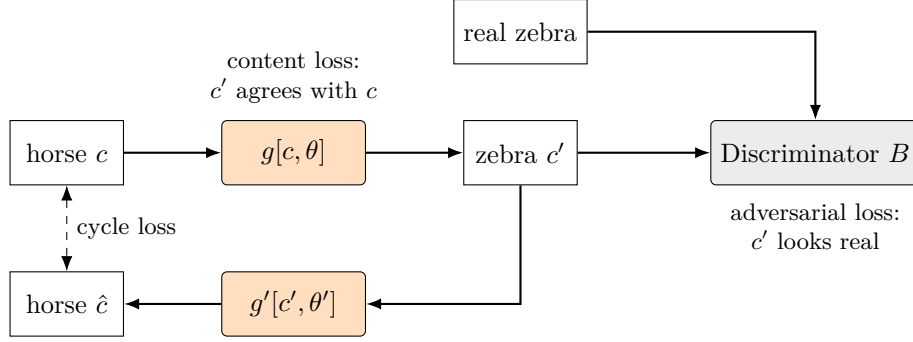


Figure 40: CycleGAN, direction horse to zebra. The generator g translates, a discriminator judges realism in the zebra domain, and the second generator g' maps back so that the cycle loss can compare $\hat{c}$ with c . The reverse direction adds a second discriminator for horses.

first turns the latent z into an intermediate variable w of size 512. At each scale, a learned linear transform turns w into a style vector y_k of 2×512 values: a per-channel scale and offset. These are applied after normalizing each channel of the feature map, as in instance normalization (Table 25); the combination is called *adaptive instance normalization*. *Noise* represents unimportant variation: at each scale, fresh random noise maps, multiplied by learned per-channel factors, are added to every channel. For faces, styles at coarse scales control face shape and head pose, styles at medium scales the shape and details of facial features, and styles at fine scales hair and skin color. Changing the noise leaves the identity unchanged and only varies small details such as the exact placement of hair strands and skin pores.

What the translation models need. Table 33 compares the translation models. Only Pix2Pix and SRGAN require paired ground truth; CycleGAN learns from unpaired image sets, and unconditional GANs need no labels at all. Thus not all GANs require labeled tuples (x, y) .

Model	Training data	Losses	Discriminator judges
Pix2Pix	paired images	ℓ_1 content, adversarial	before/after pair is real
SRGAN	paired low/high res.	VGG content, adversarial	image looks like real high-resolution image
CycleGAN	two unpaired sets	content, adversarial, cycle (each twice)	image looks like its domain (two discriminators)
StyleGAN	unlabeled images	adversarial	image looks real

Table 33: Image translation and style models. Paired data allow a direct content loss; unpaired data require cycle consistency.

9 Diffusion Models

A GAN turns random numbers into an image in a single pass of its generator and learns this mapping through an adversarial game. A diffusion model reaches the same goal on a very different path. It first destroys images on purpose: over many small steps, it mixes a training image with Gaussian noise until nothing but noise is left. This destruction needs no learning at all. A neural network then learns to undo one small step at a time. To generate a new image, one starts from pure noise and applies the learned denoising step again and again until a clean image appears. Each individual step is an easy regression problem with a known target, so training is as stable as ordinary supervised learning. This section derives the forward noising process, the noise prediction objective, and the reverse sampling procedure, then shows how Stable Diffusion runs the process in a compressed latent space conditioned on text, and how low-rank adaptation (LoRA) fine-tunes such a large model on a small image set.

9.1 Destroying and Recreating Images

Figure 41 shows the two directions. The *forward* or *diffusion process* maps the input image x_0 through a chain of latent variables $x_1, x_2, \dots, x_T$. Each step adds a little noise, and after T steps (typically $T = 1000$) the image x_T is indistinguishable from a sample of a standard normal distribution. In the language of the latent variable models of Section 8, this chain acts as an *encoder*, but a fixed one without parameters. The *reverse process* is the decoder. It passes the data back through the same latent variables and removes noise at each step. Only the reverse process is learned.

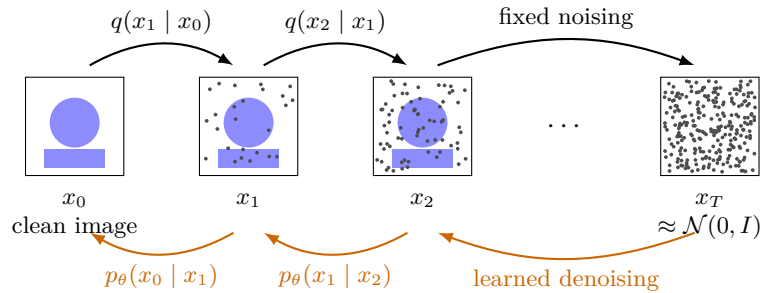


Figure 41: Diffusion model. The forward process (black, top) adds a little Gaussian noise per step until the image is pure noise x_T . The reverse process (orange, bottom) is a learned network that removes noise step by step and so turns a noise sample into an image.

Why go through a thousand small steps instead of mapping noise to an image in one pass, as a GAN does? A single jump from noise to a realistic image is a hard, highly ambiguous mapping. A small denoising step, in contrast, only has to make a slightly noisy image a little cleaner. Because every step adds only a small amount of noise, the reverse of a step is well approximated by a Gaussian, and a network can predict it with a plain squared-error loss. There is no discriminator, no minimax game, and no mode collapse: every training image is noised and must be reconstructed, so the model is pushed to cover the whole data distribution. Diffusion models belong to the probabilistic generative models of Figure 38. Their training objective is a simplified bound on the likelihood, so, like a VAE, they learn an approximation of the data distribution. Table 34 compares the three families. The price for stable training is slow sampling: generating one image requires many network evaluations instead of one.

9.2 The Forward Noising Process

Let $\mathcal{D} = \{x_0^{(1)}, \dots, x_0^{(N)}\}$ be the training images. The forward process is controlled by a *noise schedule*, a fixed sequence of small variances

$$0 < \beta_1 < \dots < \beta_T < 1, \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s. \quad (109)$$

A common choice is a linear schedule from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ with $T = 1000$. One forward step shrinks the current image a little and adds fresh Gaussian noise of variance β_t :

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I), \quad \text{i.e.} \quad x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t. \quad (110)$$

The shrinking factor $\sqrt{1 - \beta_t}$ is chosen so that the variance stays constant. If x_{t-1} has unit variance per pixel, then x_t has variance $(1 - \beta_t) + \beta_t = 1$. Without shrinking, the variance would grow with every step, and the end point would not be a standard normal distribution.

	GAN	VAE	Diffusion model
Latent space	low-dim. z	low-dim. z	same size as image, T copies
Encoder	none	learned	fixed noising, no parameters
Training	adversarial minimax	likelihood bound	likelihood bound, reduced to MSE on noise
Stability	fragile, mode collapse	stable	stable
Density	no	approximate	approximate
Sample quality	sharp	often blurry	sharp, state of the art
Sampling cost	one pass	one pass	many passes (T or ≈ 50)

Table 34: Three families of generative models. Diffusion models combine the stable likelihood-based training of VAEs with the sample quality of GANs, but pay with many network evaluations per sample.

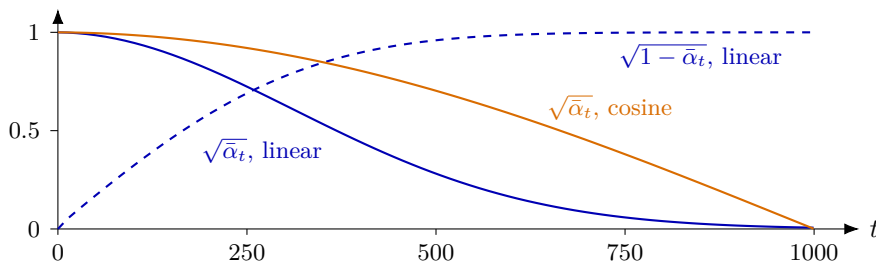


Figure 42: Signal coefficient $\sqrt{\alpha_t}$ (solid) and noise coefficient $\sqrt{1 - \alpha_t}$ (dashed) for the linear schedule with $T = 1000$, and the signal coefficient of the cosine schedule (orange). The linear schedule destroys the image early and wastes many late steps on almost pure noise; the cosine schedule spreads the destruction more evenly.

Closed form for any time step. Running a thousand steps to produce one training example would be wasteful. Fortunately, a sum of independent Gaussians is again Gaussian, so the chain collapses. Two steps give

$$x_t = \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{\beta_{t-1}} \varepsilon_{t-1}) + \sqrt{\beta_t} \varepsilon_t = \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t \beta_{t-1}} \varepsilon_{t-1} + \sqrt{\beta_t} \varepsilon_t.$$

The two noise terms are independent, so their sum is one Gaussian with variance $\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = 1 - \alpha_t\alpha_{t-1}$. The two steps thus act like a single step with signal factor $\sqrt{\alpha_t\alpha_{t-1}}$. By induction over all steps back to x_0 ,

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad \text{i.e.} \quad q(x_t | x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I\right). \quad (111)$$

This is the key result of the forward process: any noise level x_t can be sampled directly from x_0 with one Gaussian draw. The tensor ε has the same shape as the image, with one independent Gaussian value per pixel and channel. It is a *concrete sampled noise tensor*, not an abstract distribution. The two coefficients split the image into a signal part and a noise part whose squares always sum to one. The *signal-to-noise ratio* $\bar{\alpha}_t/(1 - \bar{\alpha}_t)$ falls monotonically from very large at $t = 1$ to almost zero at $t = T$.

Worked example. Take one pixel with value $x_0 = 0.8$ at a time step with $\bar{\alpha}_t = 0.64$, and suppose the sampled noise is $\varepsilon = -0.5$. Then $x_t = \sqrt{0.64} \cdot 0.8 + \sqrt{0.36} \cdot (-0.5) = 0.64 - 0.30 = 0.34$. For the linear schedule, $\ln \bar{\alpha}_T = \sum_t \ln(1 - \beta_t) \approx -\sum_t \beta_t$. The sum of the 1000 linearly spaced values is $1000 \cdot (10^{-4} + 0.02)/2 \approx 10.05$, so $\bar{\alpha}_T \approx e^{-10} \approx 4.5 \cdot 10^{-5}$ and the image coefficient $\sqrt{\bar{\alpha}_T}$ is below 0.007. At the end of the chain almost nothing of x_0 survives, and $q(x_T | x_0) \approx \mathcal{N}(0, I)$ regardless of the input image. This is exactly the distribution from which generation will start.

Choice of schedule. Figure 42 shows that the linear schedule removes most of the image within the first half of the chain. The *cosine schedule* defines $\bar{\alpha}_t$ directly, $\bar{\alpha}_t = f(t)/f(0)$ with $f(t) = \cos^2\left(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\right)$ and a small offset $s = 0.008$. It changes slowly near $t = 0$ and $t = T$ and gives better image quality at both ends. More recent schedules (sigmoid, EDM) are tuned for fast sampling. Whatever the schedule, it is fixed before training and is not learned.

9.3 Training a Network to Predict the Noise

Solving Equation 111 for the noise gives the training target:

$$\varepsilon = \frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{\sqrt{1 - \bar{\alpha}_t}}. \quad (112)$$

This is the normalized Gaussian noise component that corrupted x_0 into x_t . In the worked example, $(0.34 - 0.64)/0.6 = -0.5$ recovers the sampled value. During training this ground truth is known exactly, because we sampled ε ourselves when constructing x_t . A neural network $\varepsilon_\theta(x_t, t)$ receives the noisy image and the time step and predicts the noise; its loss is the mean squared error

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, \varepsilon} \left[\left\| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t) \right\|^2 \right]. \quad (113)$$

The expectation runs over training images, uniformly drawn time steps $t \in \{1, \dots, T\}$, and noise samples. The full derivation starts from a bound on the likelihood, as in Equation 98, and contains a different weight for each t . Dropping these weights gives this *simplified loss*, which works better in practice. One network with shared parameters θ serves all noise levels, so it must be *time-conditioned*. Figure 43 shows one training step.

Training thus repeats a short loop:

1. draw a training image $x_0 \sim \mathcal{D}$;
2. draw a time step $t \sim \text{Uniform}\{1, \dots, T\}$;
3. draw noise $\varepsilon \sim \mathcal{N}(0, I)$;
4. form $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$;

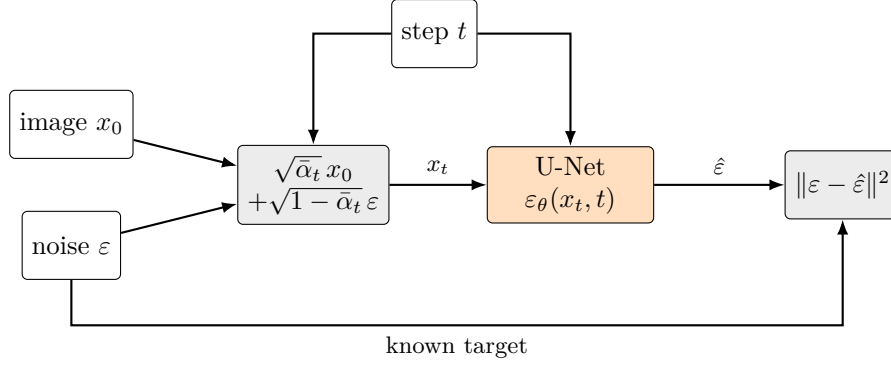


Figure 43: One training step. A clean image, a noise tensor, and a time step are drawn; the mixture x_t goes into the network, and its noise estimate $\hat{\varepsilon}$ is compared with the noise that was actually added.

5. compute $\ell = \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ and take a gradient step on θ .

The chain is never run step by step during training. Every example is an independent regression problem with input (x_t, t) and target ε .

What the network actually learns. The network cannot know the exact noise that was drawn. Many clean images x_0 , each combined with a suitable noise tensor, lead to the same noisy image x_t , and at high noise levels a blurry blob is compatible with a cat, a dog, or a car. For squared error, the best possible prediction is the conditional mean $\varepsilon_\theta(x_t, t) = \mathbb{E}[\varepsilon | x_t]$, averaged over all clean images that could have produced x_t under the training distribution. The network therefore does not memorize an inverse of the corruption. It learns the statistically best *denoising direction*: the direction in which x_t must move to become more like a natural image. In other words, it learns the structure of natural images. This has a precise counterpart. For the Gaussian of Equation 111,

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{x_t - \sqrt{\alpha_t} x_0}{1 - \alpha_t} = -\frac{\varepsilon}{\sqrt{1 - \alpha_t}}, \quad (114)$$

so predicting the noise is, up to a scale factor, the same as learning the *score*, the gradient of the log density at each noise level. The score points uphill in probability, toward regions where natural images lie. This links diffusion models to score-based generative models. Because x_t and $\hat{\varepsilon}$ determine each other, a noise prediction can also be turned into an estimate of the clean image,

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \alpha_t} \varepsilon_\theta(x_t, t)}{\sqrt{\alpha_t}}, \quad (115)$$

which at large t is a blurry average of plausible images and at small t is sharp.

The U-Net backbone. The noise prediction has the same size as the input, and it must be accurate both in fine detail and in global layout. The U-Net of Figure 36 meets both requirements, and it is the standard denoising network. The diffusion U-Net modifies the segmentation U-Net in a few places. Its encoder consists of residual blocks with spatial downsampling, for example from 64×64 over 32×32 and 16×16 down to 8×8 . At the low resolutions (16×16 , 8×8), self-attention layers (Section 7) capture global context at moderate cost, since the number of attention pairs grows with the square of the number of positions. The decoder upsamples and receives skip connections from the encoder. The time step t is encoded with sinusoidal features, as in the positional encoding of Equation 88, passed through a small MLP, and added to the feature maps of every residual block. The same weights therefore behave differently at different noise levels: at large t they sketch coarse layout, at small t they refine texture.

9.4 Generating Images by Reverse Denoising

To generate a new image, start from pure Gaussian noise, $x_T \sim \mathcal{N}(0, I)$, and apply learned reverse steps $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$. The true reverse distribution $q(x_{t-1} | x_t)$ is intractable, because it depends on the unknown data distribution. Since each forward step adds only little noise, it is approximated by a Gaussian $p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$, whose mean is computed from the noise prediction:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I). \quad (116)$$

Read from the inside out, each step does four things: look at the current noisy image x_t , ask the network to estimate the noise component, subtract the fraction $\beta_t/\sqrt{1 - \bar{\alpha}_t}$ of that noise, and undo the shrinking of the forward step with the factor $1/\sqrt{\alpha_t}$. Only a part of the predicted noise is removed per step, because the network’s estimate is unreliable at high noise; the next step looks again with a better view. The added term $\sigma_t z$ with fresh noise, usually $\sigma_t^2 = \beta_t$, keeps the process stochastic. It corresponds to sampling from p_θ instead of taking its mean, and it lets different runs from the same start produce different details. At the last step no noise is added ($z = 0$ for $t = 1$), and x_0 is returned.

Sanity check at $t = 1$. At the first step $\bar{\alpha}_1 = \alpha_1 = 1 - \beta_1$, so the coefficient in Equation 116 becomes $\beta_1/\sqrt{\beta_1} = \sqrt{\beta_1}$, and without the noise term $x_0 = (x_1 - \sqrt{\beta_1} \varepsilon_\theta)/\sqrt{\alpha_1}$. This is the exact inverse of the forward step $x_1 = \sqrt{\alpha_1} x_0 + \sqrt{\beta_1} \varepsilon$ of Equation 110. If the network predicted the noise perfectly, the last step would restore the image exactly. For larger t the formula removes only the share of the total noise that belongs to one step.

Sampling cost and DDIM. Equation 116 must be applied $T = 1000$ times, and every application is a full U-Net forward pass, a *network function evaluation* (NFE). A GAN needs one. Denoising diffusion implicit models (DDIM) reduce this cost without retraining. They first estimate the clean image with Equation 115 and then re-noise it deterministically to a lower noise level, reusing the predicted noise direction:

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}} \varepsilon_\theta(x_t, t), \quad t' < t. \quad (117)$$

Since no fresh noise enters, the jump from t to t' need not be a single step. A subsequence of about 50 time steps gives images of similar quality to the full 1000-step chain. The same start noise x_T always yields the same image, which makes DDIM sampling reproducible. Table 35 summarizes training and generation.

	Training	Generation
Start	training image x_0	$x_T \sim \mathcal{N}(0, I)$
Noise	sampled ε , known	unknown, estimated by ε_θ
Time steps	one random t per example	all $t = T, \dots, 1$ (DDIM: ≈ 50)
Network use	one forward and backward pass	one forward pass per step
Formula	Eq. 111, loss Eq. 113	Eq. 116 or Eq. 117
Result	$\varepsilon_\theta(x_t, t) \approx \varepsilon$	new image x_0

Table 35: Training versus generation. Training jumps directly to a random noise level; generation must walk the whole chain back.

In one sentence: the trained object is the network ε_θ that predicts, for every noise level t , the noise tensor that should be removed to move a sample back toward the learned image distribution.

9.5 Latent Diffusion and Stable Diffusion

A diffusion model working directly on pixels is expensive. A U-Net must process a $512 \times 512 \times 3$ tensor at every one of many steps, both in training and in generation. Most of these pixel values encode fine detail that a good autoencoder can reproduce from a much smaller code. *Latent diffusion models* (LDM) therefore separate the problem into two stages.

In the first stage, a variational autoencoder with encoder $\mathcal{E}$ and decoder $\mathcal{D}$ is trained to compress and reconstruct images: $z = \mathcal{E}(x) \in \mathbb{R}^{h \times w \times c}$ and $\tilde{x} = \mathcal{D}(z)$. A typical downsampling factor $f = 8$ turns $512 \times 512 \times 3$ into $64 \times 64 \times 4$. The latent has 64 times fewer spatial positions and 48 times fewer values, yet the reconstruction is nearly lossless for human perception. In the second stage, the diffusion process of the previous subsections runs entirely in this latent space. The loss is Equation 113 with z in place of x ,

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{z, \varepsilon, t} [\|\varepsilon - \varepsilon_{\theta}(z_t, t, \tau_{\theta}(y))\|^2], \quad z = \mathcal{E}(x), \quad (118)$$

where $\tau_{\theta}(y)$ encodes a condition y such as a text prompt. The work is thus divided: the VAE performs the actual compression and decompression, and the U-Net performs multiscale noise prediction on small tensors. Stable Diffusion is such a latent diffusion model for text-to-image generation.

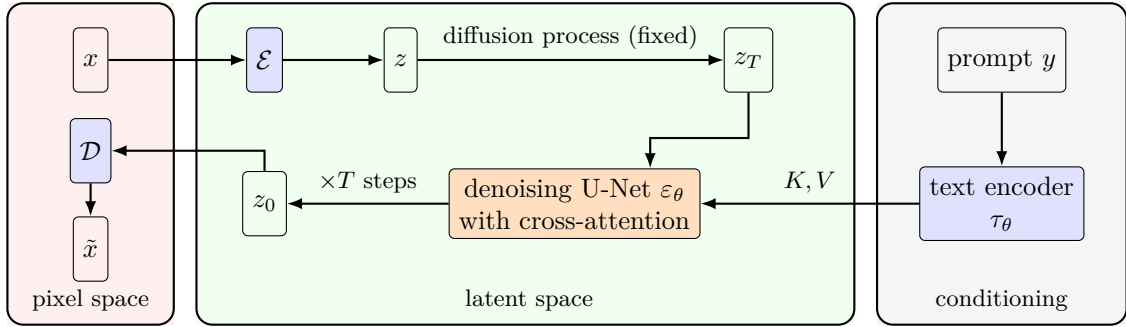


Figure 44: Stable Diffusion. The VAE encoder $\mathcal{E}$ maps an image into latent space, where noise is added. The U-Net removes it step by step, guided by the text encoding through cross-attention. The decoder $\mathcal{D}$ converts only the final latent z_0 back into an image. During generation, the upper path is skipped and z_T is drawn from $\mathcal{N}(0, I)$.

Component	Input $\rightarrow$ output	Responsible for
VAE ($\mathcal{E}, \mathcal{D}$)	$512^2 \times 3 \leftrightarrow 64^2 \times 4$	perceptual quality: edges, textures, fine detail
U-Net ε_{θ}	$(z_t, t, \tau_{\theta}(y)) \rightarrow \hat{\varepsilon}$	semantic content: layout, identity, style
Text encoder τ_{θ}	prompt $\rightarrow$ token embeddings	open-vocabulary grounding of the prompt

Table 36: Division of labor in Stable Diffusion. Only the U-Net is iterated; the text encoder runs once per prompt and the decoder once per image.

Processing flow. Figure 44 shows the three components and how data flows through them. During training, the VAE encoder compresses each image into a latent tensor, the forward process noises this latent, and the U-Net learns to predict the noise, conditioned on the caption of the image. During generation, the encoder is not needed at all: a random latent z_T is denoised by

the U-Net over many steps, conditioned on the prompt, and the VAE decoder is used only once, at the very end, to convert the final denoised latent into an image.

Conditioning by cross-attention. The prompt is tokenized and turned into a sequence of embeddings by a pretrained text encoder τ_θ ; Stable Diffusion uses the CLIP text encoder, which was trained to align images with their captions. Inside the U-Net, cross-attention layers (Equation 96) connect the two modalities. The queries come from the spatial U-Net features, and keys and values come from the text tokens, $\text{Attn}(Q, K, V)$ with $Q = W^Q \varphi(z_t)$ and $K = W^K \tau_\theta(y)$, $V = W^V \tau_\theta(y)$. Every image position can thus look up the words relevant to it: the region that becomes the dog attends to “dog,” the background to “forest.” The same mechanism accepts other conditions, such as semantic maps or reference images, if a suitable encoder τ_θ is used. Table 36 lists what each component contributes.

Classifier-free guidance. A conditional model $\varepsilon_\theta(z_t, t, y)$ tends to follow the prompt only loosely. Classifier-free guidance strengthens the condition without any extra classifier. During training, the condition is replaced by an empty prompt $\emptyset$ with a probability of about 0.1, so one network learns both the conditional and the unconditional prediction. At inference, both are evaluated and extrapolated:

$$\tilde{\varepsilon} = (1 + w) \varepsilon_\theta(z_t, t, y) - w \varepsilon_\theta(z_t, t, \emptyset) = \varepsilon_\theta(z_t, t, \emptyset) + (1 + w)(\varepsilon_\theta(z_t, t, y) - \varepsilon_\theta(z_t, t, \emptyset)). \quad (119)$$

The difference between the two predictions is the direction in which the prompt pushes the image; the guidance scale w amplifies it. With $w = 0$ the plain conditional model is used, with higher diversity but weaker prompt adherence. Large w makes images very faithful to the prompt but less diverse and eventually oversaturated or distorted. The default guidance scale of Stable Diffusion is 7.5; software libraries usually quote the factor in front of the difference, i.e. $1 + w$. Guidance doubles the cost per step, since two predictions are needed.

9.6 Fine-Tuning with Low-Rank Adaptation

A pretrained Stable Diffusion model knows many concepts, but not a particular dog. Teaching it a new subject from about 100 photographs by updating all of its roughly one billion parameters needs much GPU memory and easily destroys what the model already knows. The transfer strategies of Section 7 offer a spectrum between full fine-tuning and a frozen backbone (Table 37). *Low-rank adaptation* (LoRA) keeps the pretrained weights frozen and learns only a small correction for selected weight matrices, typically the attention projections of the U-Net.

Let a pretrained layer have weight matrix $W_0 \in \mathbb{R}^{k \times d}$. Full fine-tuning would update all $k \cdot d$ entries. LoRA instead writes

$$W = W_0 + \Delta W, \quad \Delta W = BA, \quad A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{k \times r}, \quad r \ll \min(d, k), \quad (120)$$

and trains only A and B . The layer computes $Wx = W_0x + B(Ax)$: the input is projected down to r dimensions and back up. The correction ΔW has rank at most r , so it can change the layer only along r directions. This restriction works because adapting a pretrained model to a narrow new task usually needs only a small change of low intrinsic dimension. B is initialized with zeros, so $\Delta W = 0$ at the start and training begins exactly at the pretrained model. After training, BA can be added to W_0 once, so inference costs no extra time.

Worked example. For an attention projection with $d = k = 1280$, full fine-tuning updates $1280^2 = 1,638,400$ parameters. LoRA with $r = 8$ trains $r(d + k) = 8 \cdot 2560 = 20,480$ parameters, about 1.25 % of the full matrix. The gradients and optimizer state shrink by the same factor, which makes training feasible on a simple GPU, and the stored result is a file of a few megabytes that is loaded on top of the unchanged base model.

Strategy	What is trained	Typical use
Full fine-tuning	all weights	large target dataset, enough memory
Linear probing	only a new linear head	fixed features, e.g. classification
Prompt/prefix tuning	a few learnable input tokens	steer a frozen transformer
LoRA	low-rank A , B beside frozen W_0	adapt a large generative model on small data and a small GPU

Table 37: Adaptation strategies for large pretrained models, ordered from most to fewest changed weights (prompt tuning and LoRA are both very light). LoRA changes the behavior of every adapted layer while training only a tiny fraction of the parameters.

Practical example: a personalized dog. The task is to adapt a pretrained Stable Diffusion v1.5 model so that it generates new images of one dog, using 100 photos of it. Every training image receives the same caption, “a photo of sks dog.” The token “sks” is a rare identifier that carries almost no prior meaning, so the model binds the appearance of this dog to it without overwriting the general concept “dog.” Training runs the loss of Equation 118 on the dog photos, but gradients update only the LoRA matrices. A fine-tuning run looks like this:

```
python finetune_lora_sd.py \
  --pretrained_model stable-diffusion-v1-5/stable-diffusion-v1-5 \
  --train_dir data/dogs --output_dir dog_lora \
  --caption "a photo of sks dog" \
  --validation_prompt "a photo of sks dog running through snow" \
  --max_train_steps 800 --learning_rate 1e-4 --lora_rank 8 \
  --batch_size 1 --gradient_accumulation_steps 4 \
  --mixed_precision
```

Table 38 explains the settings. The batch size of 1 with 4 accumulation steps sums the gradients of 4 images before each update, so the effective batch size is 4 while memory holds only one image at a time. With 800 update steps the model sees $800 \cdot 4 = 3200$ images, i.e. about 32 passes over the 100 photos. The validation prompt combines the new token with a scene that never occurs in the training photos; it tests whether the model has learned the dog as a reusable concept or merely memorized the training pictures.

Setting	Meaning
--caption	shared caption; binds the new subject to “sks”
--lora_rank 8	rank r of $\Delta W = BA$
--learning_rate 1e-4	step size for A and B only
--max_train_steps 800	number of parameter updates
--batch_size 1	images per forward/backward pass
--gradient_accumulation_steps	passes summed per update (4); effective batch size $1 \cdot 4 = 4$
--mixed_precision	16-bit arithmetic where safe; less memory, faster
--seed 123 (prediction)	fixes the start noise z_T , so results are reproducible

Table 38: Settings of the LoRA fine-tuning and prediction runs. All of them aim at training a large model with the memory of a small GPU.

For prediction, the LoRA weights are loaded on top of the base model and new prompts are sampled:

```
python predict_lora_sd.py \
  --pretrained_model stable-diffusion-v1-5/stable-diffusion-v1-5 \
  --lora_dir dog_lora \
  --prompt "a photo of sks dog sitting in a forest, sharp, natural light" \
  --output_dir dog_predictions --num_images 4 --seed 123
```

Each of the four images starts from a different noise latent, derived from the fixed seed, and is denoised under the prompt with classifier-free guidance. The results show dogs in a forest scene that did not appear in the training set. The pretrained model contributes the forest, the lighting, and the composition; the LoRA correction contributes the appearance of the dog. If the generated dogs vary in breed or coloring, the adaptation is still too weak, for example because of too few steps or too small a rank; if every image reproduces a training photo, it has overfitted.

10 Weakly Supervised Learning

Every classifier so far was trained on a dataset in which each example carries a correct and precise label. Such datasets are expensive. A pixel-accurate segmentation mask takes minutes per image, medical labels need experts, and labels collected cheaply from web tags or crowd workers are often wrong. In practice one usually has something weaker: many unlabeled images and only a few labeled ones, labels for a whole image but not for the object inside it, or labels of which some fraction is simply wrong. *Weakly supervised learning* (WSL) collects the methods that still train useful models from such data. This section first sorts weak supervision into three types, then treats each in turn: unlabeled data in active, semi-supervised, and transductive learning; coarse labels in multi-instance learning; and wrong labels, where early-learning regularization and negative learning prevent a network from memorizing the errors. It ends with a critical look at how much weak supervision actually gains compared with a few clean labels.

10.1 Three Types of Weak Supervision

Weak supervision can fail to be “strong” in three different ways. Figure 45 shows them side by side for a binary problem with labels Y and N; each bar stands for one feature vector.

- *Incomplete supervision*: only a small subset of the training data is labeled; the rest has no label at all. Example: 1000 labeled and 100 000 unlabeled images of skin lesions.
- *Inexact supervision*: labels exist but are coarser than the task requires. Example: the image is tagged “tiger,” but the task is to find the tiger pixels or patches.
- *Inaccurate supervision*: every example has a label of the desired granularity, but some labels are wrong. Example: web images labeled by the surrounding text.

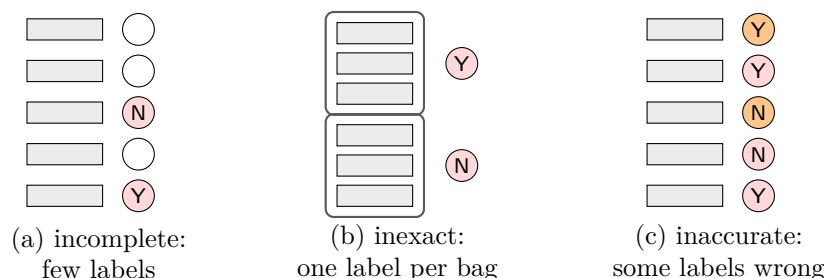


Figure 45: The three types of weak supervision. Bars are feature vectors, circles their labels. (a) Most examples have no label. (b) Labels belong to bags of instances, not to the instances. (c) Every example has a label, but some are wrong (orange); the learner does not know which.

The types call for different tools, summarized in Table 39. Incomplete supervision asks how unlabeled data can help at all. Inexact supervision asks how a label for a group transfers to its members. Inaccurate supervision asks how to stop a network from learning the wrong labels. Real datasets often combine the types, e.g. a few noisy image-level tags and many untagged images.

Type	What is given	Central question	Methods here
Incomplete	few labeled, many unlabeled examples	what do unlabeled data reveal?	active learning, semi-supervised and transductive learning
Inexact	labels for bags (e.g. images), task on instances (patches)	which instance caused the label?	multi-instance learning
Inaccurate	a label for every example, some wrong	how to avoid learning the errors?	early-learning regularization, negative learning, SelNLPL

Table 39: Taxonomy of weak supervision. Each type has its own question and its own family of methods.

10.2 Incomplete Supervision: Learning with Unlabeled Data

With incomplete supervision, the training set consists of n labeled examples $\{(x_i, y_i) : i = 1, \dots, n\}$ and m unlabeled examples $x'_1, \dots, x'_m$, usually with $m \gg n$. There are two ways to use the unlabeled part. *Active learning* assumes an *oracle*, typically a human expert, who can be asked for the true label of selected unlabeled examples. The learner picks the examples whose labels would help most, for instance those closest to the current decision boundary, and asks only for those. The goal is a good model with as few queries as possible. *Semi-supervised learning* uses the unlabeled data automatically, without any human in the loop. It comes in two variants that differ in what the unlabeled data are and what the output is.

- *Pure semi-supervised learning* works under an *open-world* assumption. The test data are unknown at training time; the unlabeled data are not necessarily the test data. The output is a classifier f that can label any future input.
- *Transductive learning* works under a *closed-world* assumption. The test data are given in advance, and they are exactly the unlabeled examples. The goal is to label them as well as possible; no general function f is needed.

Figure 46 compares the three workflows. In all three, a model is trained from labeled and unlabeled data. They differ in whether a human adds labels during training and in what the model predicts afterwards.

Induction versus transduction. The difference between the settings is clearest when written as mappings. Ordinary supervised learning is *induction*: from labeled examples, infer a general function,

$$\{(x_i, y_i) : i = 1, \dots, n\} \mapsto f. \quad (121)$$

Pure semi-supervised learning is induction with unlabeled data,

$$\{(x_i, y_i) : i = 1, \dots, n\} \cup \{x'_1, \dots, x'_m\} \mapsto f, \quad (122)$$

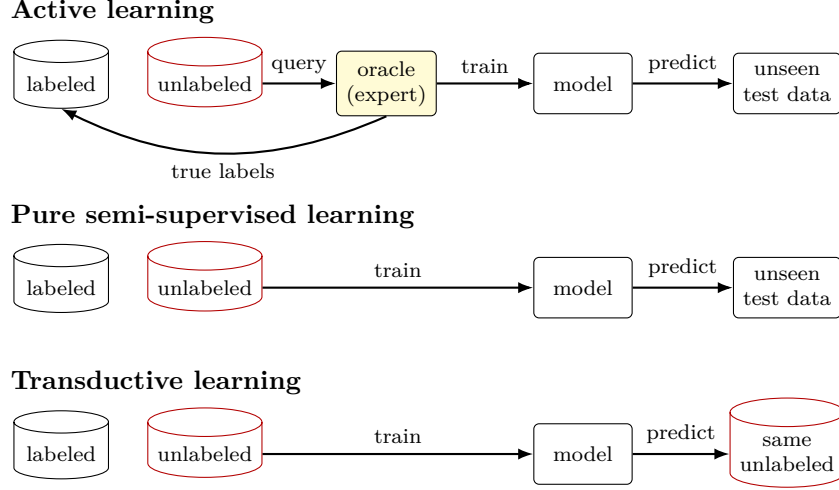


Figure 46: Three ways to use unlabeled data. Active learning asks an oracle for labels of selected examples. Pure semi-supervised learning uses the unlabeled data without a human and predicts on unseen test data. Transductive learning predicts exactly on the unlabeled data it was trained with.

and *transduction* skips the function and outputs labels directly,

$$\{(x_i, y_i) : i = 1, \dots, n\} \cup \{x'_1, \dots, x'_m\} \mapsto (y'_1, \dots, y'_m). \quad (123)$$

The output of transduction is a vector of labels, not a function. The idea, due to Vapnik, is that one should not solve a harder problem (find f for every possible input) as an intermediate step when only a simpler one (m particular labels) is asked. Information is transferred directly from the labeled to the unlabeled examples. The price is that a new test image requires a new transductive run. Which setting to choose depends on whether unlabeled data are available, whether an interpretable function f is needed, and whether one can afford to retrain for every new batch of test data.

Why unlabeled data help: the cluster assumption. An unlabeled example carries no label, so it is not obvious why it helps. It does carry information about *where the data lie*. Figure 47 gives the standard example. With one labeled point of each class, a test point halfway between them cannot be decided. After observing unlabeled points, it becomes clear that the test point lies in the same dense cluster as the positive example. The *cluster assumption* states that points in the same cluster tend to share a label, or equivalently that the decision boundary should pass through low-density regions. Semi-supervised methods rely on this or a related assumption. If it fails, for example because the classes overlap in one dense blob, unlabeled data do not help and can even hurt.

A mixture model with unlabeled data. The cluster assumption can be made precise with a mixture model. Assume the data come from a mixture of components $g = 1, \dots, K$ with weights α_j and densities $f(x | \theta_j)$, for example Gaussians. A binary classifier with classes $c \in \{Y, N\}$ decides via the component that probably generated x :

$$h(x) = \arg \max_{c \in \{Y, N\}} \sum_{j=1}^K P(y = c | g = j, x) P(g = j | x), \quad (124)$$

$$P(g = j | x) = \frac{\alpha_j f(x | \theta_j)}{\sum_{k=1}^K \alpha_k f(x | \theta_k)}.$$

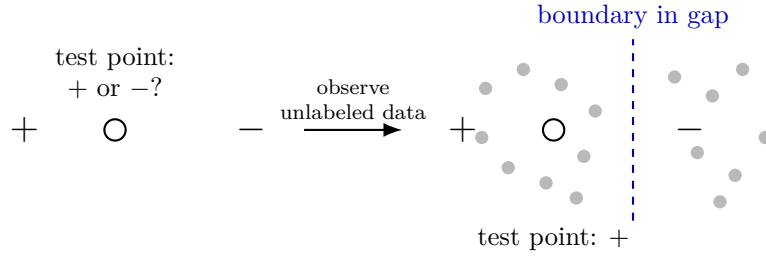


Figure 47: The cluster assumption. With two labeled points alone, the test point (circle) is ambiguous. Unlabeled points (gray) reveal two clusters; the boundary should run through the empty gap, so the test point belongs to the positive cluster.

The second factor, the posterior of component j , depends only on x . It can therefore be estimated from *all* examples, labeled or not, e.g. with the EM algorithm of Appendix D. Only the first factor, which component corresponds to which class, requires labels. If the clusters are well separated, a handful of labeled examples per component suffices to fix this assignment, while the shape and position of the components come from the unlabeled majority.

Worked example. Two Gaussian components have been fitted with EM on 10 000 images, of which only 20 are labeled. Of the labeled images that fall into component 1 (highest $P(g = 1 | x)$), 9 are Y and 1 is N; of those in component 2, 1 is Y and 9 are N. Then $P(y = Y | g = 1) \approx 0.9$ and $P(y = Y | g = 2) \approx 0.1$. A new image with $P(g = 1 | x) = 0.7$ gets

$$P(Y | x) \approx 0.9 \cdot 0.7 + 0.1 \cdot 0.3 = 0.66, \quad P(N | x) \approx 0.1 \cdot 0.7 + 0.9 \cdot 0.3 = 0.34,$$

so $h(x) = Y$. Twenty labels could not have estimated two high-dimensional Gaussians, but they suffice to name the components.

Measuring label efficiency. A frequent practical question is how many labels a representation needs. A *learning curve* answers it: train the same classifier, e.g. a linear SVM, on stratified fractions of the labeled set (1 %, 5 %, 10 %, 25 %, 50 %, 100 %) and plot test accuracy against the fraction. Stratification keeps the class proportions equal in every fraction, so small fractions are not missing whole classes, and the SVM parameter C should be re-chosen by cross-validation at each fraction because the best regularization depends on the amount of data. Comparing a handcrafted feature such as HOG with frozen self-supervised features such as DINO (Appendix G) on the same curve shows directly how much the unlabeled pretraining saves: a good representation reaches a given accuracy with a much smaller fraction. This is a form of exploiting unlabeled data too, only the unlabeled data were used before, during pretraining.

10.3 Inexact Supervision: Multi-Instance Learning

In inexact supervision some label information is present, but not at the level the task needs. The best-known case is *multi-instance learning* (MIL). The training data are organized into *bags*, i.e. sets of instances, and only the bag carries a label. In the binary case the rule is: a bag is positive if at least one of its instances is positive, and negative if all of its instances are negative.

Images fit this pattern naturally, as Figure 48 shows. An image is a bag; its patches are the instances. The label “tiger” says that some patches show the tiger, but not which ones; most patches show grass, trees, or background. An image without a tiger guarantees that none of its patches show a tiger. Formally, with bag $B = \{x_1, \dots, x_K\}$, hidden instance labels $y_k \in \{0, 1\}$, and bag label Y ,

$$Y = \max_{k=1, \dots, K} y_k. \quad (125)$$

The goal is to predict labels of unseen bags, and often, as a by-product, to find the positive instances inside them.

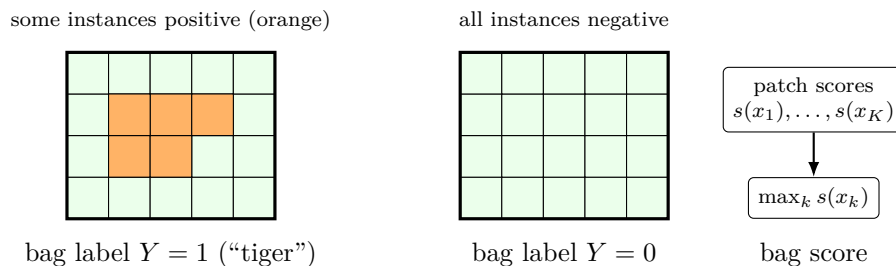


Figure 48: Multi-instance learning on images. An image is a bag of patches. A positive image contains at least one positive patch, whose position is unknown; a negative image contains none. A network can score every patch and pool the scores with a maximum to obtain the bag score.

The asymmetry of Equation 125 matters for learning. Every instance of a negative bag is a certain negative example, so negative bags provide clean instance-level labels. A positive bag says only that the maximum is positive, so it constrains just its most positive instance. A common neural implementation scores each patch with a shared network $s(x_k)$, pools the scores with a maximum (or a smooth version such as log-sum-exp or attention pooling), and trains the pooled bag score with the binary cross-entropy on Y . The gradient then flows mainly into the patch that currently has the highest score. After training, the patch scores themselves act as a coarse localization, although the image label never said where the object was.

Worked example. A bag of four patches receives scores 0.1, 0.8, 0.3, 0.2 after the sigmoid. With max pooling the bag probability is 0.8. If the bag label is $Y = 1$, the loss is $-\log 0.8 \approx 0.22$ and only the second patch receives a gradient, pushing its score further up. If the bag label is $Y = 0$, the loss is $-\log(1 - 0.8) \approx 1.61$, again concentrated on the second patch, now pushing it down. Repeated over many bags, patches that appear only in positive images are pushed up and patches that also appear in negative images are pushed down.

10.4 Inaccurate Supervision and Memorization

In inaccurate supervision every training example has a label, but some labels are wrong. The learner does not know which ones. Two noise models are common in experiments. With *symmetric noise*, a fraction of the labels is replaced by a class drawn uniformly at random. In the *symmetric-inclusive* variant the random draw includes the true class, so with 30 % noise and $C = 10$ classes, $30\% \cdot \frac{9}{10} = 27\%$ of the labels are actually wrong. With *asymmetric noise*, labels are flipped to specific similar classes, e.g. truck to automobile or cat to dog, which mimics real annotator confusions.

Memorization. A deep network has enough capacity to fit any labeling of its training set, even random labels. With noisy labels, perfect training accuracy therefore means that the network has also learned the wrong labels. This is called *memorization*. Figure 49 shows how it unfolds with the ordinary cross-entropy loss. For clean examples, the fraction predicted correctly rises and stays high. For the wrongly labeled examples, the prediction is compared with their *true* class. Surprisingly, in the first epochs the network predicts the true class for most of them, although it was never told that class: it has learned the general pattern of each class from the clean majority and applies it to the mislabeled examples too. Later, the fraction predicted as the *wrong training label* rises, and at the end almost all wrong labels are memorized. With early-learning regularization (next subsection), the curve for the wrong labels stays as it was after early learning.

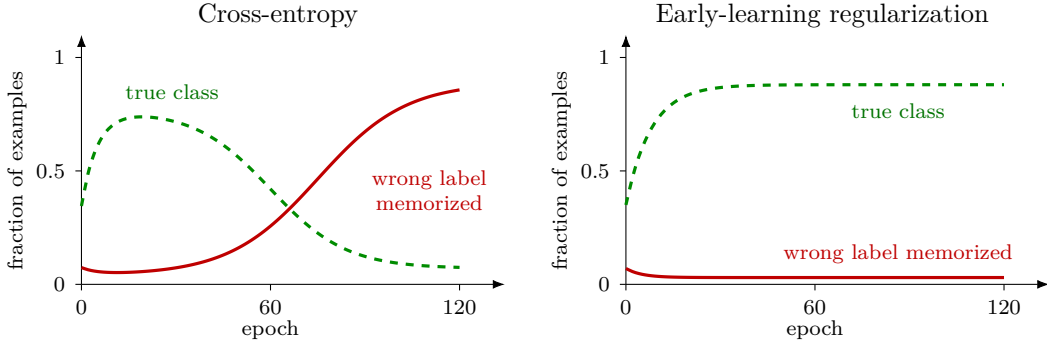


Figure 49: Schematic training curves for the wrongly labeled examples of a noisy dataset. With cross-entropy, the network first predicts their true class (early learning) and later reproduces their wrong training label (memorization). Early-learning regularization keeps the early behavior. For the clean examples, the fraction predicted correctly rises in both cases.

Why memorization comes late. The cross-entropy for n examples with one-hot training labels $\mathbf{y}^{[i]}$ and softmax outputs $\mathbf{p}^{[i]}$ is

$$\mathcal{L}_{\text{CE}}(\Theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_c^{[i]} \log p_c^{[i]}. \quad (126)$$

Its gradient with respect to the logit $z_c^{[i]}$ of example i is the simple residual

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_c^{[i]}} = \frac{1}{n} (p_c^{[i]} - y_c^{[i]}). \quad (127)$$

Early in training, all examples have large residuals. The clean examples form the majority and agree with each other, so their gradients add up and dominate; the network learns the true class structure. Once the clean examples are fitted, $\mathbf{p}^{[i]} \approx \mathbf{y}^{[i]}$ and their gradients vanish. The wrongly labeled examples are still predicted as their true class, which disagrees with their training label, so their residuals stay close to 1. From this point on, they dominate the total gradient and the network gradually bends its decision boundary around them.

The simulated experiment. This mechanism was shown analytically for a simple case. The inputs come from two Gaussians with opposite means,

$$x \sim \mathcal{N}(\mathbf{v}, \sigma^2 I_{p \times p}) \text{ if } \mathbf{y}^* = (1, 0), \quad x \sim \mathcal{N}(-\mathbf{v}, \sigma^2 I_{p \times p}) \text{ if } \mathbf{y}^* = (0, 1), \quad (128)$$

where $\mathbf{v}$ is a unit vector and $\mathbf{y}^*$ the true label. The optimal separator is the hyperplane through the origin perpendicular to $\mathbf{v}$. Each training label equals the true label with probability $1 - \Delta$

and is replaced by the other label with probability Δ . A linear classifier trained by gradient descent on the cross-entropy then behaves as follows (informal theorem): for any noise rate $\Delta \in (0, 1)$, under further conditions on dimension and sample size, there is an iteration T such that

1. *early learning succeeds*: for $t < T$, the classifier predicts the *true* labels of the wrongly labeled examples better than at initialization;
2. the gradients of the *correctly* labeled examples vanish between $t = 0$ and $t = T$;
3. *memorization* occurs as $t \rightarrow \infty$.

The network output in the early-learning phase therefore contains useful information about which labels are wrong. The methods below exploit it. Early stopping would be the naive remedy, but the right stopping time is unknown without clean validation data, and the model is still immature at that point.

10.5 Early-Learning Regularization

Early-learning regularization (ELR) keeps the network close to what it predicted during early learning. For each training example it maintains a *target* vector $\mathbf{t}^{[i]}$ computed from the network's own past outputs, typically as a running average over epochs,

$$\mathbf{t}^{[i]} \leftarrow \beta \mathbf{t}^{[i]} + (1 - \beta) \mathbf{p}^{[i]}, \quad \beta \in [0, 1), \quad (129)$$

and adds a term that rewards agreement between the current prediction and the target:

$$\mathcal{L}_{\text{ELR}}(\Theta) = \mathcal{L}_{\text{CE}}(\Theta) + \frac{\lambda}{n} \sum_{i=1}^n \log(1 - \langle \mathbf{p}^{[i]}, \mathbf{t}^{[i]} \rangle). \quad (130)$$

The dot product $\langle \mathbf{p}, \mathbf{t} \rangle = \sum_c p_c t_c$ measures the similarity of two probability vectors; it is large when both put their mass on the same class. Maximizing it makes $\log(1 - \langle \mathbf{p}, \mathbf{t} \rangle)$ very negative, which lowers the loss. Because the target is computed from past outputs rather than from the noisy label, the regularizer pulls every example towards the class the network believed in early, which for mislabeled examples is mostly the true class. How to estimate the target best is still a research question; the running average is a simple and effective choice.

Effect on the gradient. Differentiating the regularizer of one example with respect to its logit z_c (softmax derivative $\partial p_k / \partial z_c = p_k(\delta_{kc} - p_c)$) gives

$$\frac{\partial}{\partial z_c} \log(1 - \langle \mathbf{p}, \mathbf{t} \rangle) = -\frac{p_c(t_c - \langle \mathbf{p}, \mathbf{t} \rangle)}{1 - \langle \mathbf{p}, \mathbf{t} \rangle}. \quad (131)$$

Let c^* be the true class and assume the target puts most weight on it, so $t_{c^*} > \langle \mathbf{p}, \mathbf{t} \rangle$. Then the ELR gradient for c^* is negative, and gradient descent increases z_{c^*} . Compare this with the cross-entropy gradient $p_{c^*} - y_{c^*}$ of Equation 127:

- For a *clean* example, $y_{c^*} = 1$ and the CE gradient $p_{c^*} - 1$ is negative early and approaches 0 once the example is fitted. The ELR term adds another negative contribution, so both terms agree.
- For a *wrongly labeled* example, $y_{c^*} = 0$ and the CE gradient p_{c^*} is positive; it pushes the true class *down*, which is memorization. The ELR gradient has the opposite sign and roughly cancels it, so the wrong label cannot take over.

In the early phase, the CE gradients of the clean labels dominate and drive learning. Later, when they vanish, ELR neutralizes the wrong labels instead of letting them dominate. The clean labels thus stay in charge throughout training.

Worked example. A mislabeled image has true class c^* , current output $p_{c^*} = 0.6$, and target $\mathbf{t}$ with $t_{c^*} = 0.9$; assume $\langle \mathbf{p}, \mathbf{t} \rangle = 0.56$. The CE gradient for z_{c^*} is $+0.6$ (push down). The ELR gradient is $-0.6 \cdot (0.9 - 0.56)/(1 - 0.56) \approx -0.46$ per unit of λ . With $\lambda = 3$, the regularizer contributes ≈ -1.39 , and the sum is negative: the true class is strengthened despite the wrong label. Note the factor $1/(1 - \langle \mathbf{p}, \mathbf{t} \rangle)$: the closer the prediction already is to the target, the stronger the regularizer holds it there.

10.6 Negative Learning for Noisy Labels

The standard training signal, called *positive learning* (PL) in this context, tells the network “this input belongs to this label.” If the label is wrong, the statement is wrong, and the network learns a wrong decision boundary. *Negative learning* (NL) uses a weaker but much safer statement: “this input does *not* belong to this other label.” Take an image of a dog labeled “car” by mistake. PL trains “this is a car,” which is false. NL picks a random class other than the given label, say “bird,” and trains “this is not a bird,” which is true.

The two losses for C classes are

$$\text{PL: } \mathcal{L}(f, y) = - \sum_{k=1}^C y_k \log p_k, \quad (132)$$

$$\text{NL: } \mathcal{L}(f, \bar{y}) = - \sum_{k=1}^C \bar{y}_k \log(1 - p_k), \quad (133)$$

where $\mathbf{y}$ is the one-hot given label and $\bar{\mathbf{y}}$ the one-hot *complementary label*. PL pushes p_y towards 1; NL pushes $p_{\bar{y}}$ towards 0. The complementary label is drawn anew in every iteration, uniformly from all classes except the given one:

$$\bar{y} \sim \mathcal{U}(\{1, \dots, C\} \setminus \{y\}). \quad (134)$$

Why NL is robust. A complementary label is wrong only if it hits the true class. For a clean example this is impossible, because the true class is the excluded label. For a mislabeled example, the true class is one of the $C - 1$ candidates, so the chance is $1/(C - 1)$. On CIFAR-10 ($C = 10$) with 27 % wrong labels:

$$P(\text{PL statement wrong}) = 0.27, \quad P(\text{NL statement wrong}) = 0.27 \cdot \frac{1}{9} = 0.03.$$

NL thus reduces the rate of misleading training signals by a factor $C - 1$. The price is less information per step: “not a bird” excludes one of ten classes, while “a car” fixes the class. NL converges more slowly and on its own reaches a lower final accuracy than PL on clean data.

Observed behavior. With 30 % symmetric-inclusive noise on CIFAR-10, a CNN trained with PL reaches almost 100 % accuracy on the noisy training labels, while its accuracy on the clean test set peaks early and then drops as it memorizes; the test loss rises again. The same CNN trained with NL fits the noisy training labels much less, but its test accuracy keeps increasing and ends clearly above PL. The difference is visible in the confidence p_y that each network assigns to the *given* label of each training image (Figure 50). After PL, both clean and wrongly labeled images have p_y near 1: the wrong labels are treated as correct. After NL, the wrongly labeled images have p_y near 0, while the clean ones spread over higher values. The given label is no longer believed where it is wrong, so p_y becomes a noise detector.

Selective NL and selective PL. The separation after NL is good but not perfect, and plain NL learns slowly. Two selective stages refine it, together called SelNLPL:

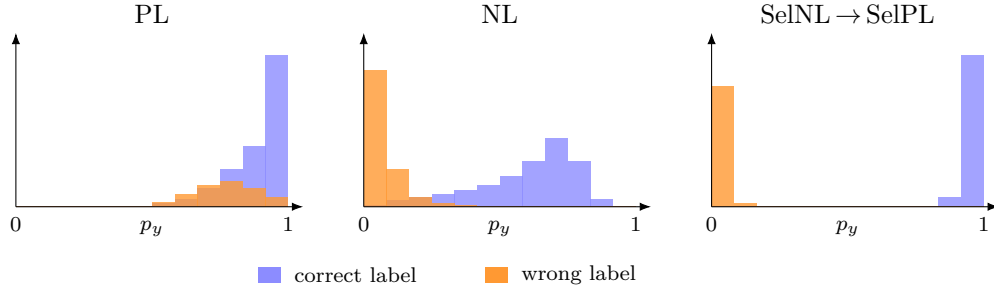


Figure 50: Schematic histograms of the confidence p_y in the given label for all training images (CIFAR-10, 30 % symmetric-inclusive noise). After PL, wrong labels look as confident as correct ones. After NL, they collect near 0. Selective NL and PL sharpen the separation until clean and noisy data are almost perfectly split.

- *Selective negative learning* (SelNL) continues NL, but only on examples whose confidence in the given label exceeds $1/C$, i.e. is higher than uniform guessing. Examples with lower confidence are likely mislabeled and are excluded, so they no longer disturb training. After SelNL, the clean examples move towards $p_y \approx 1$.
- *Selective positive learning* (SelPL) then switches to PL, the faster and more informative loss, but only for examples with $p_y > \gamma$, with $\gamma = 0.5$ in practice. These are almost exclusively clean examples, so PL can now be used without learning wrong labels. After SelPL, nearly all clean data have confidence close to 1, and the noisy data stay near 0.

Table 40 lists the three stages. Each runs for a number of epochs T on the same network $f(x; \theta)$.

Stage	Examples used in a batch	Loss	Purpose
1. NL	all	NL, Eq. 133	robust start; separate clean from noisy
2. SelNL	only if $p_y > 1/C$	NL, Eq. 133	ignore likely noisy examples
3. SelPL	only if $p_y > \gamma$ ($\gamma = 0.5$)	PL, Eq. 132	fast, precise learning on likely clean data

Table 40: The SelNLPL procedure. The selection threshold rises from stage to stage, and the loss switches from the safe negative loss to the informative positive loss once the data are filtered.

Pseudo-labeling the noisy part. After SelNLPL, the training data are split into a *clean* set (high p_y) and a *noisy* set (low p_y). The noisy images are still useful images, only their labels are not. Treating them as unlabeled data turns the problem into semi-supervised learning, and *pseudo-labeling* closes the loop (Figure 51):

1. Divide the training data into clean and noisy data using the CNN trained with SelNLPL.
2. Train a freshly initialized CNN on the clean data only.
3. Relabel the noisy data with the *soft* output of this CNN, i.e. the full probability vector instead of a one-hot label, which keeps the network’s uncertainty.
4. Train another freshly initialized CNN on the clean data plus the relabeled data.

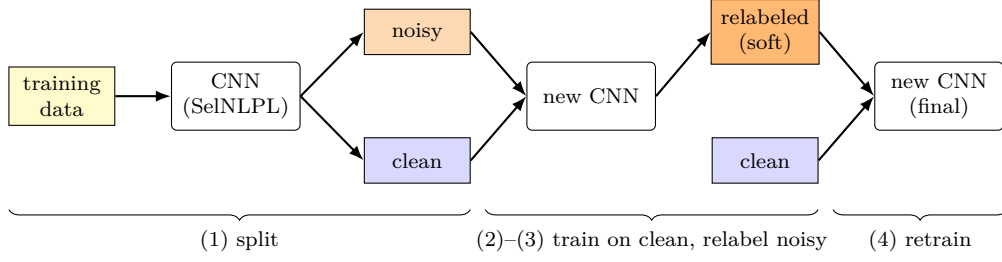


Figure 51: Pseudo-labeling after SelNLPL. The noisy labels are discarded and replaced by soft labels predicted by a network trained on the clean part; the final network learns from all images.

Results. Table 41 shows test accuracies on CIFAR-10 and CIFAR-100 (ResNet-34). Plain cross-entropy degrades quickly as symmetric noise grows; the full method stays far above it, e.g. 88.3% instead of 74.1% on CIFAR-10 with 60% noise. With asymmetric noise on CIFAR-100, the *forward correction* method (Forward T) is slightly better. That method, however, uses the noise transition matrix T , i.e. knowledge of which class is mainly confused with which other class. This is information the negative learning approach does not need.

Dataset	Method	symmetric noise			asymmetric noise		
		20 %	40 %	60 %	10 %	20 %	40 %
CIFAR-10	CE	86.98	81.88	74.14	90.69	88.59	80.11
	Forward T	88.63	85.07	79.12	91.32	90.35	88.12
	SelNLPL + pseudo-labels	94.23	92.43	88.32	94.57	93.35	89.86
CIFAR-100	CE	58.72	48.20	37.41	66.54	59.20	42.74
	Forward T	63.16	54.65	44.62	71.05	71.08	70.82
	SelNLPL + pseudo-labels	71.52	66.39	56.51	70.35	63.12	45.70

Table 41: Test accuracy (%) under label noise, excerpt from the NLNL paper. Negative learning wins clearly under symmetric noise. Under strong asymmetric noise on CIFAR-100, forward correction wins, but it assumes a known confusion matrix.

10.7 How Much Does Weak Supervision Really Gain?

Weakly supervised learning is an active research area, and a critical look at published results shows that the gains are often smaller than claimed. Many WSL methods silently rely on some *clean* data, for example a clean validation set to choose hyperparameters or the stopping epoch. If these clean samples are available anyway, a natural baseline is to train a standard model, or fine-tune a pretrained one, directly on them. Such experiments showed that a few clean labels per class often match or beat WSL methods trained on thousands of weakly labeled examples. Three recommendations follow for evaluating any WSL approach:

- Report how much the method relies on clean data, including clean validation data for model selection.
- Report how many cleanly annotated samples a few-shot approach needs to reach the same performance. If thousands of weakly labeled samples are worth only a handful of clean ones, WSL may not be the right choice.
- If the method needs extra clean data, include standard non-WSL methods trained on that clean data in the comparison.

Table 42 collects the methods of this section with their key idea and assumption. The assumption column is the practical checklist: a method only helps if its assumption holds for the data at hand.

Method	Key idea	Assumption / requirement
Active learning	ask an oracle for the most useful labels	an expert is available; query cost matters
Semi-supervised	unlabeled data shape the clusters; labels name them	cluster assumption holds
Transductive	label the given unlabeled set directly	test data known in advance
Multi-instance	bag positive iff one instance positive; max pooling	bag-level labels; negative bags fully negative
ELR	pull predictions towards early-training targets	early learning finds the true classes
Negative learning	train “not this class” with random complementary labels	noise rate moderate; C not too small
SelNLPL	filter by confidence, then PL on clean data and relabel the rest	NL confidence separates clean from noisy

Table 42: Summary of weakly supervised methods. Each exploits a specific structure in weak labels and fails when its assumption is violated.

11 Multimodal Language Models and LLM Applications

Large language models (LLMs) show strong reasoning abilities in dialogue, step-by-step argumentation, math problems, and code writing. They learn all of this from text alone. A robot that should “bring me the rice chips from the drawer” needs more: it must know where the drawer is, whether it is open, and which object in the camera image is the bag of chips. This is the *grounding problem*. LLM representations must be connected to what the system perceives (images, sensor states), to what it can do (actions in the physical world), and to facts about the

concrete scene, such as its geometric configuration. Text-only training never supplies this link between words and percepts.

This section treats two answers to the grounding problem. The first is architectural: PaLM-E injects continuous sensor observations directly into the embedding space of a pretrained LLM and trains the combination end-to-end, so that one model plans robot actions, answers visual questions, and writes captions. The second is practical: an application program calls an existing vision-capable LLM through an API and wraps it in a small workflow graph that decides what to do with each image. Both build on the decoder-only transformer of Section 7 and on the Vision Transformer (ViT) of Equation 97.

11.1 Decoder-Only Language Models and Prompts

A decoder-only LLM is a generative model of text. Text is represented as a token sequence $w_{1:L} = (w_1, \dots, w_L)$, and the model assigns it a probability by the autoregressive factorization of Equation 95,

$$p(w_{1:L}) = \prod_{l=1}^L p_{\text{LM}}(w_l \mid w_{1:l-1}). \quad (135)$$

Here p_{LM} is the transformer with causal masking: the output at position l is a softmax over the vocabulary and predicts the next token. Before the first layer, every token passes through the embedding table, written here as a function $\gamma : \mathcal{W} \rightarrow \mathbb{R}^k$ that maps a vocabulary entry w to a vector $x = \gamma(w)$ in a k -dimensional embedding space.

A *prefix-decoder* LLM splits the sequence into a given prefix $w_{1:n}$ and a continuation that the model predicts:

$$p(w_{n+1:L} \mid w_{1:n}) = \prod_{l=n+1}^L p_{\text{LM}}(w_l \mid w_{1:l-1}). \quad (136)$$

The prefix, often called the *prompt*, provides the context. It may describe the task (“Q: How can the yellow block be moved onto the blue plate? A:”), contain earlier dialogue, or show a few solved examples. The LLM continues the prefix, and that continuation is the answer. Three prompt regimes follow from this view and appear repeatedly below:

- *Zero-shot*: the prefix contains only the task, no solved examples. The model must rely on what it learned in pretraining.
- *Few-shot* (in-context learning): the prefix contains a few input–output examples, and the model continues the pattern. The weights do not change; only the input does.
- *Chain of thought* (CoT): the prefix asks for, or demonstrates, intermediate reasoning steps before the final answer (“Let’s think step by step”). Spelling out steps turns one hard prediction into several easier next-token predictions, each conditioned on the previous steps.

All three change behavior at inference time without training. This makes a pretrained LLM a general interface: whoever can express a task as a prefix can use it. The rest of the section asks what happens when parts of that prefix are not words but images and sensor readings.

11.2 PaLM-E: Injecting Observations into the Language Embedding Space

An *embodied language model* incorporates continuous inputs from the real-world sensor and actuator modalities of an agent, such as a robot, into a language model. PaLM-E (“Pathways Language Model, Embodied”) builds it from PaLM, a large pretrained decoder-only LLM. The central idea is simple. The LLM already processes a sequence of k -dimensional vectors. If an image or a state estimate is encoded into vectors of the same dimension k , it can be placed into

the same sequence as the word embeddings and processed by the self-attention layers *in the same way as text*. Nothing in the transformer needs to change.

The input is a *multimodal sentence*: text with interleaved placeholders for observations, for example

Q: What happened between <img_1> and <img_2>?

Formally, an encoder ϕ_j maps observation O_j (an image, a state vector, a 3D scene) to a sequence of q_j vectors, $\phi_j(O_j) \in \mathbb{R}^{q_j \times k}$. The input vector at position i is then

$$x_i = \begin{cases} \gamma(w_i) & \text{if position } i \text{ is a text token,} \\ \phi_j(O_j)_i & \text{if position } i \text{ belongs to observation } O_j. \end{cases} \quad (137)$$

The placeholder <img_1> is thus replaced by q_1 vectors, not by one token. Observation vectors can sit anywhere between the text tokens, so the text can refer to an image before or after it and several images can be compared. Figure 52 shows the complete model.

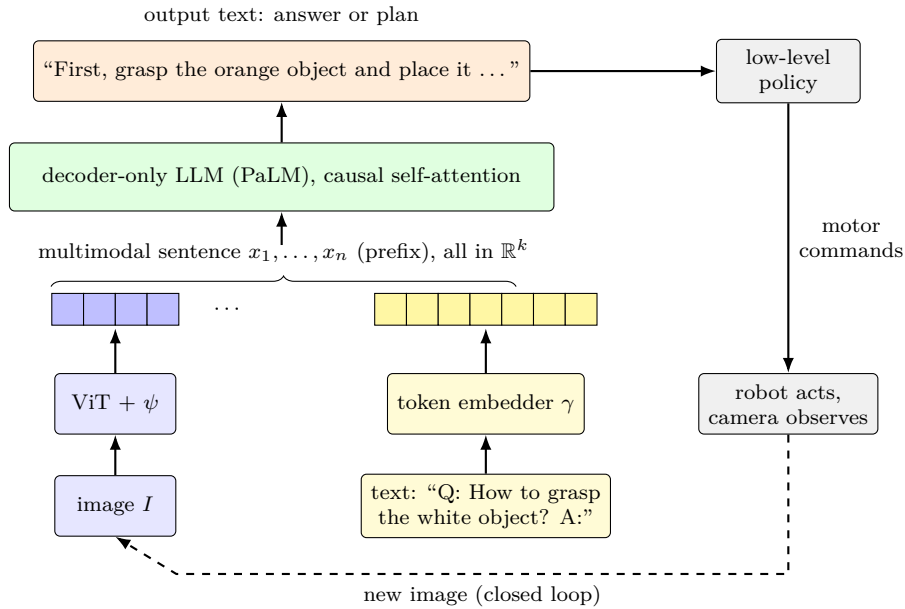


Figure 52: PaLM-E. An encoder turns each observation into a few vectors of the LLM’s embedding dimension k . They are interleaved with the word embeddings to a multimodal prefix, and the pretrained LLM continues it with text. For robot tasks, the text is a plan step that a low-level policy executes; the next camera image starts the next planning step.

The output is always text. For visual question answering (VQA) and captioning, the text is the answer or description. For robotics, the text is a sequence of high-level decisions (“pick the green block,” “push it to the corner”). PaLM-E does not output joint angles itself. A separate *low-level policy*, a smaller learned controller that understands such short language commands, translates each step into motor commands. After the step, the robot takes a new image, and PaLM-E is queried again with it. This *closed-loop* planning lets the model react to failures and disturbances: if a person moves the chips back into the drawer, the next image shows it, and the plan adapts.

Worked example: sequence length. An image of 224×224 pixels encoded by a ViT with patch size $P = 14$ gives $(224/14)^2 = 16^2 = 256$ patch features (Equation 97). A learned projection ψ maps each feature from the ViT width to the LLM width k . The image therefore occupies $q = 256$ positions of the prefix. The question “What happened between <img_1> and <img_2>?” with about 10 text tokens becomes a prefix of roughly $2 \cdot 256 + 10 = 522$ vectors.

Images, not words, dominate the sequence length and thus the quadratic attention cost. This is one reason to prefer compact encoders, such as the object-centric ones below, that describe a scene with few vectors.

11.3 Encoders for Different Sensor Modalities

Equation 137 leaves the encoder ϕ open. Its job is to map an observation into vectors the LLM can use, and different modalities call for different encoders. PaLM-E compares several (Table 43).

- *State estimation vectors.* If the robot or a perception system already provides a state $s \in \mathbb{R}^S$, e.g. the poses of all objects, a multilayer perceptron maps it into the embedding space, $\phi_{\text{state}}(s) \in \mathbb{R}^k$. This is the simplest input but requires a working state estimator and fixes which quantities are known.
- *Vision Transformer.* A pretrained ViT $\tilde{\phi}_{\text{ViT}}$ maps the image I to m token features. Its feature width generally differs from k , so a learned affine map ψ projects each feature: $\phi_{\text{ViT}}(I) = \psi(\tilde{\phi}_{\text{ViT}}(I)) \in \mathbb{R}^{m \times k}$. This works on raw pixels and needs no state estimator. Its tokens, however, describe image patches, not objects.
- *Object-centric representations.* Planning statements are about objects (“put the red block on the plate”). Object-centric encoders separate the visual input into distinct objects before injecting it into the LLM. Conceptually this requires instance masks M_j : the object token is $\phi_{\text{ViT}}(I \circ M_j)$, the ViT applied to the masked image. Ground-truth masks are available in simulation but not in the real world.
- *Object Scene Representation Transformer (OSRT).* OSRT learns a neural 3D scene representation from several views without supervision. It discovers objects by itself and represents the scene by a set of *object slots* o_j . Each slot is projected by an MLP to one or more tokens. This gives object-centric, 3D-aware inputs without ground-truth masks.

Object-centric encoders enable *entity referrals*. The prefix can name each object with its own tokens, e.g. “Object 1 is <obj_1>. Object 2 is <obj_2>. ...” The model can then answer “Push obj_2 to the left” and the low-level policy knows exactly which object is meant. A plain ViT encoding offers no such handle; the plan must describe objects by color or position, which is ambiguous when several objects look alike.

Encoder	Input	Tokens produced	Main strength / limitation
State MLP	state vector s	one vector per state	compact and exact; needs a state estimator
ViT + ψ	raw image	one per patch (e.g. 256)	general, pretrained at scale; not object-based
Object-centric ViT	image and masks M_j	tokens per object	supports entity referrals; needs masks
OSRT	multi-view images	tokens per object slot	3D-aware, objects learned without masks; most data-efficient

Table 43: Input encoders compared in PaLM-E. All map an observation to vectors of the LLM embedding dimension k ; they differ in what structure they impose on the scene.

11.4 Training PaLM-E and Its Design Choices

PaLM-E is trained end-to-end on a dataset of multimodal examples. Example i consists of u_i continuous observations $O_{1:u_i}^i$, a token sequence $w_{1:L_i}^i$ with placeholders for them, and an index n_i that marks where the prefix ends. The loss is the ordinary next-token cross-entropy, but only on the continuation:

$$\mathcal{L} = - \sum_i \sum_{l=n_i+1}^{L_i} \log p_{\text{LLM}}(w_l^i | x_{1:l-1}^i), \quad (138)$$

where x^i is the multimodal sentence of Equation 137. The prefix, containing the images and the question, is only conditioned on; the model is never trained to predict it. Observation vectors are not tokens from the vocabulary, so there is no target to predict at those positions anyway. Gradients flow through the LLM into the projection ψ and the encoder, which learn to produce vectors that the LLM can “read.”

Worked example: which tokens are trained. Take the training example “ Q: What color is the cube? A: red.” with 256 image vectors, 8 question tokens, and 2 answer tokens (“red,” “.”). Then $n_i = 264$, and the loss sums over just the two answer positions. If the model predicts $p(\text{red} | \text{prefix}) = 0.5$ and $p(. | \text{prefix}, \text{red}) = 0.9$, the example contributes $-\log 0.5 - \log 0.9 \approx 0.69 + 0.11 = 0.80$. The 264 prefix positions still matter: every answer prediction attends to them, so their encoders receive gradients through attention.

Frozen or fine-tuned LLM. Two training regimes are possible:

- *Full fine-tuning*: encoder, projection, and LLM weights are all updated. The model can adapt its language layers to the new inputs but may lose some of its original abilities.
- *Frozen LLM*: only the encoder ϕ and projection ψ are trained. The encoder must learn to translate observations into vectors that the fixed LLM already understands. Because these vectors act like a learned, image-dependent prompt, this is called *input-conditioned soft prompting*. It keeps the language abilities intact by construction.

Encoders can themselves be pretrained (a large ViT, a pretrained OSRT) or trained from scratch. The data question is *sample complexity*: robot data are expensive to collect, often only a few hundred demonstrations per task, while images with text are abundant on the web.

Multi-task training and transfer. PaLM-E is trained on a *mixture* of tasks: robot planning in several environments (tabletop task and motion planning, the Language-Table pushing environment, mobile manipulation in a kitchen), general vision-language data (VQA, captioning), and pure language tasks. A key finding is *positive transfer*: the model trained on the full mixture solves each robot task better than a model trained on that task alone. Knowledge from web-scale vision-language data carries over to robotics, so a robot task needs considerably fewer of its own examples. Transfer thus yields high data efficiency for robotics, and the same model supports zero-shot multimodal CoT, few-shot prompting, OCR-free math reasoning from images, and multi-image reasoning.

Scaling and catastrophic forgetting. PaLM-E was built at several sizes by combining a PaLM LLM with a ViT, e.g. an 8B-parameter LLM with a 4B ViT (PaLM-E-12B) up to a 540B LLM with a 22B ViT (PaLM-E-562B). When the LLM is fine-tuned on multimodal data, it can *catastrophically forget* its pure-language abilities: new gradients overwrite what the old weights encoded. PaLM-E observed that this forgetting shrinks strongly with scale. The smallest model lost most of its language-generation performance, the largest only a few percent. Larger models have enough capacity to add the new modality without giving up the old skills. Freezing the LLM avoids forgetting entirely, at the price of less adaptation.

Emergent capabilities. Some abilities appear in the largest model without having been trained explicitly. With *zero-shot multimodal chain of thought*, the model, given only the prompt, spells out a step-by-step reasoning narrative that mixes textual and visual evidence before stating its final answer, e.g. “The image shows two bottles of water and one of juice. . . . So the answer is three.” With *multi-image reasoning*, it compares several images in one prompt, although training examples mostly contained single images. It also shows *zero-shot generalization* in robotics to unseen object pairings or unseen objects. Table 44 collects the research questions and the answers PaLM-E gives.

Question	Finding
Can pixels or states be input to an LLM while keeping its generalization?	yes, encoded into the word embedding space and interleaved with text
Which input representation works best?	object-centric, 3D-aware (OSRT) is most data-efficient; ViT is the general choice for raw images
Frozen or fine-tuned LLM?	frozen LLM with trained encoder already works (soft prompting); fine-tuning adapts more but risks forgetting
Single-task or mixture training?	mixture gives positive transfer and higher data efficiency on robot tasks
Effect of scale?	less catastrophic forgetting of language; emergent multimodal CoT and multi-image reasoning

Table 44: Research questions of PaLM-E and its answers. One generalist model covers robot planning, vision–language, and language tasks.

PaLM-E fits the general notion of a foundation model, a model trained on broad data and adapted to many tasks (Appendix G). Its vision encoder is exactly such a pretrained model, and the LLM is another. PaLM-E shows how two foundation models from different modalities can be joined through a shared embedding space.

11.5 Using an LLM Through an API: A Visual Task Agent

Training a model like PaLM-E requires resources few groups have. In practice, vision-capable LLMs are available as services: an application sends images and text over an HTTP API and receives text back. Such a model can already describe images, read text in them, and answer questions. What remains for the application developer is to decide *what* to ask, and *what to do* with the answer. A program that makes these decisions, possibly calling other tools, is called an *agent*.

Consider a *visual task helper* that receives images of different kinds and must treat them differently:

- a *family image* should get a friendly scene summary;
- a *product image* should trigger a secondary tool, e.g. a search API queried with the detected objects;
- a *document or sign* should go to optical character recognition (OCR) so that its text is extracted verbatim.

The flow is: take an image (a local file or a webcam frame); let a vision model describe it; let the agent decide, based on that description, whether to summarize the scene, extract text, or call a tool; return the result. Figure 53 shows this flow as a graph.

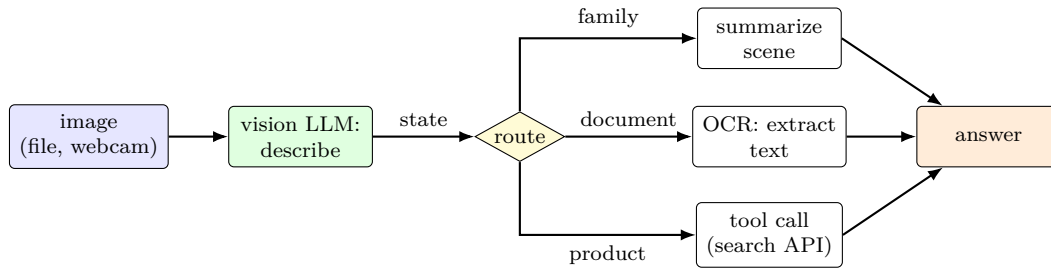


Figure 53: Visual task agent as a workflow graph. Nodes are processing steps, which may call an LLM or a tool. The routing node reads the shared state, here the image description, and selects the branch.

LangChain and LangGraph. Two open-source frameworks support such programs. *LangChain* provides building blocks for LLM-powered applications and agents: uniform wrappers around model providers, message and prompt templates, output parsers, and tool definitions. *LangGraph* orchestrates LLM and agent workflows as a *graph*. Each node is a function that reads and updates a shared *state* (here: the image, its description, the chosen route, the result). Edges fix the order of nodes; *conditional edges* call a routing function on the state and branch accordingly. Loops are allowed, so an agent can also retry or ask a follow-up question. Making the control flow an explicit graph keeps it inspectable and testable, instead of hiding every decision inside one long prompt.

A request to a vision LLM. A chat API receives a list of *messages*, each with a role (**system** for standing instructions, **user** for the request, **assistant** for earlier model replies). A user message can mix text and images; a local image is usually sent base64-encoded. The core of the “describe” node and the graph wiring look like this in Python:

```

from openai import OpenAI
client = OpenAI() # reads OPENAI_API_KEY from the environment

def describe(state):
    b64 = state["b64"] # base64-encoded JPEG
    msg = [{"role": "user", "content": [
        {"type": "text", "text": "Describe the image. Is it a family photo,"
                                " a product, or a document?"},
        {"type": "image_url",
         "image_url": {"url": f"data:image/jpeg;base64,{b64}"}}]]
    r = client.chat.completions.create(model="gpt-4o-mini", messages=msg)
    return {"description": r.choices[0].message.content}

from langgraph.graph import StateGraph, END
g = StateGraph(State) # State: a typed dict
g.add_node("describe", describe)
g.add_node("summarize", summarize); g.add_node("ocr", ocr)
g.add_node("search", search)
g.set_entry_point("describe")
g.add_conditional_edges("describe", route, # route(state) -> key
    {"family": "summarize", "document": "ocr", "product": "search"})
for n in ("summarize", "ocr", "search"): g.add_edge(n, END)
app = g.compile(); result = app.invoke({"b64": image_b64})

```

Here `route` can be a plain keyword test on the description or a second LLM call that answers with one of the three keys. The multimodal message is the practical counterpart of PaLM-E’s

multimodal sentence: the provider’s model turns the image into embedding vectors and places them in the prompt next to the text tokens. The developer only sees text in and text out.

API keys and running the agent. The service authenticates each request with a secret *API key*. It is created on the provider’s developer platform (for billing, a small prepaid balance suffices) and stored in a `.env` file in the project folder, e.g. `OPENAI_API_KEY="sk-..."`. The program loads it into the environment (e.g. with `python-dotenv`); the client library reads it from there. The key must never appear in code or in a public repository: anyone who has it can make requests billed to the owner. Add `.env` to `.gitignore` and, where possible, create the key with restricted permissions. A reproducible Python environment is set up with conda,

```
conda env create -f environment.yml
conda activate vision-agent
python main.py
```

and the agent then runs on local images or the webcam.

Practical limits. The API model is a black box that can change between versions, and its answers are not deterministic. Routing decisions should therefore be constrained, e.g. by asking for exactly one of a fixed set of labels and falling back to a default otherwise. Every call costs money and time, and images consume many input tokens (compare the worked example on sequence length above), so large batches should be sent at reduced resolution when the task allows. Finally, images sent to an external service leave the local machine, which matters for private family photos or confidential documents.

11.6 Trained Integration Versus API Composition

PaLM-E and the visual task agent answer the grounding problem at different levels. PaLM-E changes the model: observations become part of the LLM input, and training teaches the model to reason about them and to plan actions. The agent keeps the model fixed and changes the program around it: a graph decides which question to ask and which tool to call. Table 45 contrasts the two.

Aspect	Embodied multimodal LLM (PaLM-E)	LLM via API (visual agent)
Where images enter	encoder ϕ , vectors in the embedding space	image part of a user message; encoding hidden
What is trained	encoder, projection, optionally the LLM	nothing; behavior set by prompts and graph
Output	text: answers, captions, robot plans	text, plus results of tool calls
Control flow	learned, inside the model (closed-loop replanning)	explicit graph with nodes and conditional edges
Requirements	large compute, multimodal and robot data	API key, budget, network access
Typical use	research, robotics, generalist models	applications built quickly from existing models

Table 45: Two ways to connect language models with images. Trained integration grounds the model itself; API composition grounds an application built around a fixed model.

Both views share the same core mechanism. An image is turned into a set of vectors, those vectors join the text tokens in one sequence, and a decoder-only transformer continues that

sequence with text. Everything else, whether the text is a VQA answer, a robot plan, or a routing label, depends on the prompt and on what the text is used for afterwards. Image *captioning*, one of the basic computer vision tasks, is the simplest instance: the prefix is an image and “Describe the image,” and the continuation is the caption.

A Edge Detection Essentials

Every voting method in Section 3 starts from edge pixels and their gradients. The intuition is simple: an edge is where the brightness changes abruptly, so its location is where the first derivative of the image is large. This appendix collects what is needed to compute gradients and to understand the reference edge detector, Canny.

Gradients from derivative filters. On a pixel grid the derivatives $G_x = \partial I / \partial x$ and $G_y = \partial I / \partial y$ are approximated by small convolution kernels. The Sobel kernels combine a central difference in one direction with a smoothing in the other:

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad S_y = S_x^\top = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}. \quad (139)$$

The Prewitt kernels are the same with the weights 2 replaced by 1. From the two responses follow the gradient magnitude $\|\nabla I\| = \sqrt{G_x^2 + G_y^2}$ (edge strength) and the orientation $\theta = \text{atan2}(G_y, G_x)$. The gradient points from dark to bright and is perpendicular to the edge; the edge itself runs along $\theta \pm 90^\circ$.

The Canny detector. Thresholding the gradient magnitude alone gives thick, broken, noisy edges. Canny fixes this in four steps:

1. smooth the image with a Gaussian to suppress noise;
2. compute gradient magnitude and orientation, e.g. with Sobel kernels;
3. *non-maximum suppression*: keep a pixel only if its magnitude is a local maximum along the gradient direction, which thins every edge to a one-pixel-wide line;
4. *hysteresis thresholding* with a high and a low threshold: pixels above the high threshold are strong edges and are kept; pixels between the thresholds are weak edges and are kept only if they connect to a strong edge; everything below the low threshold is discarded.

Hysteresis removes isolated noise responses while preserving long, coherent boundaries. The detector fails where the image offers no contrast: low-contrast boundaries, haze, and gradual transitions stay below threshold, and strong texture produces fragmented contours. The smoothing width and the two thresholds must be tuned to the imaging conditions.

B Fourier Transform and Fourier Descriptors

The Fourier transform rewrites a signal as a sum of sinusoids of different frequencies; the list of their complex weights is the *spectrum*. Low frequencies describe the coarse shape of a signal, high frequencies its fine detail and noise. Two applications appear in object models: correlating gradient images quickly as an alternative to R-tables, and describing a closed contour by the spectrum of its boundary (Fourier descriptors). Both need only the facts collected here.

B.1 The Discrete Fourier Transform

For a discrete signal $x(0), \dots, x(N-1)$ the discrete Fourier transform (DFT) and its inverse are

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-2\pi i k n / N}, \quad x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{2\pi i k n / N}. \quad (140)$$

The spectrum has as many coefficients as the signal has samples, and it is periodic, $X(k) = X(k+N)$, so the upper half of the indices can equally be read as negative frequencies: $X(N-1) = X(-1)$, $X(N-2) = X(-2)$, and so on. Implementations therefore often store the spectrum as $[X(0), X(1), \dots, X(N/2), X(-N/2), \dots, X(-2), X(-1)]$. The zero-frequency term $X(0) = \sum_n x(n)$ is N times the mean of the signal.

Real signals. If x is real-valued, the spectrum is *Hermitian*: $X(-k) = X(k)^*$, where $*$ is complex conjugation. Half of the spectrum then determines the other half. For example, if a real signal with $N = 17$ samples has $X(0), \dots, X(8) = [10, -5 + i, -0.5 + 0.2i, -0.04 + 0.03i, 0, 0, 0, 0, 0]$, the remaining entries $X(-8), \dots, X(-1)$ are $[0, 0, 0, 0, 0, -0.04 - 0.03i, -0.5 - 0.2i, -5 - i]$: conjugate and mirror, without changing the sign of the real parts.

Standard spectra. Table 46 lists the transform pairs used most often.

Signal	Spectrum
$\cos(2\pi f_0 x)$	two real spikes at $\pm f_0$
$\sin(2\pi f_0 x)$	two imaginary spikes at $\pm f_0$ with opposite signs
rectangle of width w	sinc function, $\propto \text{sinc}(wf)$; a narrower rectangle gives a wider sinc
sinc function	rectangle (ideal low-pass)
Gaussian	Gaussian; a narrow Gaussian gives a wide one

Table 46: Common Fourier pairs. Width in the signal domain and width in the frequency domain are inversely related.

Convolution and correlation theorems. Convolution in the signal domain is multiplication in the frequency domain, and correlation is the same with *one* factor conjugated:

$$f * g = \text{DFT}^{-1}(\text{DFT}(f) \cdot \text{DFT}(g)), \quad f \star g = \text{DFT}^{-1}(\text{DFT}(f)^* \cdot \text{DFT}(g)). \quad (141)$$

Which of the two factors carries the conjugate is a convention; the form $\text{DFT}^{-1}(\text{DFT}(I) \cdot \text{DFT}(O)^*)$, with the template conjugated, is equally a correlation and only mirrors the shift axis. Conjugating both factors, or neither, does not give the correlation: the plain product is the convolution. The practical value is speed: a correlation of a template with a whole image costs two forward DFTs, a pointwise product, and one inverse DFT.

Gradient correlation instead of R-tables. Write the gradients of the image I and of the template O as complex numbers,

$$I_I = \frac{\partial I}{\partial x} + i \frac{\partial I}{\partial y}, \quad O_I = \frac{\partial O}{\partial x} + i \frac{\partial O}{\partial y}. \quad (142)$$

For two complex numbers $a = a_x + ia_y$ and $b = b_x + ib_y$ the product $a^*b = (a_x b_x + a_y b_y) + i(a_x b_y - a_y b_x)$ has the dot product of the two gradient vectors as its real part. Hence $\text{Re}(I_I \star O_I)(x, y)$ sums, over the whole template placed at (x, y) , the dot products between template gradients

and image gradients. It is large where many image edges have the position and orientation the template expects, which is almost the same as Hough voting with an R-table. The imaginary part contains cross products instead and is not the voting score. Equation (141) computes the correlation for all positions at once.

B.2 Fourier Descriptors

A closed contour can be described as a periodic complex signal. Trace the boundary of a segmented object at N points and write each point as one complex number,

$$u(n) = x(n) + i y(n), \quad n = 0, \dots, N-1. \quad (143)$$

The Fourier descriptor is its DFT, $F(\mu) = \text{DFT}(u)(\mu)$ for $\mu = 0, \dots, N-1$. From a greyscale image the pipeline is therefore: segment the object, trace its boundary at N points, form u , apply the DFT, normalize, and keep the low-frequency coefficients. Two counting facts follow directly from the DFT. The descriptor has exactly as many coefficients as the contour has points, so a descriptor $F(\mu)$ with $\mu \in \{0, \dots, 69\}$ describes a contour of 70 points, whatever the value of $F(0)$. And $F(0) = \sum_n u(n) = N \cdot (\bar{x} + i \bar{y})$ is N times the centroid: for a contour of $N = 20$ points centered at $(10, 20)$, $F(0) = 200 + 400i$ (with a $1/N$ -normalized DFT it would be the centroid $10 + 20i$ itself).

How transformations act. Each geometric change of the shape has a simple effect on the spectrum, which is what makes normalization possible (Table 47). A translation by $t = t_x + it_y$ adds t to every $u(n)$ and therefore only changes $F(0)$ (by Nt). A scaling by a multiplies every coefficient by a . A rotation by φ multiplies every coefficient by $e^{i\varphi}$, a pure phase change. A different start point on the contour multiplies $F(\mu)$ by a phase $e^{2\pi i \mu m/N}$ that grows with μ .

Invariance to	Effect on F	Normalization
Translation	changes only $F(0)$	set $\tilde{F}(0) := 0$
Scale	multiplies all $F(\mu)$ by a	divide by $ F(k) $ for a fixed $k \neq 0$, e.g. $k = 1$
Rotation	multiplies all $F(\mu)$ by $e^{i\varphi}$	keep magnitudes $ F(\mu) $ (or normalize the phase)
Start point	phase $e^{2\pi i \mu m/N}$	keep magnitudes $ F(\mu) $

Table 47: Normalizing a Fourier descriptor. Each step removes one nuisance transformation; mirroring and noise are not removed by any of them.

The normalization $\tilde{F}(0) := 0$, $\tilde{F}(\mu) := F(\mu)/|F(2)|$ therefore makes the descriptor invariant to translation (first step) and to scale (second step, with $k = 2$). It is not invariant to rotation, because the phases are kept. It is not invariant to mirroring, since mirroring conjugates u and exchanges positive and negative frequencies. Noise is never removed by normalization. Plots of normalized descriptors can be recognized by the same rules: the zero-frequency value is zero after translation normalization, the reference coefficient has magnitude 1 after scale normalization, and all values are real and non-negative after taking magnitudes.

Smoothing a shape. Fine detail and noise of the contour live in the high frequencies, which are the indices in the middle of the stored array (large $|\mu|$). A shape is smoothed by a low-pass operation: set those coefficients to zero or attenuate them, keep the low frequencies at both ends of the array, and apply the inverse DFT. Keeping only magnitudes destroys the shape, because the phases carry the positions of the points. Dividing the whole spectrum by a constant only rescales the shape and does not smooth it.

Limitations. Fourier descriptors need a clean segmentation and one closed, connected contour. They are global: a partial occlusion, or two touching objects, changes the whole contour and therefore every coefficient. The number of contour points and the sampling must be consistent across shapes to compare descriptors.

C Classical Classifiers and Part-Based Detectors

The classification stage of the processing chain in Figure 6 can be filled by many learners. Part-based detectors build on them. This appendix covers the classifiers and two detectors that exercises ask about beyond the implicit shape model: the Hough forest and the deformable part model. For each, the aim is to know its working principle and what exactly it outputs.

Linear classifiers and SVMs. A linear classifier decides by the sign of an affine function, $g(x) = w^\top x + b$; its decision boundary is a hyperplane. For linearly separable data many hyperplanes classify the training set correctly. A *linear support vector machine* (SVM) picks the one with the *maximum margin*: the hyperplane whose distance to the nearest training samples of both classes is largest, running through the middle of the gap. Only these nearest samples, the support vectors, determine it. A soft-margin variant tolerates some misclassified samples, and kernels allow nonlinear boundaries. In a figure with several separating lines, the SVM line is the one that keeps equal and largest distance to both classes, not one that touches a sample.

Random forests. A decision tree routes a sample through binary node tests (for example “is feature j larger than τ ?”) to a leaf that stores a class distribution. During training, each node picks, from a set of randomly generated candidate tests, the one that best reduces the class entropy of the samples it splits (information gain). A random forest trains many trees, each on a random subset of the data and with random candidate tests, and averages the leaf distributions of all trees. The randomness makes the trees differ, so their average has lower variance than any single tree.

Hough forests. A Hough forest is a random forest whose trees vote like the implicit shape model. Training samples are image patches, each labeled as object or background and, for object patches, with the offset vector to the object center. The node tests are simple and fast: compare the values of one feature channel (intensity, gradient, and so on) at two pixel positions inside the patch, $f(p) < f(q) + \tau$. Each node is optimized for one of two energies, chosen at random: the *class-label uncertainty* (entropy of object versus background) or the *offset uncertainty* (variance of the offset vectors of the object patches). Each leaf stores the fraction of object patches that reached it (a class probability) and the list of their offset vectors. At detection time every patch of the image descends all trees, and each leaf casts its offsets as votes for the object center, weighted by its class probability. The codebook and its matching step are thus replaced by a learned mapping from appearance to votes.

Deformable part models. A deformable part model (DPM) describes an object by a coarse *root* filter on HOG features, several *part* filters at twice the resolution, and a quadratic deformation cost for each part’s displacement from its anchor position. The score of a hypothesis with root position p_0 is

$$\text{score}(p_0) = F_0 \cdot \phi(p_0) + \sum_{i=1}^n \max_{p_i} \left[F_i \cdot \phi(p_i) - d_i \cdot (dx_i, dy_i, dx_i^2, dy_i^2) \right] + b, \quad (144)$$

where $\phi(p)$ is the HOG feature window at position p , F_i are the filter weights, and (dx_i, dy_i) is part i ’s displacement from its anchor. The part positions are latent variables, and the filters

are trained with a latent SVM. Localization is a sliding-window search: the score is evaluated at every position and scale of a HOG feature pyramid, and the high-scoring windows, after non-maxima suppression, are the detections. The SVM only outputs a score for a given window, not a location. Objects are found by classifying windows at many locations, not by a codebook and Hough voting.

D Density Models and Calibration

Section 4 estimates class-conditional densities with histograms, Parzen windows, and single Gaussians, and then turns them into posteriors. Exercises go three small steps further: they repair empty histogram bins, fit densities with several modes, and check whether the resulting probabilities can be trusted. Each step needs only a few formulas.

Laplace correction. A histogram estimated from few samples has empty bins, and an empty bin gives probability zero. One zero factor makes a whole naive-Bayes product zero, however strongly the other features speak for the class. The Laplace correction adds a pseudo-count λ (usually $\lambda = 1$) to every one of the K bins before normalizing:

$$\hat{P}(\text{bin } i) = \frac{n_i + \lambda}{n + \lambda K}, \quad (145)$$

where n_i is the count of bin i and n the total number of samples. The denominator grows by λK so that the corrected values still sum to one. For example, with $n = 12$ samples on a 3×3 grid ($K = 9$) and $n_i = 2$ samples in the queried cell, $\hat{P} = (2 + 1)/(12 + 9) = 1/7$; an empty cell gets $1/21$ instead of 0. For a density, divide by the cell area; on unit cells the value is the same. Count carefully which cells the grid really has, including empty ones, because K enters the denominator.

Gaussian mixture models. A single Gaussian cannot describe a class with several subgroups, for example benign lesions of different colors. A Gaussian mixture model (GMM) writes the density as a weighted sum of K components,

$$p(x | \theta) = \sum_{m=1}^K w_m \mathcal{N}(x; \mu_m, \Sigma_m), \quad \sum_{m=1}^K w_m = 1, \quad w_m \geq 0. \quad (146)$$

It is a parametric model, but its ML estimate has no closed form, because we do not know which component produced which sample. The *expectation-maximization* (EM) algorithm alternates two steps. The E-step assigns every sample soft memberships, the *responsibilities*

$$r_{im} = \frac{w_m \mathcal{N}(x_i; \mu_m, \Sigma_m)}{\sum_{l=1}^K w_l \mathcal{N}(x_i; \mu_l, \Sigma_l)}. \quad (147)$$

The M-step re-estimates each component as a weighted ML fit, with $N_m = \sum_i r_{im}$:

$$w_m = \frac{N_m}{n}, \quad \mu_m = \frac{1}{N_m} \sum_i r_{im} x_i, \quad \Sigma_m = \frac{1}{N_m} \sum_i r_{im} (x_i - \mu_m)(x_i - \mu_m)^\top. \quad (148)$$

The two steps repeat until the likelihood stops improving. The likelihood never decreases, but EM can end in a local maximum, so the initialization matters. On the five points $\{1.0, 1.2, 1.4, 4.1, 4.4\}$ with $K = 2$ and starting means near 1 and 4, the responsibilities become sharp after one or two iterations, the means move to the cluster centers 1.2 and 4.25, and the weights approach $3/5$ and $2/5$.

Choosing K with BIC. More components always fit the training data at least as well, so the likelihood alone would choose an overfitted model. The Bayesian information criterion adds a penalty for the number of parameters k :

$$\text{BIC} = k \ln n - 2 \ln \hat{L}, \quad (149)$$

where $\hat{L}$ is the maximized likelihood and n the number of samples. Lower BIC is better. A one-dimensional GMM has $k = 3K - 1$ parameters (K means, K variances, $K - 1$ free weights). Compare values of K only on the same features and data split.

Calibration. A posterior is only useful for risk-based decisions if its values mean what they say. A classifier is *calibrated* if, among all cases where it predicts confidence p , a fraction p is actually positive: of 100 cases predicted at 0.8, about 80 should be positive. Accuracy and calibration are different properties, and deep networks are often accurate but overconfident. A *reliability diagram* sorts predictions into confidence bins (for example 15) and plots the empirical accuracy of each bin against its mean confidence; the diagonal is perfect calibration, and bars below it indicate overconfidence. The *expected calibration error* summarizes the diagram as the weighted mean gap,

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|. \quad (150)$$

Temperature scaling is the simplest fix for a network with logits z : replace the softmax by $\hat{p}_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$ with one scalar T . A value $T > 1$ makes the predictions less extreme without changing their ranking, so the accuracy stays the same. T is fitted on validation data, never on the test set.

E Estimators, Loss Gradients, and Approximation

Three short facts about learning from data appear in exam questions without a lecture of their own: how to judge an estimator, how the choice of regression loss changes the gradient, and what a neural network can represent in principle.

Bias, variance, and MSE of an estimator. An estimator is a rule that turns a random dataset into a number, so the estimate is itself a random variable. Its *bias* is how far it is off on average, its *variance* is how much it scatters from dataset to dataset, and its mean squared error combines both:

$$\text{MSE}(\hat{\mu}) = \mathbb{E}[(\hat{\mu} - \mu)^2] = (\mathbb{E}[\hat{\mu}] - \mu)^2 + \text{Var}(\hat{\mu}). \quad (151)$$

For N samples $X_i \sim \mathcal{N}(\mu, \sigma^2)$, compare three estimators of the mean:

Estimator	Bias	Variance	MSE
$\hat{\mu}_A = 0$ (ignore the data)	$-\mu$	0	μ^2
$\hat{\mu}_B = \frac{1}{N} \sum_i X_i$ (sample mean)	0	σ^2/N	σ^2/N
$\hat{\mu}_C = X_1$ (first sample only)	0	σ^2	σ^2

A constant has zero variance, the lowest possible, and a single sample the highest. The sample mean has the lowest MSE only as long as $\sigma^2/N < \mu^2$: if the true mean happens to be close to 0, the useless constant $\hat{\mu}_A$ wins. So no estimator is “always” best in MSE; an unbiased one is merely the safe default. When two estimators are compared, the sum decides, not bias or variance alone: bias 1 with variance 5 gives $\text{MSE} = 6$, bias 2 with variance 1 gives $\text{MSE} = 5$, so the more biased estimator is better. Only an estimator that is at least as good in both $|\text{bias}|$ and variance wins without computing.

Squared versus absolute error. For a prediction y and target y_{GT} with error $e = y - y_{GT}$, the squared error $\ell_1 = e^2$ and the absolute error $\ell_2 = |e|$ have gradients of size

$$\left| \frac{\partial \ell_1}{\partial y} \right| = 2|e|, \quad \left| \frac{\partial \ell_2}{\partial y} \right| = 1 \quad (e \neq 0). \quad (152)$$

The squared error pushes harder on large errors and more gently on small ones; the two gradients are equal exactly at $|e| = 0.5$. This is why the squared error is sensitive to outliers, while the absolute error keeps a constant pull even when the prediction is almost right. With the common form $\ell_1 = \frac{1}{2}e^2$ the gradient is $|e|$, and the two gradients are equal at $|e| = 1$ instead.

y	y_{GT}	$ e $	$ \partial_y \ell_1 $ vs. $ \partial_y \ell_2 $	
5	10	5	$10 > 1$	squared larger
6.5	7	0.5	$1 = 1$	equal
1.2	1	0.2	$0.4 < 1$	absolute larger
0.1	0	0.1	$0.2 < 1$	absolute larger

Universal approximation. A network with a single hidden layer, a nonlinear activation, and enough units can approximate *any continuous function on a compact (closed and bounded) domain* to any desired accuracy. Continuity is required, differentiability is not: $|x|$ or a triangle wave are reproduced exactly by ReLU units, since a ReLU network is itself piecewise linear. The theorem does not cover arbitrary functions (discontinuous ones, or unbounded domains), and it is a pure existence statement: it says neither how many units are needed nor how to find the weights. Finding them remains the job of gradient descent.

F Graphical Models

A graphical model draws the dependency structure of a joint distribution as a graph: nodes are random variables, and edges show which variables interact directly. Missing edges encode independence, and they are what makes a high-dimensional distribution tractable. Graphical models solve tasks such as denoising, segmentation, stereo, and in general inferring hidden variables from observed ones. The Markov random fields of Section 4 are the undirected kind; exam questions also ask about the directed kind and about how graphs and factorizations correspond.

Conditional independence. Two variables a and b are conditionally independent given c , written $a \perp b \mid c$, if $p(a, b \mid c) = p(a \mid c)p(b \mid c)$, equivalently $p(a \mid b, c) = p(a \mid c)$: once c is known, b carries no further information about a . Naive Bayes is exactly this assumption for the features given the class, and the Markov property says that a pixel is conditionally independent of the rest of the field given its neighbors.

Bayesian networks. A Bayesian network is a *directed acyclic graph* (DAG): edges have directions, and no directed path leads from a node back to itself. Each node x_i has parents $\text{pa}(x_i)$, the nodes with an edge pointing to it, and the joint distribution factorizes into one conditional per node,

$$p(x_1, \dots, x_N) = \prod_{i=1}^N p(x_i \mid \text{pa}(x_i)). \quad (153)$$

Converting in both directions is mechanical (Figure 54). From graph to factorization, write one factor per node, conditioned on exactly its parents; a node without parents gets its marginal. From factorization to graph, draw one node per variable and, for each factor $p(x_i \mid \cdot)$, an arrow from every conditioning variable to x_i .

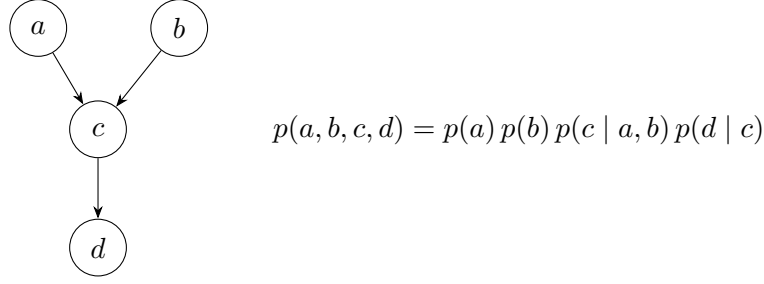


Figure 54: A Bayesian network and its factorization. Each node contributes one factor conditioned on its parents; a and b have no parents and contribute their marginals.

MRF factorization. An MRF is an undirected graph, so there are no parents and no conditionals. Instead the joint distribution factorizes over the *maximal cliques* C of the graph, the cliques that cannot be enlarged by another node:

$$p(x) = \frac{1}{Z} \prod_C \psi_C(x_C), \quad \psi_C(x_C) = \exp\{-E_C(x_C)\}, \quad Z = \sum_x \prod_C \psi_C(x_C). \quad (154)$$

The potential functions $\psi_C \geq 0$ are not probabilities, which is why the normalizer Z is needed. Writing them as exponentials of energies gives back the Gibbs form of Equation (66): the total energy is the sum of the clique energies, and potential and energy are related by $\psi = \exp(-E)$. On a 4-connected pixel grid the maximal cliques are the horizontal and vertical pixel pairs, and the energies split into unary terms on single nodes and pairwise terms on pairs.

MRF versus CRF. An MRF for labeling is generative: it models the prior $p(x)$ of the labels and the likelihood $p(y \mid x)$ of the observations, and the posterior follows with Bayes' theorem. A *conditional random field* (CRF) models the posterior $p(x \mid y)$ directly, as a Gibbs distribution whose potentials may depend on the whole observed image. A typical example is a contrast-sensitive pairwise term that penalizes a label change less where the image has a strong edge. A CRF is therefore discriminative: it never models how the image was generated, only which labelings are plausible given it.

G Foundation Models and DINOv2

A foundation model learns a reusable representation from broad data and is then adapted to several tasks. The architecture and the training objective are separate choices: a Vision Transformer describes how image patches interact; pretraining determines which information its features retain. Supervised pretraining uses class labels, self-supervised pretraining constructs targets from the images themselves, and image–text training can use paired captions as weaker supervision. On a new task, one can fine-tune the model or freeze it and train a small head. A frozen backbone still runs at inference; freezing only saves training updates.

DINOv2 is an image-only, self-supervised ViT foundation model. Its training uses different views of the same image and makes a student network agree with a teacher network, rather than requiring human class labels or image captions. The resulting class-token and patch features can be reused for image classification, retrieval, segmentation, and depth estimation. A linear probe tests a frozen representation with a trained linear head; dense tasks instead use spatial patch features. DINOv2 differs from a supervised ViT in its training signal, and from CLIP-style image–text models in that it does not learn from caption alignment. These distinctions explain how one pretrained image encoder can support both whole-image and pixel-level tasks.

H Contrastive and Energy-Based Self-Supervision

The pretext tasks of Section 8 reconstruct pixels or predict an arrangement. A second family learns representations by *comparing* images. Its central idea is that two views of the same image should receive similar representations, while views of different images should not. The views are created by *data augmentation*: random crops, flips, color jitter, blur.

Contrastive learning. An encoder maps two augmented views of image n to normalized embeddings u_n and v_n . The pair (u_n, v_n) is *positive*; the views of other images in the batch act as *negatives*. With similarity $s(u, v) = u^\top v / \tau$ and temperature τ , the InfoNCE loss

$$L = -\frac{1}{N} \sum_{n=1}^N \log \frac{\exp s(u_n, v_n)}{\sum_{m=1}^N \exp s(u_n, v_m)} \quad (155)$$

is a softmax classification: among N candidates, find the view that came from the same image. Without augmentation there would be no positive pair to compare, so data augmentation is what creates the training signal and hence the gradient. The augmentations also decide which properties the representation ignores; with color jitter, the embedding learns to disregard color. SimCLR and MoCo are examples; CLIP applies the same loss to image–text pairs.

Energy-based view. An energy-based model assigns a scalar energy $E(x, x')$ to a pair of inputs: low energy for compatible pairs, high energy for incompatible ones. In self-supervised learning, a compatible pair is two views of the same content, for example two augmentations or two video frames, and the energy is usually a distance between their embeddings. Training lowers the energy of compatible pairs. It must also keep the energy of other pairs high, either with negatives as in contrastive learning or with regularization of the embedding statistics. Without this, all inputs collapse onto one embedding with zero energy. The goal is *not* to map images with small mean-square pixel distance to the same region. Pixel distance is a poor semantic measure: a shifted or recolored image of the same object is far away in pixel space, yet should be close in the representation, while two uniform gray images of different scenes are close in pixel space.